

# CS100 Lecture 2

Variables I and Arithmetic Types

## Contents

- Variable declaration
- Arithmetic types
  - Bits and bytes
  - Integer types
  - Real floating types
  - Character types
  - Boolean type

## Variable declaration

### Type of a variable

Every variable in C has a type.

- The type is **fully deterministic** and **cannot be changed**.
- The type is **known even when the program is not run**.
  - The type is known at **compile-time**.
  - C is **statically-typed**. C has a **static type system**.
  - In contrast, Python is **dynamically-typed**.

### Statically-typed vs dynamically-typed

Python: dynamically typed

```
a = 42          # Type of a is int.
a = "hello"     # Type of a becomes str.
```

C: statically-typed

```
int a = 42;    // Type of a is int.
a = "hello";   // Error! Types mismatch!
```

The type of a variable

- is explicitly written on declaration, and is known at compile-time, and cannot be changed

A type-related error in C is (*usually*) a **compile error**:

- It stops the compiler. The executable will not be generated.

## Declare a variable

To declare a variable, we need to specify its **type** and **name**.

```
Type name;
```

We may declare multiple variables of a same type in one declaration statement, separated by `,`:

```
int x, y; // Declares two variables 'x' and 'y', both having type 'int'.
```

A **variable declaration** can be placed

- inside a function, which declares a **local variable**, or
- outside of any functions, which declares a **global variable**.

## Readability matters

**[Best practice]** Declare the variable when you first use it!

- If the declaration and use of the variable are too separated, it will become much more difficult to figure out what they are used for as the program goes longer.

**[Best practice]** Use meaningful names!

- The program would be a mess if polluted with names like `a`, `b`, `c`, `d`, `x`, `y`, `cnt`, `cnt_2`, `flag1`, `flag2`, `flag3` everywhere.
- Use meaningful names: `sumOfScore`, `student_cnt`, `open_success`, ...

**Readability is very important.** Many students debug day and night simply because their programs are not human-readable.

## Use of global variables

One reason for using global variables is to have them shared between functions:

```
void work(void) {
    // Error: `input` was not declared
    // in this scope.
    printf("%d\n", input);
}

int main(void) {
    int input;
    scanf("%d", &input);
    work();
}
```

## Initialize a variable

A variable can be **initialized** on declaration.

```
int x = 42; // Declares the variable `x` of type `int`,
           // and initializes its value to 42.
int a = 0, b, c = 42; // Declares three `int` variables, with `a` initialized
                     // to 0, `c` initialized to 42, and `b` uninitialized.
```

This is syntactically **different** (though seems equivalent) to

```
int x; // Declares `x`, uninitialized.
x = 42; // Assigns 42 to `x`.
```

**[Best practice]** Initialize the variable if possible. Prefer initialization to later assignment.

## Arithmetic types

### Integer types

Is `int` equivalent to 正整数?

- Is there a limitation on the numbers that `int` can represent?

Experiment:

```
#include <stdio.h>

int main(void) {
    int x = 1;
    while (1) {
        printf("%d\n", x);
        x *= 2; // x = x * 2
        getchar();
    }
}
```

- 1073741824  
-2147483648  
0  
0

## Bits and bytes

Information is stored in computers **in binary**.

- $42_{10} = 101010_{2}$ .

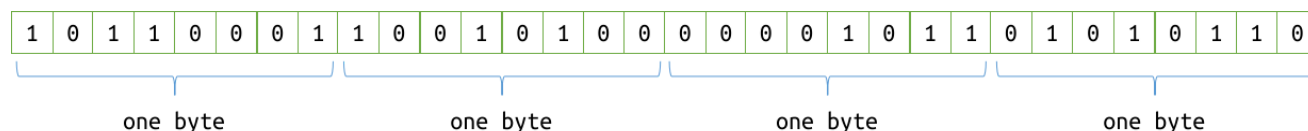
A **bit** is either \$0\$ or \$1\$.

- The binary representation of \$42\$ consists of \$6\$ bits.

A **byte** is \$8\$ bits  $\{0, 1\}^8$  grouped together like \$10001001\$.

- At least \$1\$ byte is needed to store \$42\$.
- At least \$3\$ bytes are needed to store \$142857\_{10} = 100010111000001001\_{2}\$

A 32-bit number: \$2979269462\_{\text{ten}}=10110001100101000000101101010110\_{\text{two}}\$.



## Integer types

An integer type in C is either **signed** or **unsigned**, and has a **width** denoting the number of bits that can be used to represent values.

Suppose we have an integer type of  $n$  bits in width.

- If the type is **signed**  $\textcolor{red}{3}$ , the range of values that can be represented is  $\left[-2^{n-1}, 2^{n-1}-1\right]$ .
- If the type is **unsigned**, the range of values that can be represented is  $\left[0, 2^n-1\right]$ .
- The keyword `int` is optional in types other than `int`:
  - e.g. `short int` and `short` name the same type.
  - e.g. `unsigned int` and `unsigned` name the same type.
- "Unsigned-ness" needs to be written explicitly: `unsigned int`, `unsigned long`, ...
- Types without the keyword `unsigned` are signed by default:
  - e.g. `signed int` and `int` name the same type.
  - e.g. `signed long int`, `signed long`, `long int` and `long` name the same type.

## Width of integer types

| type                   | width (at least) | width (usually) |
|------------------------|------------------|-----------------|
| <code>short</code>     | 16 bits          | 16 bits         |
| <code>int</code>       | 16 bits          | 32 bits         |
| <code>long</code>      | 32 bits          | 32 or 64 bits   |
| <code>long long</code> | 64 bits          | 64 bits         |
|                        |                  |                 |

- A signed type has the same width as its `unsigned` counterpart.
- It is also guaranteed that `sizeof(short)  $\leq$  sizeof(int)  $\leq$  sizeof(long)  $\leq$  sizeof(long long)`.
  - `sizeof(T)` is the number of **bytes** that `T` holds.

## Implementation-defined behaviors

The standard states that the exact width of the integer types is **implementation-defined**.

- **Implementation:** The compiler and the standard library.
- An implementation-defined behavior depends on the compiler and the standard library, and is often also related to the hosted environment (e.g. the operating system).

## Which one should I use?

`int` is the most optimal integer type for the platform.

- Use `int` for integer arithmetic by default.
- Use `long long` if the range of `int` is not large enough.
- Use smaller types (`short`, or even `unsigned char`) for memory-saving or other special purposes.
- Use `unsigned` types for special purposes. We will see some in later lectures.

## Real floating types

"Floating-point": The number's radix point can "float" anywhere to the left, right, or between the significant digits of the number.

Real floating-point types can be used to represent *some* real values.

- Real floating-point types  $\neq \mathbb{R}$ .

## Which one should I use?

Use `double` for real floating-point arithmetic by default.

- In some cases the precision of `float` is not enough.
- Don't worry about efficiency! `double` arithmetic is not necessarily slower than `float`.

**Do not use floating-point types for integer arithmetic!**

## scanf/printf

| type      | format specifier |
|-----------|------------------|
| short     | %hd              |
| int       | %d               |
| long      | %ld              |
| long long | %lld             |
|           |                  |

| type               | format specifier |
|--------------------|------------------|
| unsigned short     | %hu              |
| unsigned           | %u               |
| unsigned long      | %lu              |
| unsigned long long | %llu             |
|                    |                  |
|                    |                  |

- %f for float, %lf for double, and %Lf for long double.

## Character types

The C standard provides three **different** character types: `signed char`, `unsigned char` and `char`.

Let `T`  $\in$  {`signed char`, `unsigned char`, `char`}. It is guaranteed that

$1 == \text{sizeof}(T) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$ .

- `T` takes exactly 1 byte.

Question: What is the valid range of `signed char`? `unsigned char`?

- `signed char`:  $[-128, 127]$ .
- `unsigned char`:  $[0, 255]$ .

What? A character is an integer?

## ASCII (American Standard Code for Information Interchange)

| Decimal | Hex | Char                   | Decimal | Hex | Char    | Decimal | Hex | Char | Decimal | Hex | Char  |
|---------|-----|------------------------|---------|-----|---------|---------|-----|------|---------|-----|-------|
| 0       | 0   | [NULL]                 | 32      | 20  | [SPACE] | 64      | 40  | @    | 96      | 60  | `     |
| 1       | 1   | [START OF HEADING]     | 33      | 21  | !       | 65      | 41  | A    | 97      | 61  | a     |
| 2       | 2   | [START OF TEXT]        | 34      | 22  | "       | 66      | 42  | B    | 98      | 62  | b     |
| 3       | 3   | [END OF TEXT]          | 35      | 23  | #       | 67      | 43  | C    | 99      | 63  | c     |
| 4       | 4   | [END OF TRANSMISSION]  | 36      | 24  | \$      | 68      | 44  | D    | 100     | 64  | d     |
| 5       | 5   | [ENQUIRY]              | 37      | 25  | %       | 69      | 45  | E    | 101     | 65  | e     |
| 6       | 6   | [ACKNOWLEDGE]          | 38      | 26  | &       | 70      | 46  | F    | 102     | 66  | f     |
| 7       | 7   | [BELL]                 | 39      | 27  | '       | 71      | 47  | G    | 103     | 67  | g     |
| 8       | 8   | [BACKSPACE]            | 40      | 28  | (       | 72      | 48  | H    | 104     | 68  | h     |
| 9       | 9   | [HORIZONTAL TAB]       | 41      | 29  | )       | 73      | 49  | I    | 105     | 69  | i     |
| 10      | A   | [LINE FEED]            | 42      | 2A  | *       | 74      | 4A  | J    | 106     | 6A  | j     |
| 11      | B   | [VERTICAL TAB]         | 43      | 2B  | +       | 75      | 4B  | K    | 107     | 6B  | k     |
| 12      | C   | [FORM FEED]            | 44      | 2C  | ,       | 76      | 4C  | L    | 108     | 6C  | l     |
| 13      | D   | [CARRIAGE RETURN]      | 45      | 2D  | -       | 77      | 4D  | M    | 109     | 6D  | m     |
| 14      | E   | [SHIFT OUT]            | 46      | 2E  | .       | 78      | 4E  | N    | 110     | 6E  | n     |
| 15      | F   | [SHIFT IN]             | 47      | 2F  | /       | 79      | 4F  | O    | 111     | 6F  | o     |
| 16      | 10  | [DATA LINK ESCAPE]     | 48      | 30  | 0       | 80      | 50  | P    | 112     | 70  | p     |
| 17      | 11  | [DEVICE CONTROL 1]     | 49      | 31  | 1       | 81      | 51  | Q    | 113     | 71  | q     |
| 18      | 12  | [DEVICE CONTROL 2]     | 50      | 32  | 2       | 82      | 52  | R    | 114     | 72  | r     |
| 19      | 13  | [DEVICE CONTROL 3]     | 51      | 33  | 3       | 83      | 53  | S    | 115     | 73  | s     |
| 20      | 14  | [DEVICE CONTROL 4]     | 52      | 34  | 4       | 84      | 54  | T    | 116     | 74  | t     |
| 21      | 15  | [NEGATIVE ACKNOWLEDGE] | 53      | 35  | 5       | 85      | 55  | U    | 117     | 75  | u     |
| 22      | 16  | [SYNCHRONOUS IDLE]     | 54      | 36  | 6       | 86      | 56  | V    | 118     | 76  | v     |
| 23      | 17  | [ENG OF TRANS. BLOCK]  | 55      | 37  | 7       | 87      | 57  | W    | 119     | 77  | w     |
| 24      | 18  | [CANCEL]               | 56      | 38  | 8       | 88      | 58  | X    | 120     | 78  | x     |
| 25      | 19  | [END OF MEDIUM]        | 57      | 39  | 9       | 89      | 59  | Y    | 121     | 79  | y     |
| 26      | 1A  | [SUBSTITUTE]           | 58      | 3A  | :       | 90      | 5A  | Z    | 122     | 7A  | z     |
| 27      | 1B  | [ESCAPE]               | 59      | 3B  | ;       | 91      | 5B  | [    | 123     | 7B  | {     |
| 28      | 1C  | [FILE SEPARATOR]       | 60      | 3C  | <       | 92      | 5C  | \    | 124     | 7C  |       |
| 29      | 1D  | [GROUP SEPARATOR]      | 61      | 3D  | =       | 93      | 5D  | ]    | 125     | 7D  | }     |
| 30      | 1E  | [RECORD SEPARATOR]     | 62      | 3E  | >       | 94      | 5E  | ^    | 126     | 7E  | ~     |
| 31      | 1F  | [UNIT SEPARATOR]       | 63      | 3F  | ?       | 95      | 5F  | _    | 127     | 7F  | [DEL] |

Important things to remember:

- `[$ '0' '$ '9' $]=[48, 57]$`.
- `[$ 'A' '$ 'Z' $]=[65, 90]$`.
- `[$ 'a' '$ 'z' $]=[97, 122]$`.

## [Best practice] Avoid magic numbers

What is the meaning of `32` here? `$\rightarrow` a magic number.

```
char to_uppercase(char x) {
    return x - 32;
}
```

Write it in a more human-readable way:

```
char to_uppercase(char x) {
    return x - ('a' - 'A');
}
```

## Escape sequence

Some special characters are not directly representable: newline, tab, quote, ...

| escape sequence | description  |
|-----------------|--------------|
| <code>\'</code> | single quote |
| <code>\"</code> | double quote |
| <code>\\</code> | backslash    |
|                 |              |

| escape sequence | description     |
|-----------------|-----------------|
| <code>\n</code> | newline         |
| <code>\r</code> | carriage return |
| <code>\t</code> | horizontal tab  |
|                 |                 |
|                 |                 |

## Character types

`char`, `signed char` and `unsigned char` are **three different types**.

- Whether `char` is signed or unsigned is **implementation-defined**.
- If `char` is signed (unsigned), it represents the same set of values as the type `signed char` (`unsigned char`), but **they are not the same type**.
  - In contrast, `T` and `signed T` are the same type for `T` `\in{$short, int, long, long long}$`.

For almost all cases, use `char` (or, sometimes `int`) to represent characters.

`signed char` and `unsigned char` are used for other purposes.

To read/print a `char` using `scanf` / `printf`, use `%c`.

## Boolean type: `bool` (since C99)

A type that represents true/false, 0/1, yes/no, ...

To access the name `bool`, `true` and `false`, `<stdbool.h>` is needed. (until C23)

Example: Define a function that accepts a character and returns whether that character is a lowercase letter.

Before C99, using `int`, `0` and `1`:

```
int is_lowercase(char c) {
    if (c >= 'a' && c <= 'z')
        return 1;
    else
        return 0;
}
```

Since C99, using `bool`, `false` and `true`:

```
bool is_lowercase(char c) {
    if (c >= 'a' && c <= 'z')
        return true;
    else
        return false;
}
```

Before C99, using `int`, `0` and `1`:

```
int is_lowercase(char c) {
    if (c >= 'a' && c <= 'z')
        return 1;
    else
        return 0;
}
```

Since C99, using `bool`, `false` and `true`:

```
bool is_lowercase(char c) {
    if (c >= 'a' && c <= 'z')
        return true;
    else
        return false;
}
```

Both return values can be used as follows:

```
char c; scanf("%c", &c);
if (is_lowercase(c)) {
    // do something when c is lowercase ...
}
```

## CS100 Lecture 3

Operators and Control Flow I

### Contents

- Operators
  - `+`, `-`, `*`, `/`, `%`
  - Compound assignment operators
  - Signed integer overflow
  - `++` and `--`
- Control flow
  - `if-else`
  - `while`
  - `for`

## Operators

### The calculator

Accept input of the form `x op y`, where `x` and `y` are floating-point numbers and `op` `$\in\{` `'+'`, `'-'`, `'*'`, `'/'` `$\}`. Print the result.

```
#include <stdio.h>

int main(void) {
    double x, y;
    char op;
    scanf("%lf %c %lf", &x, &op, &y);
    if (op == '+')
        printf("%lf\n", x + y);
    else if (op == '-')
        printf("%lf\n", x - y);
```

```
    else if (op == '*')
        printf("%lf\n", x * y);
    else if (op == '/')
        printf("%lf\n", x / y);
    else
        printf("Invalid operator.\n");
    return 0;
}
```

`+`, `-`, `*`, `/`, `%`

- `+` and `-` have two versions: unary (`+a`, `-a`) and binary (`a+b`, `a-b`).
  - The unary `+/ -` and binary `+/ -` are **different operators**, although they use the same notation.
- Operator precedence:
 

``${$} unary +, unary - `${$} ${$} * , / , % `${$} ${$} binary +, binary - `${$}`

e.g. `a + b * c` is interpreted as `a + (b * c)`, instead of `(a + b) * c`.

``${$} \rightarrow `${$}` We will talk more about operator precedence later.

### Binary `+`, `-` and `*`, `/`

`a + b`, `a - b`, `a * b`, `a / b`

Before the evaluation of such an expression, the operands (`a`, `b`) undergo a sequence of **type conversions**.

- The [detailed rules of the conversions](#) are very complex,
  - including *promotions*, conversions between `signed` and `unsigned` types, conversions between integers and floating-point types, etc.
  - We only need to remember some common ones.
- In the end, the operands will be converted to **a same type**, denoted `T`. **The result type is also `T`**.

`a + b`, `a - b`, `a * b`, `a / b`

If any one operand is of floating-point type and the other is an integer, **the integer will be implicitly converted to that floating-point type**.

Example:

```
double pi = 3.14;
int diameter = 20;
whatType c = pi * diameter; // what is the type of this result?
```

`a + b`, `a - b`, `a * b`, `a / b`

If any one operand is of floating-point type and the other is an integer, **the integer will be implicitly converted to that floating-point type**.

Example:

```
double pi = 3.14;
int diameter = 20;
double c = pi * diameter; // 62.8
```

The value of `diameter` is implicitly converted to a value of type `double`. Then, a floating-point multiplication is performed, yielding a result of type `double`.

\* Does this rule make sense? - Yes, because  $\mathbb{Z} \subseteq \mathbb{R}$ .

`a + b`, `a - b`, `a * b`, `a / b`

If any one operand is of floating-point type and the other is an integer, **the integer will be implicitly converted to that floating-point type**, and the result type is that floating-point type.

Similarly, if the operands are of types `int` and `long long`, the `int` value will be implicitly converted to `long long`, and the result type is `long long`.

## Division: `a / b`

Assume `a` and `b` are of the same type `T` (after conversions as mentioned above).

- Then, the result type is also `T`.

Two cases:

- If `T` is a floating-point type, this is a floating-point division.
  - The result is no surprising.
- If `T` is an integer type, this is an integer division.
  - The result is **truncated towards zero** (since C99 and C++11)  $\textcolor{red}{2}$ .
  - What is the result of `3 / -2`?

Let `a` and `b` be two integers.

- What is the difference between `a / 2` and `a / 2.0`?
- What does `(a + 0.0) / b` mean? What about `1.0 * a / b`?

If `T` is an integer type, this is an integer division.

- The result is **truncated towards zero** (since C99 and C++11)  $\textcolor{red}{2}$ .
- What is the result of `3 / -2`?
  - `-1.5` truncated towards zero, which is `-1`.

What is the difference between `a / 2` and `a / 2.0`?

- `a / 2` yields an integer, while `a / 2.0` yields a `double`.

What does `(a + 0.0) / b` mean? What about `1.0 * a / b`?

- Both use floating-point division to compute  $\frac{a}{b}$ . The floating-point numbers `0.0` and `1.0` here cause the conversion of the other operands.

## Remainder: `a % b`

Example: `15 % 4 == 3`.

`a` and `b` must have integer types.

If `a` is negative, is the result negative? What if `b` is negative? What if both are negative?

Example: `15 % 4 == 3`.

`a` and `b` must have integer types.

For any integers `a` and `b`, the following always holds:

```
(a / b) * b + (a % b) == a
```



## Compound assignment operators

`+=`, `-=`, `*=`, `/=`, `%=`

- `a op= b` is equivalent to `a = a op b`.
- e.g. `x *= 2` is equivalent to `x = x * 2`.
- [Best practice]** Learn to use these operators, to make your code clear and simple.

## Signed integer overflow

If a **signed integer type** holds a value that is not in the valid range, **overflow** is caused.

Suppose `int` is 32-bit and `long long` is 64-bit.

Do the following computations cause overflow?

```
int ival = 100000; long long llval = ival;
int result1 = ival * ival;           // (1) overflow
long long result2 = ival * ival;      // (2) overflow
long long result3 = llval * ival;     // (3) not overflow
long long result4 = llval * ival * ival; // (4) not overflow
```

(1)  $(10^5)^2 = 10^{10} > 2^{31} - 1$ .

(2) The result type of the multiplication `ival * ival` is `int`, which causes overflow. This is not affected by the type of `result2`.

(3) Since `llval` is of type `long long`, the value of `ival` will be implicitly converted to `long long`, and then the multiplication yields a `long long` value.

(4) `*` is **left-associative**, so the expression `a * b * c` is interpreted as `(a * b) * c`.

→ We will talk about associativity in later lectures.

## Undefined behavior

Signed integer overflow is : **There are no restrictions on the behavior of the program**. Compilers are not required to diagnose undefined behavior (although many simple situations are diagnosed), and the compiled program is not required to do anything meaningful.

- It may yield some garbage values, or zero, or anything else;
- or, this statement may be removed if the compiler is clever enough;
- or, the program may crash;
- or, any other results beyond imagination.

More on undefined behaviors in recitations.

Unsigned integer arithmetic is always performed *modulo*  $2^n$ , where  $n$  is the number of bits in that integer type.

For example, for `unsigned int` (assuming it is 32-bit)

- adding one to  $2^{32}-1$  gives  $0$  because  $2^{32} \equiv 0 \pmod{2^{32}}$ , and
- subtracting one from  $0$  gives  $2^{32}-1$  because  $-1 \equiv 2^{32}-1 \pmod{2^{32}}$ .

\* "wrap-around"

## Increment/decrement operators

Unary operators that increment/decrement the value of a variable by `1`.

Postfix form: `a++`, `a--`

Prefix form: `++a`, `--a`

- `a++` and `++a` increment the value of `a` by `1`.
- `a--` and `--a` decrement the value of `a` by `1`.

The result of the **postfix** increment/decrement operators is the value of `a` **before incrementation/decrementation**.

\* What does "result" mean?

Unary operators that increment/decrement the value of a variable by `1`.

Postfix form: `a++`, `a--`

The result of the **postfix** increment/decrement operators is the value of `a` **before incrementation/decrementation**.

```
int x = 42;
printf("%d\n", x++); // x becomes 43, but 42 is printed.
int y = x++; // y is initialized with 43. x becomes 44.
```

Unary operators that increment/decrement the value of a variable by `1`.

Prefix form: `++a`, `--a`

The result of the **prefix** increment/decrement operators is the value of `a` **after incrementation/decrementation**.

```
int x = 42;
printf("%d\n", ++x); // x becomes 43, and 43 is printed.
int y = ++x; // y is initialized with 44. x becomes 44.
```

## CS100 Lecture 4

Operators and Control Flow II, Functions

### Contents

- Operators
  - Operator precedence, associativity and evaluation order
  - Comparison operators `<`, `<=`, `>`, `>=`, `==`, `!=`
  - Logical operators `&&`, `||`, `!`
  - Conditional operator `?:`
  - Assignment operator `=`
- Control Flow
  - `do - while`
  - `switch - case`
- Functions

## Operators

### Operator precedence

[Operator precedence](#) defines the order in which operators are bound to their arguments.

Example: `*` and `/` have higher precedence than `+` and `-`, so `a + b * c` is interpreted as `a + (b * c)` instead of `(a + b) * c`.

**Operator precedence does not determine [evaluation order](#).**

- `f() + g() * h()` is interpreted as `f() + (g() * h())`, but the order in which `f`, `g` and `h` are called is **unspecified**.

### Associativity

Each operator is either **left-associative** or **right-associative**.

Operators with the same precedence have the same associativity.

Example: `+` and `-` are **left-associative**, so `a - b + c` is interpreted as `(a - b) + c`, instead of `a - (b + c)`.

**Associativity does not determine [evaluation order](#).**

- `f() - g() + h()` is interpreted as `(f() - g()) + h()`, but the order in which `f`, `g` and `h` are called is **unspecified**.

### Evaluation order

Unless otherwise stated, the order in which the operands are evaluated is **unspecified**.

- We will see that `&&`, `||` and `?:` (and also `,` in recitations) have specified evaluation order of their operands.

Examples: In the following expressions, it is **unspecified** whether `f` is called before `g`.

- `f() + g()`
- `f() == g()`
- `some_function(f(), g())` (Note that the `,` here is not the [comma operator](#).)
- ...

### Evaluation order and undefined behavior

Let `A` and `B` be two expressions. **The behavior is undefined if**

- the order in which `A` and `B` are evaluated is unspecified  $\textcolor{red}{1}$ , and
- both `A` and `B` modify an object, or one modifies an object and the other uses its value  $\textcolor{red}{2}$ .

Examples:

```
i = ++i + i++; // undefined behavior
i = i++ + 1;   // undefined behavior
printf("%d, %d\n", i, i++); // undefined behavior
```

Recall that **undefined behavior** means "everything is possible". We cannot make any assumptions about the behavior of the program.

## Terminology: Return type/value of an operator

When it comes to "the return type/value of an operator", we are actually viewing the operator as a function:

```
int operator_plus(int a, int b) {
    return a + b;
}
int operator_postfix_inc(int &x) { // we must use a C++ notation here.
    int old = x;
    x += 1;
    return old;
}
```

The "return value" of an operator is the value of the expression it forms.

The "return type" of an operator is the type of its return value.

## Comparison operators

Comparison operators are binary operators that test a condition and return `1` if that condition is logically **true** and `0` if it is logically **false**.

| Operator              | Operator name |
|-----------------------|---------------|
| <code>a == b</code>   | equal to      |
| <code>a != b</code>   | not equal to  |
| <code>a &lt; b</code> | less than     |

| Operator               | Operator name            |
|------------------------|--------------------------|
| <code>a &gt; b</code>  | greater than             |
| <code>a &lt;= b</code> | less than or equal to    |
| <code>a &gt;= b</code> | greater than or equal to |

For most cases, the operands `a` and `b` are also converted to a same type, just as what happens for `a + b`, `a - b`, ...

Note: Comparison operators in C **cannot be chained**.

Example: `a < b < c` is interpreted as `(a < b) < c` (due to left-associativity), which means to

- compare `(a < b)` first, whose result is either `0` or `1`, and then
- compare `0 < c` or `1 < c`.

To test `$a < b < c`, use `a < b && b < c`.

## Logical operators

Logical operators apply standard boolean algebra operations to their operands.

| Operator                | Operator name | Example                     |
|-------------------------|---------------|-----------------------------|
| <code>!</code>          | logical NOT   | <code>!a</code>             |
| <code>&amp;&amp;</code> | logical AND   | <code>a &amp;&amp; b</code> |
| <code>  </code>         | logical OR    | <code>a    b</code>         |
|                         |               |                             |

`!a`, `a && b`, `a || b`

Recall the boolean algebra:

| $A \wedge B$ | $A \vee B$ | $\neg A$ | $A \wedge \neg B$ | $A \vee \neg B$ |
|--------------|------------|----------|-------------------|-----------------|
| True         | True       | False    | True              | True            |
| True         | False      | False    | False             | True            |
| False        | True       | True     | False             | True            |
| False        | False      | True     | False             | False           |
|              |            |          |                   |                 |
|              |            |          |                   |                 |

For C logical operators:

| a    | b    | !a | a && b | a    b |
|------|------|----|--------|--------|
| != 0 | != 0 | 0  | 1      | 1      |
| != 0 | == 0 | 0  | 0      | 1      |
| == 0 | != 0 | 1  | 0      | 1      |
| == 0 | == 0 | 1  | 0      | 0      |
|      |      |    |        |        |
|      |      |    |        |        |
|      |      |    |        |        |

Precedence: `!` `>` `<` `>=` `<=` `&&` `>>` `&` `<` `||`.

Typical example: lexicographical comparison of two pairs  $(a_1, b_1)$  and  $(a_2, b_2)$

```
int less(int a1, int b1, int a2, int b2) {
    return a1 < a2 || (a1 == a2 && b1 < b2);
}
```

The parentheses are optional here, but it improves readability.

## Avoid abuse of parentheses

Too many parentheses **reduce** readability:

```
int less(int a1, int b1, int a2, int b2) {
    return (((a1) < (a2)) || (((a1) == (a2)) && ((b1) < (b2))));
    // Is this a1 < b1 || (a1 == b1 && a2 < b2)
    //      or (a1 < b1 || a1 == b1) && a2 < b2 ?
}
```

[Best practice] Use **one** pair of parentheses when two binary logical operators meet.

## Short-circuit evaluation

`a && b` and `a || b` perform **short-circuit evaluation**:

- For `a && b`, `a` is evaluated first. If `a` compares equal to zero (is logically **false**), `b` is not evaluated.
  - $\text{False} \wedge \text{False} \rightarrow \text{False}$
- For `a || b`, `a` is evaluated first. If `a` compares not equal to zero (is logically **true**), `b` is not evaluated.
  - $\text{True} \vee \text{True} \rightarrow \text{True}$

The evaluation order is specified!

## Conditional operator

Syntax: `condition ? expressionT : expressionF`,

where `condition` is an expression of scalar type.

The evaluation order is specified!

- First, `condition` is evaluated.

- If `condition` compares not equal to zero (is logically **true**), `expressionT` is evaluated, and the result is the value of `expressionT`.
- Otherwise (if `condition` compares equal to zero, which is logically **false**), `expressionF` is evaluated, and the result is the value of `expressionF`.

## Conditional operator `?:`

Syntax: `condition ? expressionT : expressionF`,

Example: `to_uppercase(c)` returns the uppercase form of `c` if `c` is a lowercase letter, or `c` itself if it is not.

```
char to_uppercase(char c) {
    if (c >= 'a' && c <= 'z')
        return c - ('a' - 'A');
    else
        return c;
}
```

Use `?:` to rewrite it:

```
char to_uppercase(char c) {
    return c >= 'a' && c <= 'z' ? c - ('a' - 'A') : c;
}
```

Syntax: `condition ? expressionT : expressionF`

Use it to replace some simple and short `if-else` statement.

**Avoid abusing it!** Nested conditional operators reduces readability significantly.

```
int result = a < b ? (a < c ? a : c) : (b < c ? b : c); // Um ...
```

**[Best practice]** Avoid more than two levels of nested conditional operators.

## Assignment operator `=`

`lhs = rhs`

The assignment operator **returns the value of** `lhs` **after assignment**.

Moreover, the assignment operator is **right-associative**, making it possible to write "chained" assignments:

```
int a = 0, b = 1, c = 2;
a = b = c; // interpreted as a = (b = c)
           // Both a and b are assigned with 2.
```

## Control Flow

### `do-while`

Syntax: `do loop_body while (condition);`

Executes `loop_body` repeatedly until the value of `condition` compares equal to zero (is logically **false**).

Example:

```
int i = 0;
do {
    printf("%d", i++);
} while (i < 5);
```

Output: `01234`

Note that in each iteration, the condition is tested **after** the body is executed.

```
int i = 0;
do {
    printf("%d", i++);
} while (i < n);
```

Even if `n == 0`, `0` is printed. The loop body is always executed at least once.

Rewrite a `do-while` loop using a `while` loop.

```
do {
    // loop_body
} while (condition);
```

Use `while (1)` and `break`:

```
while (1) {
    // loop_body
    if (!condition)
        break;
}
```

## switch-case

```
switch (expression) { ... }
```

```
switch (op) {
case '+':
    printf("%1f\n", a + b); break;
case '-':
    printf("%1f\n", a - b); break;
case '*':
    printf("%1f\n", a * b); break;
case '/':
    printf("%1f\n", a / b); break;
default:
    printf("Invalid operator!\n");
    break;
}
```

- First, `expression` is evaluated.
- Control finds the `case` label to which `expression` compares equal, and then goes to that label.
- Starting from the selected label, **all subsequent statements are executed until a `break`; or the end of the `switch` statement is reached.**
- Note that `break`; here has a special meaning.
- If no `case` label is selected and `default:` is present, the control goes to the `default:` label.
- `default:` is optional, and often appears in the end, though not necessarily.
- `break`; is often needed. Modern compilers often warn against a missing ``break;``

The expression in a `case` label must be an integer [constant expression](#), whose value is known at compile-time, such as `42`, `'a'`, `true`, ...

```
int n; scanf("%d", &n);
int x = 42;
switch (value) {
    case 3.14: // Error: It must have an integer type.
        printf("It is pi.\n");
    case n:    // Error: It must be a constant expression (known at compile-time)
        printf("It is equal to n.\n");
    case 42:   // OK.
        printf("It is equal to 42.\n");
    case x:    // Error: `x` is a variable, not treated as "constant expression".
        printf("It is equal to x.\n");
}
```

Another example: Determine whether a letter is vowel or consonant.

```
switch (letter) {
case 'a':
case 'e':
case 'i':
case 'o':
case 'u':
    printf("%c is vowel.\n", letter);
    break;
default:
    printf("%c is consonant.\n", letter);
}
```

## Functions

### Call and return

```
#include <stdlib.h>
#include <stdio.h>

double divide(int a, int b) {
    if (b == 0) {
        fprintf(stderr, "Division by zero!\n");
        exit(EXIT_FAILURE);
    }
    return 1.0 * a / b;
}

int main(void) {
    int x, y; scanf("%d%d", &x, &y);
    double result = divide(x, y);
    printf("%1f\n", result);
    // ...
}
```

- For the *call expression* `divide(x, y)`: First the arguments `x` and `y` are passed into `divide` as if the parameters are initialized  $\textcolor{red}{\{ \}}$  as follows:

```
int a = x;
int b = y;
```

Then control is transferred into the function `divide`, starting from the first statement.

`return` does two things:

- passes a value out to the *call site*.
  - This value is the result of the **call expression**.
  - Such a value does not exist if the function return type is `void`.
- transfers the control to the *call site*.
- The parentheses `()` in the expression `divide(x, y)` is the **function-call operator**.
- Even if the function accepts no parameters, the function-call operator should not be omitted.
- A statement like `f;` without the function-call operator is **valid**.
  - It is a statement that has no effect, just like `5;`, `2+3;`, `;` or `{}`.

If a function has return type `void`, the function does not pass a value to the call site.

For a non-`void` function:

- A `return something;` statement must be executed to return something.
- If control reaches the end of the function without a `return` statement, the return value is undefined. **The behavior is undefined** if such value is used.

```
int do_something(int i) {
    printf("%d\n", i);
}
```

```
a.c: In function 'do_something':
a.c:5:1: warning: control reaches end of non-void function [-Wreturn-type]
  5 | }
```

The last `if (x > 0)` is not needed:

```
int abs_int(int x) {
    if (x < 0)
        return -x;
    else if (x == 0)
        return 0;
    else // x > 0 must hold. No need to test it
        return x;
}
```

It can be simplified as

```
int abs_int(int x) {
    return x < 0 ? -x : x;
}
```

## Function declaration and definition

A **definition** of a function contains the function body.

```
int sum(int a, int b) {
    return a + b;
}
```

A **declaration** of a function contains only its return type, its name and the types of its parameters, ended with `;`.

The following statements declare **the same function**:

```
int sum(int, int);
int sum(int x, int y);
int sum(int a, int);
```

- A function should have only one definition, but can be declared many times.
- A definition is also a declaration, since it contains all the information that a declaration has.
- When a function is called, its declaration must be present.

```
int sum(int, int);    // declares the function
int main(void) {
    int x = sum(2, 3);    // ok
}
int sum(int x, int y) { // gives its definition afterwards
    return x + y;
}
```

## Scopes

```
int add(int x, int y) {
    return x + y;
}
int square(int x) {
    return x * x;
}
int main(void) {
    int x; scanf("%d", &x);
    printf("%d\n", square(x));
    if (x == 42) {
        int x = 35;
        printf("%d\n", square(square(x)));
    }
    for (int x = 1; x <= 10; ++x)
        printf("%d\n", square(x + 1));
    return 0;
}
```

- The scopes form a tree structure:

```
global---add
    |-square
    |-main---if
        |-for
```

## Name lookup

- The scopes form a tree structure:

```
global---add
    |-square
    |-main---if
        |-for
```

- When a name `x` is referenced, the **name lookup** for `x` is performed:
  - Only the declarations before the current position can be seen.
  - Lookup is performed from the innermost scope to the outer scopes, until a declaration is found.



- A declaration in an inner scope may hide a declaration in an outer scope that introduces the same name.

## Scopes and name lookup

**[Best practice]** Declare a variable right before the use of it. Declare it in a scope as small as possible.

**[Best practice]** Don't worry about the same names in different scopes.

```
// The three `i`'s are local to the three loops. They will not collide.
for (int i = 0; i < n; ++i)
    do_something(i);
for (int i = 0; i < n; ++i)
    do_another_thing(i);
if (condition()) {
    for (int i = 0; i < n; ++i)
        do_something_else(i);
}
```

## CS100 Lecture 5

Variables II, Pointers and Arrays I

### Contents

- Variables
  - Local `static` variables
  - Initialization
  - `const` type qualifier
- Pointers
- Arrays

### Local `static` variables

```
void start_game(Player *p1, Player *p2, int difficulty, GameWorld *world) {
    static bool called = false;
    if (called)
        report_an_error("You cannot start the game twice!");
    called = true;
    // ...
}
```

The lifetime of a local `static` variable is **as long as** that of a global variable. (They both have [static storage duration](#).)

- A local `static` variable is initialized **during program startup**, and is destroyed **on program termination**.

It behaves just like a global variable, but its name is inside a function, which does not pollute the global name space.

### Initialization

If we declare a variable without explicit initialization, what is the value of it?

Experiment:

```
#include <stdio.h>

int global;

int main(void) {
    int local;
    static int local_static;
    printf("%d, %d, %d\n", global, local,
           local_static);
    return 0;
}
```

- Compiled without `-O2` (a kind of optimization):

```
0, 22031, 0
```

- Compiled with `-O2`:

```
0, 0, 0
```

## Implicit initialization [Very important]

If a variable is declared without explicit initialization:

- For local non-`static` variables, they are initialized to **indeterminate values**. In other words, they are **uninitialized**.
- For global or local `static` variables, they are **empty-initialized**  $\textcolor{red}{1}$ :
  - Pointers are initialized to *null pointer values* of their types. (later in this lecture)
  - Objects of integral types are initialized to `0`.
  - Objects of floating types are initialized to positive zero (`0.0`).
  - Other cases will be discussed in later lectures.

\* Intuitively, such variables are initialized to some kind of "zero"  $\textcolor{red}{2}$ . This is called [zero-initialization](#) in C++.

## Uninitialized garbage can be deadly!

**[Best practice]** Always initialize the variable.

Except in certain cases, e.g.

```
// in some function
int n;           // uninitialized
scanf("%d", &n); // A value is assigned to `n` immediately. This is OK.
// Now the value of `n` is not indeterminate. It can be used normally.
```

## const type qualifier

Each type `T` (not `const`-qualified) has a `const`-qualified version of that type, written as `T const` or `const T`.

Any direct modification of variables with `const`-qualified types is not allowed:

```
const int n = 100; // Type of `n` is `const int`.
++n; // Error.
```

(Any indirect modification of `const` variables is undefined behavior; see in later lectures.)

A `const` variable cannot be modified after initialization.

Therefore, an uninitialized `const` local non-`static` variable is almost a non-stop ticket to undefined behavior.

```
// in some function
const int n; // `n` has indeterminate values
n = 42; // Error: cannot modify a const variable.
scanf("%d", &n); // Error: cannot modify a const variable.
```

In C++, `const` variables of built-in types must be initialized.

## Pointers

### Pointers

A pointer *points* to a variable. The **value** of a pointer is the address of the variable that it points to.

```
int i = 42;
int* pi = &i;
printf("%d\n", *pi);
```

- `int* pi;` declares a pointer named `pi`.
  - The type of `pi` is `int*`.
  - The type of the variable that `pi` points to ("pointee") is `int`.
- `&` is the **address-of operator**, used for taking the address of a variable.
- `*` in the expression `*pi` is the **indirection (dereference) operator**, used for obtaining the variable that a pointer points to.

A pointer *points* to a variable.

We can access and modify a variable through its address (or a pointer pointing to it).

```
int num = 3;
int* ptr = &num;
printf("%d\n", *ptr); // 3
*ptr = 10;
printf("%d\n", num); // 10
++num;
printf("%d\n", *ptr); // 11
```

## Declare a pointer

To declare a pointer: `PointeeType* ptr;`

- The type of `ptr` is `PointeeType*`.
  - Pointer types with different pointee types are **different types**: `int*` and `double*` are different.
- The asterisk `*` can be placed near either `PointeeType` or `ptr`:
  - `PointeeType* ptr;` and `PointeeType *ptr;` are the same declaration.
  - `PointeeType * ptr;`, `PointeeType * ptr;` and `PointeeType*ptr;` are also correct.

The asterisk `*` can be placed near either `PointeeType` or `ptr`:

- `PointeeType* ptr;` may be more intuitive?

However, when declaring more than one pointers in one declaration statement, an asterisk is needed **for every identifier**:

```
int* p1, p2, p3; // `p1` is of type `int*`, but `p2` and `p3` are ints.
int *q1, *q2, *q3; // `q1`, `q2` and `q3` all have the type `int*`.
int* r1, r2, *r3; // `r1` and `r3` are of the type `int*`,
                  // while `r2` is an int.
```

**[Best practice]** Either `PointeeType *ptr` or `PointeeType* ptr` is ok. Choose one style and stick to it. But if you choose the second one, never declare more than one pointers in one declaration statement.

## & and \*

`&var` returns the address of the variable `var`.

- The result type is `Type *`, where `Type` is the type of `var`.
- `var` must be an object that has an identity (an *lvalue*) `{}^{\textcolor{red}{3}}$`: `&42` or `&(a + b)` are not allowed.

`*expr` returns **the variable** whose address is the value of `expr`.

- `expr` must have a pointer type `PointeeType *`. The result type is `PointeeType`.
- The variable** is returned, not only its value. This means that we can modify the returned variable: `++*ptr` is allowed.

## \*

In a **declaration** `PointeeType *ptr`, `*` is a part of the pointer type `PointeeType *`.

In an **expression** like `*ptr`, `*` is the **indirection (dereference) operator** used to obtain the variable whose address is the value of `ptr`.

Do not mix them up!

## The null pointer

The **null pointer value** is the "zero" value for pointer types `{}^{\textcolor{red}{4}}$`.

- It can be obtained from the macro `NULL`, which is available from many standard library header files (e.g. `<stddef.h>`, `<stdio.h>`, `<stdlib.h>`):

```
int *ptr = NULL; // `ptr` is a null pointer.
```

- It can also be obtained from the integer literal `0`.

```
double *ptr = 0; // same as `double *ptr = NULL;`
```

- Conversion from a null pointer to an integer type results in `0`.

Note: Better null pointer values (`nullptr`) are available [in C23](#) and [in C++11](#).

The **null pointer value** is the "zero" value for pointer types `{}^{\textcolor{red}{4}}$`.

A null pointer compares unequal to any pointer pointing to an object.

It is used for representing a pointer that "points nowhere".

**Dereferencing a null pointer is undefined behavior, and often causes severe runtime errors!**

- Because it is not pointing to an object.

```
int *ptr = NULL;
printf("%d\n", *ptr); // undefined behavior
*ptr = 42; // undefined behavior
```

## Implicit initialization of pointers

If a pointer is not explicitly initialized:

- Global or local `static`: Initialized to the null pointer value.

- Local non-`static`: Initialized to indeterminate values, or in other words, **uninitialized**.
  - Uninitialized pointers are often called **wild pointers**.

A wild pointer do not point to a specific object, and is not a null pointer either.

**Dereferencing a wild pointer is undefined behavior, and often causes severe runtime errors.**

**[Best practice]** Avoid wild pointers.

## Pointers that are not dereferenceable

A pointer `ptr` is dereferenceable.  $\rightarrow *ptr$  has no undefined behavior.  $\rightarrow ptr$  points to an existing object.

A pointer that does not point to an existing object may be

- uninitialized (wild), or
- a null pointer, or
- dangling (We will discuss this in later lectures.), or
- holding some other meaningless address: `int *p = 123`

Dereferencing such a pointer is undefined behavior, and usually causes severe runtime errors.

- Recall the "short-circuit" evaluation of binary logical operators:

```
if (ptr != NULL && *ptr == 42) { /* ... */ }
```

When `ptr` is a null pointer, the right-hand side operand `*ptr == 42` won't be evaluated, so `ptr` is not dereferenced.

## Argument passing

What is the output? Is the value of `i` changed to `42`?

```
void fun(int x) {
    x = 42;
}
int main(void) {
    int i = 30;
    fun(i);
    printf("%d\n", i); // 30
}
```

The output is still `30`. `i` is not changed.

- The parameter `x` is initialized as if `int x = i;`, thus obtaining the **value** of `i`.
  - `x` and `i` are two independent variables.
- Modification on `x` does not influence `i`.

## Arrays

### Arrays

An array is a sequence of `N` objects of an *element type* `ElemType` stored **contiguously** in memory, where `N`  $\in \mathbb{Z}_+$  is the *length* of it.

```
ElemType arr[N];
```

`N` must be a **constant expression** whose value is known at compile-time.

```
int a1[10]; // OK. A literal is a constant expression.
#define MAXN 10
int a2[MAXN]; // OK. `MAXN` is replaced with `10` by the preprocessor.
int n; scanf("%d", &n);
int a[n]; // A C99 VLA (Variable-Length Array), whose length is
// determined at runtime.
```

For now, we do not recommend the use of VLAs. We will talk more about it in recitations.

An array is a sequence of `N` objects of an *element type* `ElemType` stored **contiguously** in memory, where `N`  $\in \mathbb{Z}_+$  is the *length* of it.

```
ElemType arr[N]; // The type of `arr` is `ElemType [N]`.
```

The type of an array consists of two parts:

- the element type `ElemType`, and
- the length of the array `[N]`.

```
ElementType arr[N];
```

Use `arr[i]` to obtain the `i`-th element of `arr`, where `i`  $\in [0, N)$ .

```
int a[10];

bool find(int value) {
    for (int i = 0; i < 10; ++i)
        if (a[i] == value)
            return true;
    return false;
}
```

```
int main(void) {
    int n; scanf("%d", &n);
    for (int i = 0; i < n; ++i)
        scanf("%d", &a[i]);
    for (int i = 0; i < n; ++i)
        a[i] *= 2;
    // ...
}
```

The subscript `i` is an integer within the range  $[0, N)$ . **Array subscript out of range is undefined behavior, and usually causes severe runtime errors.**

The compiler may assume that the program is free of undefined behaviors:

If an array is declared without explicit initialization:

- Global or local `static`: Empty-initialization  $\rightarrow$  Every element is empty-initialized.
- Local non-`static`: Every element is initialized to indeterminate values (uninitialized).

Arrays can be initialized from [brace-enclosed lists](#):

- Initialize the beginning few elements:

```
int a[10] = {2, 3, 5, 7}; // Correct: Initializes a[0], a[1], a[2], a[3]
int b[2] = {2, 3, 5};    // Error: Too many initializers
int c[] = {2, 3, 5};     // Correct: 'c' has type int[3].
int d[100] = {};         // Correct in C++ and since C23.
```

- Initialize designated elements (since C99):

```
int e[10] = {[0] = 2, 3, 5, [7] = 7, 11, [4] = 13};
```

If an array is explicitly initialized, all the elements that are not explicitly initialized are **empty-initialized**.

```
int main(void) {
    int a[10] = {1, 2, 3}; // a[3], a[4], ... are all initialized to zero.
    int b[100] = {0};      // All elements of b are initialized to zero.
    int c[100] = {1};      // c[0] is initialized to 1,
                          // and the rest are initialized to zero.
}
```

`= {x}` is not initializing all elements to `x`!

## Nested arrays

The C answer to "multidimensional arrays" is **nested arrays**, which is in fact **arrays of arrays**:

```
int a[10][20];

bool find(int value) {
    for (int i = 0; i < 10; ++i)
        for (int j = 0; j < 20; ++j)
            if (a[i][j] == value)
                return true;
    return false;
}
```

## Initialization of nested arrays

```
int a[4][3] = { // array of 4 arrays of 3 ints each (4x3 matrix)
    { 1 },      // row 0 initialized to {1, 0, 0}
    { 0, 1 },   // row 1 initialized to {0, 1, 0}
    { [2]=1 },  // row 2 initialized to {0, 0, 1}
};             // row 3 initialized to {0, 0, 0}
int b[4][3] = { // array of 4 arrays of 3 ints each (4x3 matrix)
    1, 3, 5, 2, 4, 6, 3, 5, 7 // row 0 initialized to {1, 3, 5}
};                             // row 1 initialized to {2, 4, 6}
                                // row 2 initialized to {3, 5, 7}
                                // row 3 initialized to {0, 0, 0}
int y[4][3] = {[0][0]=1, [1][1]=1, [2][0]=1}; // row 0 initialized to {1, 0, 0}
                                                // row 1 initialized to {0, 1, 0}
                                                // row 2 initialized to {1, 0, 0}
                                                // row 3 initialized to {0, 0, 0}
```

# CS100 Lecture 6

Pointers and Arrays II

## Contents

Pointers and Arrays

- Pointer arithmetic
- Array-to-pointer conversion
- Pass an array to a function
- Pass a nested array to a function
- Do we need an array?

## Pointers and Arrays

### Pointer arithmetic

Let `p` be a pointer of type `T *` and let `i` be an integer.

- `p + i` returns the address equal to the value of `(char *)p + i * sizeof(T)`. In other words, pointer arithmetic uses the unit of the pointed-to type.
- If we let `p = &a[0]` (where `a` is an array of type `T [N]`), then
  - `p + i` is equivalent to `&a[i]`, and
  - `*(p + i)` is equivalent to `a[i]`.
- Arithmetic operations `i + p`, `p += i`, `p - i`, `p -= i`, `++p`, `p++`, `--p`, `p--` are defined in the same way.

### Array-to-pointer conversion

If we let `p = &a[0]` (where `a` is an array of type `T [N]`), then

- `p + i` is equivalent to `&a[i]`, and
- `*(p + i)` is equivalent to `a[i]`.

Considering the close relationship between arrays and pointers, an array can be **implicitly converted** to a pointer to the first element: `a`  $\rightarrow$  `&a[0]`, `T [N]`  $\rightarrow$  `T *`.

- `p = &a[0]` can be written as `p = a` directly.
- `*a` is equivalent to `a[0]`.

We can use pointers to traverse an array:

```
int a[10];

bool find(int value) {
    for (int *p = a; p < a + 10; ++p)
        if (*p == value)
            return true;
    return false;
}
```

### Subtraction of pointers

Let `a` be an array of length `N`. If `p1 == a + i` and `p2 == a + j` (where `i` and `j` are nonnegative integers), the expression `p1 - p2`

- has the value equal to `i - j`, and
- has the type `ptrdiff_t`, which is a **signed** integer type declared in `<stddef.h>`.
  - The size of `ptrdiff_t` is implementation-defined. For example, it might be 64-bit on a 64-bit machine, and 32-bit on a 32-bit machine.
- Here `i`, `j`  $\in [0, N]$  (closed interval), i.e. `p1` or `p2` may point to the "past-the-end" position of `a`.

### Pointer arithmetic

Pointer arithmetic can only happen within the range of an array and its "past-the-end" position (indexed `[0, N]`). For other cases, **the behavior is undefined**.

Examples of undefined behaviors:

- `p1 - p2`, where `p1` and `p2` point to the positions of two different arrays.
- `p + 2 * N`, where `p` points to some element in an array of length `N`.
- `p - 1`, where `p` points to the first element `a[0]` of some array `a`.

Note that the evaluation of the innocent-looking expression `p - 1`, without dereferencing it, is still undefined behavior and may fail on some platforms.

## Pass an array to a function

The only way  $\textcolor{red}{\text{\texttt{}}}$  of passing an array to a function is to **pass the address of its first element**.

The following declarations are equivalent:

```
void fun(int *a);
void fun(int a[]);
void fun(int a[10]);
void fun(int a[2]);
```

In all these declarations, the type of the parameter `a` is `int *`.

- How do you verify that?

## Pass an array to a function

```
void fun(int a[100]);
```

The type of the parameter `a` is `int *`. How do you verify that?

```
void fun(int a[100]) {
    printf("%d\n", (int)sizeof(a));
}
```

Output: (On 64-bit Ubuntu 22.04, GCC 13)

```
8
```

- If the type of `a` is `int[100]` as declared, the output should be `400` (assuming `int` is 32-bit).

## Pass an array to a function

Even if you declare the parameter as an array (either `T a[N]` or `T a[]`), its type is still a pointer `T*`: **You are allowed to pass anything of type `T*` to it.**

- Array of element type `T` with any length is allowed to be passed to it.

```
void print(int a[10]) {
    for (int i = 0; i < 10; ++i)
        printf("%d\n", *(a + i));
}
int main(void) {
    int x[20] = {0}, y[10] = {0}, z[5] = {0}, w = 42;
    print(x); // OK
    print(y); // OK
    print(z); // Allowed by the compiler, but undefined behavior!
    print(&w); // Still allowed by the compiler, also undefined behavior!
}
```

Even if you declare the parameter as an array (either `T a[N]` or `T a[]`), its type is still a pointer `T*`: **You are allowed to pass anything of type `T*` to it.**

- Array of element type `T` with any length is allowed to be passed to it.

The length `n` of the array is often passed explicitly as another argument, so that the function can know how long the array is.

```
void print(int *a, int n) {
    for (int i = 0; i < n; ++i)
        printf("%d\n", *(a + i));
}
```

## Subscript on pointers

```
void print(int *a, int n) {
    for (int i = 0; i < n; ++i)
        printf("%d\n", a[i]); // Look at this!
}
```

Subscript on pointers is also allowed! `a[i]` is equivalent to `*(a + i)`.  $\textcolor{red}{2}$

## Return an array?

There is no way of returning an array from the function.

Returning the address of its first element is ok, **but be careful**:

This is OK:

```
int a[10];

int *foo(void) {
    return a;
}
```

This returns an **invalid address**! (Why?)

```
int *foo(void) {
    int a[10] = {0};
    return a;
}
```

## Return an array?

These two functions have made the same mistake: **returning the address of a local variable**.

```
int *foo(void) {
    int a[10] = {0};
    return a;
}

int main(void) {
    int *a = foo();
    a[0] = 42; // undefined behavior
}
```

```
int *fun(void) {
    int x = 42;
    return &x;
}

int main(void) {
    // undefined behavior
    printf("%d\n", *fun());
}
```

- When the function returns, all the parameters and local objects are destroyed.
  - `a` and `x` no longer exist.
- The objects on the returned addresses are **"dead"** when the function returns!

## Pointer type (revisited)

The type of a pointer is `PointeeType *`.

For two different types `T1` and `T2`, the pointer types `T1 *` and `T2 *` are **different types**, although they may point to the same location.

```
int i = 42;
float *fp = &i;
++*fp; // Undefined behavior. It is not ++i.
```

In C, pointers of different types can be implicitly converted to each other (with possibly a warning). This is **extremely unsafe** and an error in C++.

Dereferencing a pointer of type `T1 *` when it is actually pointing to a `T2` is *almost always* undefined behavior.

- We will see one exception in the next lecture.  $\textcolor{red}{3}$

## Pass a nested array to a function

When passing an array to a function, we make use of the **array-to-pointer conversion**:

- `Type [N]` will be implicitly converted to `Type *`.



A "2d-array" is an "array of array":

- `Type [N][M]` is an array of `N` elements, where each element is of type `Type [M]`.
- `Type [N][M]` should be implicitly converted to a "pointer to `Type[M]`".

What is a "pointer to `Type[M]`"?

---

## Pointer to array

A pointer to an array of `N` `int`s:

```
int (*parr)[N];
```

An array of `N` pointers (pointing to `int`):

```
int *arrp[N];
```

Too confusing! How can I remember them?

- `int (*parr)[N]` has a pair of parentheses around `*` and `parr`, so
  - `parr` is a pointer (`*`), and
  - points to something of type `int[N]`.
- Then the other one is different:
  - `arrp` is an array, and
  - stores `N` pointers, with pointee type `int`.

---

## Pass a nested array to a function

```
void print(int (*a)[5], int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < 5; ++j)
            printf("%d ", a[i][j]);
        printf("\n");
    }
}

int main(void) {
    int a[2][5] = {{1, 2, 3, 4, 5}, {6, 7, 8, 9, 10}};
    int b[3][5] = {0};
    print(a, 2); // OK
    print(b, 3); // OK
}
```

In each of the following declarations, what is the type of `a`? Does it accept an argument of type `int[N][M]`?

1. `void fun(int a[N][M])`: A pointer to `int[M]`. Yes.
2. `void fun(int (*a)[M])`: Same as 1.
3. `void fun(int (*a)[N])`: A pointer to `int[N]`. **Yes iff `N == M`.**
4. `void fun(int **a)`: A pointer to `int *`. **No.**
5. `void fun(int *a[])`: Same as 4.
6. `void fun(int *a[N])`: Same as 4.
7. `void fun(int a[100][M])`: Same as 1.
8. `void fun(int a[N][100])`: A pointer to `int[100]`. Yes iff `M == 100`.

---

# CS100 Lecture 7

Pointers and Arrays III, Dynamic memory, Strings

---

## Contents

- Pointers and Arrays
  - Pointers and `const`
  - The `void *` type
- Dynamic memory
- Strings

---

## Pointers and Arrays

---

### Pointer to `const`

A pointer to `const` is a pointer whose pointee type is `const`-qualified:

```
const int x = 42;
int *pi = &x; // Dangerous: It discards the const qualifier.
const int *cpi = &x; // Better.
```

The type of `cpi` is `const int *` (or equivalently, `int const *`), which is a pointer to `const int`.

## const is a "lock"

`const` is like a lock, guarding against modifications to the variable.

It is very dangerous to let a pointer to non-`const` point to a `const` variable: It is an attempt to remove the lock!

- Warning in C, error in C++.

```
const int x = 42;
int *pi = &x; // Dangerous: It discards the const qualifier.
const int *cpi = &x; // Better.
++*pi; // No error is generated by the compiler, but actually undefined behavior.
```

Any indirect modification to a `const` variable is **undefined behavior**.

`const` is like a lock, guarding against modifications to the variable.

A pointer to `const` can point to a non-`const` variable: This is adding a lock.

```
int x = 42;
int *pi = &x;
const int *cpi = &x; // OK.
++*pi; // Correct, same as ++x.
++*cpi; // Error!
```

- A pointer to `const` *thinks* that it is pointing to a `const` variable. Therefore, it does not allow you to modify the variable through it.

Such `const` ness on the **pointee type** is often called "low-level `const` ness".

## const can be helpful

It tells the compiler "this variable should not be modified!".

```
int count(const int *a, int n, int value) {
    int cnt = 0;
    for (int i = 0; i < n; ++i)
        if (a[i] == value) // Error: cannot modify a[i]
            ++cnt;
    return cnt;
}
```

**[Best practice]** Use `const` whenever possible.

We will see more usage of `const` in C++.

## Top-level const ness

A pointer itself can also be `const`. The type of such pointer is `PointeeType *const`.

- Such `const` ness is often called "top-level `const` ness".

```
int x = 42;
int *const pc = &x;
++*pc; // OK.
int y = 30;
pc = &y; // Error.
```

A `const` pointer cannot switch to point to other variables after initialization.

A pointer can have both low-level and top-level `const` ness:

```
const int *const cipc = &x;
```

## void \*

A special pointer type:

- Any pointer can be implicitly converted to that type.
- A pointer of type `void *` can be implicitly converted to any pointer type.
  - This must happen explicitly in C++.
- Use `printf("%p", ptr);` to print the value of a pointer `ptr` of type `void *`.
  - If `ptr` is a pointer of some other type, a conversion is needed:

```
printf("%p", (void *)ptr);
```

The C type system is weak. Many kinds of implicit conversions are very dangerous, although allowed by C.

C does not have a static type system as powerful as C++'s. `void *` is often used to represent "pointer to anything", "location of some memory", or even "any object".

- Typically, the memory allocation function `malloc` (see below) returns `void *`, the address of the block of memory allocated.
  - Memory does not have types. - We say "a disk of 1TB" instead of "a disk that can hold  $2^{38}$  `int`'s".

## Dynamic memory

### A "dynamic array"

Create an "array" whose size is determined at runtime?

- We need a block of memory, the size of which can be determined at runtime.
- If we run out of memory, **we need to know**.
- We may require a pretty large chunk of memory.

## Stack memory vs heap (dynamic) memory

- Stack memory is generally smaller than heap memory.
- Stack memory is often used for storing local and temporary objects.
- Heap memory is often used for storing large objects, and objects with long lifetime.
- Operations on stack memory is faster than on heap memory.
- Stack memory is allocated and deallocated automatically, while heap memory needs manual management.

## Use `malloc` and `free`

Declared in `<stdlib.h>`.

```
void *malloc(size_t size);
```

```
T *ptr = malloc(sizeof(T) * n); // sizeof(T) * n bytes
for (int i = 0; i != n; ++i)
    ptr[i] = /* ... */
// Now you can use `ptr` as if it points to an array of `n` objects of type `T`
// ...
free(ptr);
```

To avoid **memory leaks**, the starting address of that block memory must be passed to `free` when the memory is not used anymore.

## Use `malloc` and `free`

Declared in `<stdlib.h>`.

```
void free(void *ptr);
```

Deallocates the space previously allocated by an allocation function (such as `malloc`).

**The behavior is undefined** if the memory area referred to by `ptr` has already been deallocated.

- In other words, "double `free`" is undefined behavior (and often causes severe runtime errors).

After `free(ptr)`, `ptr` no longer points to an existing object, so it is no longer dereferenceable.

- Often called a "dangling pointer".

## Use `malloc` and `free`

We can also create one single object dynamically (on heap):

```
int *ptr = malloc(sizeof(int));
*ptr = 42;
printf("%d\n", *ptr);
// ...
free(ptr);
```

But why? Why not just create one normal variable like `int ival = 42;`?

Benefit: The lifetime of a dynamically allocated object goes beyond a local scope.

It is not destroyed until we `free` it.

```
int *create_array(void) {
    int a[N];
    return a; // Returns the address of the local object `a`.
              // when the function returns, `a` will be destroyed, so that
              // the returned address becomes invalid.
              // Dereferencing the returned address is undefined behavior.
}
int *create_dynamic_array(int n) {
    return malloc(sizeof(int) * n); // OK. The allocated memory is valid until
                                   // we free it.
}
```

Create a "2-d array" on heap?

```
int **p = malloc(sizeof(int *) * n);
for (int i = 0; i < n; ++i)
    p[i] = malloc(sizeof(int) * m);
for (int i = 0; i < n; ++i)
    for (int j = 0; j < m; ++j)
        p[i][j] = /* ... */
// ...
for (int i = 0; i < n; ++i)
    free(p[i]);
free(p);
```

## Use `calloc`

Declared in `<stdlib.h>`

```
void *calloc(size_t num, size_t each_size);
```

Allocates memory for an array of `num` objects (each of size `each_size`), and initializes all bytes in the allocated storage to zero `{0}`.

Similar as `malloc(num * each_size)`. `{0}` Returns a null pointer on failure.

## `malloc`, `calloc` and `free`

The behaviors of `malloc(0)`, `calloc(0, N)` and `calloc(N, 0)` are **implementation-defined**:

- They may or may not allocate memory.
- If no memory is allocated, a null pointer is returned.
- They may allocate *some* memory, for some reasons. In that case, the address of the allocated memory is returned.
  - You cannot dereference the returned pointer.
  - It still constitutes **memory leak** if such memory is not `free`d.

## Arrays vs `malloc`

- An array has limited lifetime (unless it is global or `static`). It is destroyed when control reaches the end of its scope.
- Objects allocated by `malloc` are not destroyed until their address is passed to `free`.
- The program crashes if the size of an array is too large (running out of stack memory). There is no way of recovery.
- Attempt to `malloc` a block of memory that is too large results in a null pointer. We can know if there is no enough heap memory by doing a null check.

```
int *ptr = malloc(1ull << 60); // unrealistic size
if (!ptr)
    report_an_error("Out of memory.");
```

## Summary

Pointer to `const`

- A pointer to `const` *thinks* that it is pointing to a `const` variable (though it may not), so it prevents you from modifying the pointed-to variable through it.
- Use `const` whenever possible.

`void *`

- A pointer type that can contain anything.
- Often used for representing "any pointer", "any object", or memory address.

## Summary

Dynamic memory

```
void *malloc(size_t size);
void *calloc(size_t num, size_t each_size);
void free(void *ptr);
```

`malloc`: Allocates `size` bytes of **uninitialized** memory and returns its starting address.

`calloc`: Allocates `num * each_size` bytes of memory `${}^{\textcolor{red}{2}}$`, each byte initialized to zero, and returns its starting address.

Both `malloc` and `calloc` return a null pointer on failure.

`free`: Deallocates the memory block starting at `ptr`.

## Strings

### C-style strings

C does not have a special construct for "string".

A string is a sequence of characters stored contiguously. We often use an array or a pointer to the first character to represent a string.

- It can be stored in an array, or in dynamically allocated memory.
- **It must be null-terminated: There should be a null character `'\0'` at the end.**

```
char s[10] = "abcde"; // s = {'a', 'b', 'c', 'd', 'e', '\0'}
printf("%s\n", s);    // prints abcde
printf("%s\n", s + 1); // prints bcde
s[2] = ';';           // s = "ab;de"
printf("%s\n", s);    // prints ab;de
s[2] = '\0';
printf("%s\n", s);    // prints ab
```

The position of the first `'\0'` is the end of the string. Anything after that is discarded.

### The null character `'\0'`

`'\0'` is the "null character" whose [ASCII](#) value is 0.

It is **the only way** to mark the end of a C-style string.

Every standard library function that handles strings will search for `'\0'` in that string.

- If there is no `'\0'`, they will search nonstop, and eventually go out of range (undefined behavior).

```
char s[5] = "abcde"; // OK, but no place for '\0'.
printf("%s\n", s);    // undefined behavior (missing '\0')
```

**Remember to allocate one more byte storage for `'\0'`!**

### Empty string

An empty string contains no characters before the null character.

```
char empty[] = ""; // `empty` is of type char[1], which contains only '\0'.
printf("%s\n", empty); // Prints only a newline.
printf(""); // Nothing is printed
```

## String I/O

`scanf` / `printf`: `"%s"`

- `%s` in `scanf` matches a sequence of **non-whitespace** characters.
  - Leading whitespaces are discarded.
  - Reading starts from the first non-whitespace character, and stops right before the next whitespace character.
  - `'\0'` will be placed at the end.

Suppose the input is `123 456`:

```
char str[100] = "abcdef";
scanf("%s", str); // Reads "123". `str` becomes {'1', '2', '3', '\0', 'e', 'f'}
printf("%s\n", str); // Prints "123".
// 'e' and 'f' are not considered as part of the string.
```

## String I/O

`scanf` is not memory safe:

```
char str[10];
scanf("%s", str);
```

- `str` is decayed (implicitly converted) to `char *` when passed as an argument.
- `scanf` receives only a pointer `char *`. **It has no idea how big the array is.**
- If the input content has more than 9 characters, it causes disaster!

That's why it is banned by MSVC. An alternative is to use `scanf_s`, but not necessarily supported by every compiler.

## String I/O

`gets` reads a string without bounds checking. **It has been removed since C11.**

- An alternative for `gets` that does bounds checking is `gets_s`, but not supported by every compiler.

**The best alternative:** `fgets`. It is more portable, more generic, and safer (with bounds checking).

```
char str[100];
fgets(str, 100, stdin);
```

`puts(str)`: Prints the string `str`, followed by a newline.

## String I/O

Homework Read the [cppreference documentation for fgets](#). Answer the following questions:

- How many characters does it read at most?
- When does it stop?

## String manipulation / examination

Some common standard library functions: declared in `<string.h>`.

- `strlen(str)`: Returns the length of the string `str`.
- `strcpy(dest, src)`: Copies the string `src` to `dest`.
- `strcat(dest, src)`: Appends a copy of `src` to the end of `dest`.
- `strcmp(s1, s2)`: Compares two strings in lexicographical order.
- `strchr(str, ch)`: Finds the first occurrence of `ch` in `str`.

**This page is only a brief introduction which cannot be relied on.** The detailed documentations can be found [here](#).

## String manipulation / examination

Read the documentation of a function before using it.

- Is `'\0'` counted in `strlen`?
- Does `strcpy` put a null character at the end? What about `strncpy`?
- For `strcpy(dest, src)`, what will happen if `dest` and `src` refer to the same memory address? What if they overlap? What about `strcat`?
- What is the result of `strcmp`? Is it  $\in\{-1,0,1\}$ ? Is it `true` / `false`?

If you use the function without making these clear, **you are heading for late-night debugging sessions!**

## String literals

A string literal is something like `"abcde"`, **surrounded by double quotes** `"`.

- The type of a string literal is `char [N+1]`, where `N` is the length of the string.
  - `+1` is for the terminating null character.
- **But a string literal will be placed in read-only memory!!**
  - In C++, its type is `const char [N+1]`, which is more reasonable.

When initializing a pointer with a string literal,

```
char *p = "abcde";
```

we are actually letting `p` point to the address of the string literal.

Using a pointer to non-`const` to point to a string literal is **allowed in C** (not allowed in C++), but **very dangerous**:

```
char *p = "abcde"; // OK
p[3] = 'a'; // No compile-error, but undefined behavior,
           // and possibly severe runtime-error.
```

Correct ways:

Use low-level `const`ness to protect it:

```
const char *str = "abcde";
str[3] = 'a'; // compile-error
```

**Copy** the contents into an array:

```
char arr[] = "abcde";
arr[3] = 'a'; // OK.
// `arr` contains a copy of "abcde".
```

## Array of strings

```
const char *translations[] = {
    "zero", "one", "two", "three", "four",
    "five", "six", "seven", "eight", "nine"
};
```

- `translations` is an array of pointers, where each pointer points to a string literal.
- `translations` **is not a 2-d array!**

## CS100 Lecture 8

Dynamic Memory and Strings Revisited

### Contents

- Recap
- Command line arguments
- Example: Read a string of unknown length

## Command line arguments

### Command line arguments

The following command executes `gcc.exe`, and tells it the file to be compiled and the name of the output:

```
gcc hello.c -o hello
```

How are the arguments `hello.c`, `-o` and `hello` passed to `gcc.exe`?

- It is definitely different from "input".

## A new signature of `main`

```
int main(int argc, char **argv) { /* body */ }
```

Run this program with some arguments: `.\program one two three`

```
int main(int argc, char **argv) {
    for (int i = 0; i < argc; ++i)
        puts(argv[i]);
}
```

Output:

```
.\program
one
two
three
```

```
int main(int argc, char **argv) { /* body */ }
```

where

- `argc` is a non-negative value representing the number of arguments passed to the program from the environment in which the program is run.
- `argv` is a pointer to the first element of an array of `argc + 1` pointers, of which
  - the last one is null, and
  - the previous ones (if any) point to strings that represent the arguments.

If `argv[0]` is not null (or equivalently, if `argc > 0`), it points to a string representing the program name.

## Command line arguments

```
int main(int argc, char **argv) { /* body */ }
```

`argv` is an array of pointers that point to the strings representing the arguments:

## Example: Read a string of unknown length

### Read a string

`fgets(str, count, stdin)` reads a string, but at most `count - 1` characters.

`scanf("%s", str)` reads a string, but not caring about whether the input content is too long to fit into the memory that `str` points to.

For example, the following code is likely to crash if the input is `responsibility`:

```
char word[6];
scanf("%s", word);
```

`scanf` does nothing to prevent the disaster.

- It does not even know how long the array `word` is!

Suppose we want to read a sequence of non-whitespace characters, the length of which is unknown.

- Use `malloc` / `free` to allocate and deallocate memory dynamically.
- When the current buffer is not large enough, we allocate a larger one and copies the stored elements to it!

```
char *read_string(void) {
    // ...
    while (!isspace(c)) {
        if (cur_pos == capacity - 1) { // `-1` is for '\0'.
            // ...
        }
    }
}
```



```

    buffer[cur_pos++] = c;
    c = getchar();
}

// Now, `c` is a whitespace. This is not part of the contents we need.
ungetc(c, stdin); // Put that whitespace back to the input.

buffer[cur_pos] = '\0'; // Remember this!!!

return buffer;
}

```

```

int main(void) {
    char *content = read_string();
    puts(content);
    free(content);
}

```

Remember to `free` it after use!

## CS100 Lecture 9

`struct`, Recursion

### Contents

- `struct`
- Recursion
  - Factorial
  - Print a non-negative integer
  - Selection-sort

## struct

### struct type

The name of the type defined by a `struct` is `struct Name`.

- Unlike C++, the keyword `struct` here is necessary.

```

struct Student stu; // `stu` is an object of type `struct Student`
struct Point3d polygon[1000]; // `polygon` is an array of 1000 objects,
                             // each being of type `struct Point3d`.
struct TreeNode *pNode; // `pNode` is a pointer to `struct TreeNode`.

```

\* The term "*object*" is used interchangeably with "*variable*".

- *Objects* often refer to variables of `struct` (or `class` in C++) types.
- But in fact, there's nothing wrong to say "an `int` object".

### Members of a struct

Use `obj.mem`, the **member-access operator** `.` to access a member.

```

struct Student stu;
stu.name = "Alice";
stu.id = "2024533000";
stu.entrance_year = 2024;
stu.dorm = 8;
printf("%d\n", student.dorm);
++student.entrance_year;
puts(student.name);

```

### Dynamic allocation

Create an object of `struct` type dynamically: Just allocate `sizeof(struct Student)` bytes of memory.

```

struct Student *pStu = malloc(sizeof(struct Student));

```

Member access through a pointer: `ptr->mem`, or `(*ptr).mem` (**not** `*ptr.mem`!).

```
pStu->name = "Alice";
pStu->id = "2024533000";
(*pStu).entrance_year = 2024; // equivalent to pStu->entrance_year = 2024;
printf("%d\n", pStu->entrance_year);
puts(pStu->name);
```

As usual, don't forget to `free` after use.

```
free(pStu);
```

## Size of a `struct`

```
struct Student {
    const char *name;
    const char *id;
    int entrance_year;
    int dorm;
};
```

```
struct Student *pStu = malloc(sizeof(struct Student));
```

What is the value of `sizeof(struct Student)`?

## Size of `struct`

It is guaranteed that

$$\text{sizeof}(\text{struct } X) \geq \sum_{\text{member} \in X} \text{sizeof}(\text{member}).$$

The inequality is due to **memory alignment requirements**, which is beyond the scope of CS100.

## Implicit initialization

What happens if an object of `struct` type is not explicitly initialized?

```
struct Student gStu;

int main(void) {
    struct Student stu;
}
```

## Implicit initialization

What happens if an object of `struct` type is not explicitly initialized?

```
struct Student gStu;

int main(void) {
    struct Student stu;
}
```

- Global or local `static`: "empty-initialization", which performs **member-wise** empty-initialization.
- Local non-`static`: every member is initialized to indeterminate values (in other words, uninitialized).

## Explicit initialization

Use an initializer list:

```
struct Student stu = {"Alice", "2024533000", 2024, 8};
```

Use **C99 designators**: (highly recommended)

```
struct Student stu = {.name = "Alice", .id = "2024533000",  
                      .entrance_year = 2024, .dorm = 8};
```

The designators greatly improve the readability.

**[Best practice]** Use designators, especially for `struct` types with lots of members.

## Compound literals

```
struct Student *student_list = malloc(sizeof(struct Student) * n);  
for (int i = 0; i != n; ++i) {  
    student_list[i].name = A(i); // A, B, C and D are some functions  
    student_list[i].id = B(i);  
    student_list[i].entrance_year = C(i);  
    student_list[i].dorm = D(i);  
}
```

Use a **compound literal** to make it clear and simple:

```
struct Student *student_list = malloc(sizeof(struct Student) * n);  
for (int i = 0; i != n; ++i) {  
    student_list[i] = (struct Student){.name = A(i), .id = B(i),  
                                       .entrance_year = C(i), .dorm = D(i)};  
}
```

## struct-typed parameters

The semantic of argument passing is **copy**:

```
void print_student(struct Student s) {  
    printf("Name: %s, ID: %s, dorm: %d\n", s.name, s.id, s.dorm);  
}  
  
print_student(student_list[i]);
```

In a call `print_student(student_list[i])`, the parameter `s` of `print_student` is initialized as follows:

```
struct Student s = student_list[i];
```

The copy of a `struct`-typed object: **Member-wise copy**.

## struct-typed parameters

In a call `print_student(student_list[i])`, the parameter `s` of `print_student` is initialized as follows:

```
struct Student s = student_list[i];
```

The copy of a `struct`-typed object: **Member-wise copy**. It is performed as if

```
s.name = student_list[i].name;  
s.id = student_list[i].id;  
s.entrance_year = student_list[i].entrance_year;  
s.dorm = student_list[i].dorm;
```

## Return a struct-typed object

Strictly speaking, returning is also a **copy**:

```

struct Student fun(void) {
    struct Student s = something();
    some_operations(s);
    return s;
}
student_list[i] = fun();

```

The object `s` is returned as if

```
student_list[i] = s;
```

But in fact, the compiler is more than willing to optimize this process. We will talk more about this in C++.

## Array member

```

struct A {
    int array[10];
    // ...
};

```

Although an array cannot be copied, **an array member can be copied**.

The copy of an array is **element-wise copy**.

```

int a[10];
int b[10] = a; // Error!

```

```

struct A a;
struct A b = a; // OK

```

## Summary

A `struct` is a type consisting of a sequence of members.

- Member access: `obj.mem`, `ptr->mem` (equivalent to `(*ptr).mem`, but better)
- `sizeof(struct A)`, no less than the sum of size of every member.
  - But not necessarily equal, due to memory alignment requirements.
- Implicit initialization: recursively performed on every member.
- Initializer-lists, designators, compound literals.
- Copy of a `struct`: member-wise copy.
- Argument passing and returning: copy.

## Recursion

### Problem 1. Calculate $n!$

```

int factorial(int n) {
    return n == 0 ? 1 : n * factorial(n - 1);
}

```

This is perfectly valid and reasonable C code!

- The function `factorial` **recursively** calls itself.

Problem 2. Print a non-negative integer

If we only have `getchar`, how can we read an integer?

- We have solved this in recitations.

If we only have `putchar`, how can we print an integer?

- Declared in `<stdio.h>`.
- `putchar(c)` prints a character `c`. That's it.

For convenience, suppose the integer is non-negative (unsigned).

## Print a non-negative integer

To print  $x$ :

- If  $x < 10$ , just print the digit and we are done.
- Otherwise ( $x \geq 10$ ), we first print  $\left\lfloor \frac{x}{10} \right\rfloor$ , and then print the digit on the last place.

```
void print(unsigned x) {
    if (x < 10)
        putchar(x + '0'); // Remember ASCII?
    else {
        print(x / 10);
        putchar(x % 10 + '0');
    }
}
```

---

## Simplify the code

To print  $x$ :

1. If  $x \geq 10$ , we first print  $\left\lfloor \frac{x}{10} \right\rfloor$ . Otherwise, do nothing.
2. Print  $x \bmod 10$ .

```
void print(unsigned x) {
    if (x >= 10)
        print(x / 10);
    putchar(x % 10 + '0');
}
```

---

## Print a non-negative integer

To print  $x$ :

1. If  $x \geq 10$ , we first print  $\left\lfloor \frac{x}{10} \right\rfloor$ . Otherwise, do nothing.
2. Print  $x \bmod 10$ .

```
void print(unsigned x) {
    if (x >= 10)
        print(x / 10);
    putchar(x % 10 + '0');
}
```

---

## Design a recursive algorithm

Suppose we are given a problem of scale  $n$ .

1. Divide the problem into one or more **subproblems**, which are of smaller scales.
2. Solve the subproblems **recursively** by calling the function itself.
3. Generate the answer to the big problem from the answers to the subproblems.

\* Feels like mathematical induction?

---

## Problem 3. Selection-sort

How do you sort a sequence of  $n$  numbers? (In ascending order)

Do it **recursively**.

How do you sort a sequence of  $n$  numbers  $\langle a_0, \dots, a_{n-1} \rangle$ ? (In ascending order)

Do it **recursively**: Suppose we are going to sort  $\langle a_k, a_{k+1}, \dots, a_{n-1} \rangle$ , for some  $k$ .

- If  $k = n-1$ , we are done.
- Otherwise ( $k < n-1$ ):
  1. Find the minimal number  $a_m = \min\{a_k, a_{k+1}, \dots, a_{n-1}\}$ .
  2. Put  $a_m$  at the first place by swapping it with  $a_k$ .
  3. Now  $a_k$  is the smallest number in  $\langle a_k, \dots, a_{n-1} \rangle$ . All we have to do is to sort the rest part  $\langle a_{k+1}, \dots, a_{n-1} \rangle$  **recursively**.

```
void sort_impl(int *a, int k, int n) {
    if (k == n - 1) return;

    int m = k;
    for (int i = k + 1; i < n; ++i)
        if (a[i] < a[m]) m = i;

    swap(&a[m], &a[k]); // the "swap" function we defined in previous lectures

    sort_impl(a, k + 1, n); // sort the rest part recursively
}
```

# CS100 Lecture 11

## IOStream: Input and Output Stream

`std::cin >> x`: Reads something and stores it in the variable `x`.

- `x` can be of any supported type: integers, floating-points, characters, strings, ...
- C++ has a way of identifying the type of `x` and selecting the correct way to read it. We don't need the annoying `"%d"`, `"%f"`, ... anymore.
- C++ functions have a way of obtaining the *reference* of the argument. We don't need to take the address of `x`.

## Standard library file names

The names of C++ standard library files **have no extensions**: `<iostream>` instead of `<iostream.h>`, `<string>` instead of `<string.h>`.

## Namespace `std`

`std::cin` and `std::cout`: names from the standard library.

C++ has a large standard library with a lot of names declared.

To avoid **name collisions**, all the names from the standard library are placed in a **namespace** named `std`.

- You can write `using std::cin`; to introduce `std::cin` into **the current scope**, so that `cin` can be used without `std::`.
- You may write `using namespace std`; to introduce **all the names in `std`** into the current scope, but **you will be at the risk of name collisions again**.

[Best practice] Use `<cxxx>` instead of `<xxx.h>` when you need the C standard library in C++.

## `std::string`

Defined in the standard library file `<string>` (**not** `<string.h>`, **not** `<cstring>` !!)

## Define and initialize a string

```
std::string str = "Hello world";
// equivalent: std::string str("Hello world");
// equivalent: std::string str{"Hello world"}; (modern)
std::cout << str << std::endl;

std::string s1(7, 'a');
std::cout << s1 << std::endl; // aaaaaa

std::string s2 = s1; // s2 is a copy of s1
std::cout << s2 << std::endl; // aaaaaa

std::string s; // "" (empty string)
```

Default-initialization of a `std::string` will produce **an empty string**, not indeterminate value and has no undefined behaviors!

## Strings

- The memory of `std::string` is **allocated and deallocated automatically**.
- We can insert or erase characters in a `std::string`. **The memory of storage will be adjusted automatically**.
- `std::string` **does not need an explicit `'\0'` at the end**. It has its way of recognizing the end.
- When you use `std::string`, **pay attention to its contents** instead of the implementation details.

## Length of a string

### Member function `s.size()` || Member function `s.empty()`

### Use `+=`

In C, `a = a + b` is equivalent to `a += b`. **This is not always true in C++**.

For two `std::string s` `s1` and `s2`, `s1 = s1 + s2` **is different from** `s1 += s2`.

- `s1 = s1 + s2` constructs a temporary object `s1 + s2` (so that the contents of `s1` are copied), and then assigns it to `s1`.
- `s1 += s2` appends `s2` directly to the end of `s1`, without copying `s1`.

## Traversing a string: Use range-based `for` loops.

Example: Print all the uppercase letters in a string.

```
for (char c : s) // The range-based for loops
    if (std::isupper(c)) // in <cctype>
        std::cout << c;
std::cout << std::endl;
```

Equivalent way: Use subscripts, which is verbose and inconvenient.

## String IO

Use `std::cin >> s` and `std::cout << s`, as simple as handling an integer.

- Does `std::cin >> s` ignore leading whitespaces? Does it read an entire line or just a sequence of non-whitespace characters? Do some experiments on it.

`std::getline(std::cin, s)`: Reads a string starting from the current character, and stops at the first `'\n'`.

- Is the ending `'\n'` consumed? Is it stored? Do some experiments.

## CS100 Lecture 12

References, `std::vector`

### Contents

- References
- `std::vector`

## References

### Declare a reference

A **reference** defines an **alternative name** for an object ("refers to" that object).

Similar to pointers, the type of a reference is `ReferredType &`, which consists of two things:

- `ReferredType` is the type of the object that it refers to, and
- `&` is the symbol indicating that it is a reference.

Example:

```
int ival = 42;
int &ri = ival; // `ri` refers to `ival`.
               // In other words, `ri` is an alternative name for `ival`.
std::cout << ri << '\n'; // prints the value of `ival`, which is `42`.
++ri;                  // Same effect as `++ival`.
```

### Declare a reference

```
int ival = 42;
int x = ival;           // `x` is another variable.
++x;                   // This has nothing to do with `ival`.
std::cout << ival << '\n'; // 42
int &ri = ival;         // `ri` is a reference that refers to `ival`.
++ri;                  // This modification is performed on `ival`.
std::cout << ival << '\n'; // 43
```

Ordinarily, when we initialize a variable, the value of the initializer is **copied** into the object we are creating.

When we define a reference, instead of copying the initializer's value, we **bind** the reference to its initializer.

### A reference is an alias

When we define a reference, instead of copying the initializer's value, we **bind** the reference to its initializer.

```
int ival = 42;
int &ri = ival;
++ri;           // Same as `++ival;`.
ri = 50;        // Same as `ival = 50;`.
int a = ri + 1; // Same as `int a = ival + 1;`.
```

After a reference has been defined, **all** operations on that reference are actually operations on the object to which the reference is bound.

```
ri = a;
```

What is the meaning of this?

## A reference is an alias

```
int ival = 42;
int &ri = ival;
++ri;           // Same as `++ival;`.
ri = 50;        // Same as `ival = 50;`.
int a = ri + 1; // Same as `int a = ival + 1;`.
```

When we define a reference, instead of copying the initializer's value, we **bind** the reference to its initializer.

After a reference has been defined, **all** operations on that reference are actually operations on the object to which the reference is bound.

```
ri = a;
```

- This is the same as `ival = a;`. It is **not** rebinding `ri` to refer to `a`.

## A reference must be initialized

```
ri = a;
```

- This is the same as `ival = a;`. It is **not** rebinding `ri` to refer to `a`.

Once initialized, a reference remains bound to its initial object. **There is no way to rebind a reference to refer to a different object.**

Therefore, **references must be initialized.**

## References must be bound to *existing objects* ("lvalues")

It is not allowed to bind a reference to temporary objects or literals `{}^{\textcolor{red}{1}}{}`:

```
int &r1 = 42;    // Error: binding a reference to a literal
int &r2 = 2 + 3; // Error: binding a reference to a temporary object
int a = 10, b = 15;
int &r3 = a + b; // Error: binding a reference to a temporary object
```

In fact, the references we learn today are "lvalue references", which must be bound to *lvalues*. We will talk about *value categories* in later lectures.

## References are not objects

A reference is an alias. It is only an alternative name of another object, but the reference itself is **not an object**.

Therefore, there are no "references to references".

```
int ival = 42;
int &ri = ival; // binding `ri` to `ival`.
int &&rr = ri;  // Error! No such thing!
```

What is the meaning of this code? Does it compile?

```
int &ri2 = ri;
```



---

## References are not objects

A reference is an alias. It is only an alternative name of another object, but the reference itself is **not an object**.

Therefore, there are no "references to references".

```
int ival = 42;
int &ri = ival; // binding `ri` to `ival`.
int &&rr = ri; // Error! No such thing!
```

What is the meaning of this code? Does it compile?

```
int &ri2 = ri; // Same as `int &ri2 = ival;`.
```

- `ri2` is a reference that is bound to `ival`.
- **Any use of a reference is actually using the object that it is bound to!**

---

## References are not objects

A reference is an alias. It is only an alternative name of another object, but the reference itself is **not an object**.

Pointers must also point to objects. Therefore, there are no "pointers to references".

```
int ival = 42;
int &ri = ival; // binding `ri` to `ival`.
int *&pr = &ri; // Error! No such thing!
```

What is the meaning of this code? Does it compile?

```
int *pi = &ri;
```

---

## References are not objects

A reference is an alias. It is only an alternative name of another object, but the reference itself is **not an object**.

Pointers must also point to objects. Therefore, there are no "pointers to references".

```
int ival = 42;
int &ri = ival; // binding `ri` to `ival`.
int *&pr = ri; // Error! No such thing!
```

What is the meaning of this code? Does it compile?

```
int *pi = &ri; // Same as `int *pi = &ival;`.
```

---

## Reference declaration

Similar to pointers, the ampersand `&` only applies to one identifier.

```
int ival = 42, &ri = ival, *pi = &ival;
// `ri` is a reference of type `int &`, which is bound to `ival`.
// `pi` is a pointer of type `int *`, which points to `ival`.
```

Placing the ampersand near the referred type does not make a difference:

```
int& x = ival, y = ival, z = ival;
// Only `x` is a reference. `y` and `z` are of type `int`.
```

---

## \* and &

Both symbols have many identities!

- In a **declaration** like `type *x = expr`, `*` is a **part of the pointer type** `type *`.
- In a **declaration** like `type &r = expr`, `&` is a **part of the reference type** `type &`.
- In an **expression** like `*opnd` where there is only one operand, `*` is the **dereference operator**.
- In an **expression** like `&opnd` where there is only one operand, `&` is the **address-of operator**.
- In an **expression** like `a * b` where there are two operands, `*` is the **multiplication operator**.
- In an **expression** like `a & b` where there are two operands, `&` is the **bitwise-and operator**.

### Example: Use references in range-for

Recall the range-based `for` loops (range-`for`):

```
std::string str;
std::cin >> str;
int lower_cnt = 0;
for (char c : str)
    if (std::islower(c))
        ++lower_cnt;
std::cout << "There are " << lower_cnt << " lowercase letters in total.\n";
```

The range-`for` loop in the code above traverses the string, and declares and initializes the variable `c` in each iteration as if `char c;`

```
for (std::size_t i = 0; i != str.size(); ++i) {
    char c = str[i]; // Look at this!
    if (std::islower(c))
        ++lower_cnt;
}
```

### Example: Use references in range-for

```
for (char c : str)
    // ...
```

The range-`for` loop in the code above traverses the string, and declares and initializes the variable `c` in each iteration as if `for (char c : s)`

```
for (std::size_t i = 0; i != str.size(); ++i) {
    char c = str[i];
    // ...
}
```

Here `c` is a copy of `str[i]`. Therefore, modification on `c` does not affect the contents in `str`.

### Example: Use references in range-for

What if we want to change all lowercase letters to their uppercase forms?

```
for (char c : str)
    c = std::toupper(c); // This has no effect.
```

We need to declare `c` as a reference.

```
for (char &c : str)
    c = std::toupper(c);
```

This is the same as

```
for (std::size_t i = 0; i != str.size(); ++i) {
    char &c = str[i];
    c = std::toupper(c); // Same as `str[i] = std::toupper(str[i]);`.
}
```

### Example: Pass by reference-to-const

Write a function that accepts a string and returns the number of lowercase letters in it:

```
int count_lowercase(std::string str) {
    int cnt = 0;
    for (char c : str)
        if (std::islower(c))
            ++cnt;
    return cnt;
}
```

To call this function:

```
int result = count_lowercase(my_string);
```

## Example: Pass by reference-to-const

```
int count_lowercase(std::string str) {
    int cnt = 0;
    for (char c : str)
        if (std::islower(c))
            ++cnt;
    return cnt;
}
```

```
int result = count_lowercase(my_string);
```

When passing `my_string` to `count_lowercase`, the parameter `str` is initialized as if

```
std::string str = my_string;
```

The contents of the entire string `my_string` are copied!

## Example: Pass by reference-to-const

```
int result = count_lowercase(my_string);
```

When passing `my_string` to `count_lowercase`, the parameter `str` is initialized as if

```
std::string str = my_string;
```

The contents of the entire string `my_string` are copied! Is this copy necessary?

## Example: Pass by reference-to-const

```
int result = count_lowercase(my_string);
```

When passing `my_string` to `count_lowercase`, the parameter `str` is initialized as if

```
std::string str = my_string;
```

The contents of the entire string `my_string` are copied! This copy is unnecessary, because `count_lowercase` is a read-only operation on `str`.

How can we avoid this copy?

## Example: Pass by reference-to-const

```
int count_lowercase(std::string &str) { // `str` is a reference.
    int cnt = 0;
    for (char c : str)
        if (std::islower(c))
            ++cnt;
    return cnt;
}
```

```
int result = count_lowercase(my_string);
```

When passing `my_string` to `count_lowercase`, the parameter `str` is initialized as if

```
std::string &str = my_string;
```

Which is just a reference initialization. No copy is performed.

## Example: Pass by reference-to-`const`

```
int count_lowercase(std::string &str) { // `str` is a reference.
    int cnt = 0;
    for (char c : str)
        if (std::islower(c))
            ++cnt;
    return cnt;
}
```

However, this has a problem:

```
std::string s1 = something(), s2 = some_other_thing();
int result = count_lowercase(s1 + s2); // Error: binding reference to
// a temporary object.
```

`a + b` is a temporary object, which `str` cannot be bound to.

## Example: Pass by reference-to-`const`

References must be bound to existing objects, not literals or temporaries.

**There is an exception to this rule: References-to-`const` can be bound to anything.**

```
const int &rci = 42; // OK.
const std::string &rsc = a + b; // OK.
```

`rsc` is bound to the temporary object returned by `a + b` as if

```
std::string tmp = a + b;
const std::string &rsc = tmp;
```

\$\rightarrow\$ We will talk more about references-to-`const` in recitations.

## Example: Pass by reference-to-`const`

*The* answer:

```
int count_lowercase(const std::string &str) { // `str` is a reference-to-`const`.
    int cnt = 0;
    for (char c : str)
        if (std::islower(c))
            ++cnt;
    return cnt;
}
```

```
std::string a = something(), b = some_other_thing();
int res1 = count_lowercase(a);           // OK.
int res2 = count_lowercase(a + b);       // OK.
int res3 = count_lowercase("hello");     // OK.
```

## Benefits of passing by reference-to-`const`

Apart from the fact that it avoids copy, declaring the parameter as a reference-to-`const` also prevents some potential mistakes:

```
int some_kind_of_counting(const std::string &str, char value) {
    int cnt = 0;
    for (std::size_t i = 0; i != str.size(); ++i) {
        if (str[i] = value) // Oops! It should be `==`.
            ++cnt;
        else {
            // do something ...
            // ...
        }
    }
    return cnt;
}
```

`str[i] = value` will trigger a compile-error, because `str` is a reference-to-`const`.

## Benefits of passing by reference-to-`const`

1. Avoids copy.
2. Accepts temporaries and literals (*rvalues*).
3. The `const` qualification prevents accidental modifications to it.

**[Best practice]** Pass by reference-to-`const` if copy is not necessary and the parameter should not be modified.

## References vs pointers

A reference

- is not itself an object. It is an alias of the object that it is bound to.
- cannot be rebound to another object after initialization.
- has no "default" or "zero" value. It must be bound to an object.

A pointer

- is an object that stores the address of the object it points to.
- can switch to point to another object at any time.
- can be set to a null pointer value `nullptr`.

Both a reference and a pointer can be used to refer to an object, but references are more convenient - no need to write the annoying `*` and `&`.

Note: `nullptr` is *the* null pointer value in C++. Do not use `NULL`.

## `std::vector`

Defined in the standard library file `<vector>`.

A "dynamic array".

## Class template

`std::vector` is a **class template**.

Class templates are not themselves classes. Instead, they can be thought of as instructions to the compiler for *generating* classes.

- The process that the compiler uses to create classes from the templates is called **instantiation**.

For `std::vector`, what kind of class is generated depends on the type of elements we want to store, often called **value type**. We supply this information inside a pair of angle brackets following the template's name:

```
std::vector<int> v; // `v` is of type `std::vector<int>`
```

---

## Create a `std::vector`

`std::vector` is not a type itself. It must be combined with some `<T>` to form a type.

```
std::vector v;           // Error: missing template argument.
std::vector<int> vi;      // An empty vector of `int`s.
std::vector<std::string> vs; // An empty vector of strings.
std::vector<double> vd;    // An empty vector of `double`s.
std::vector<std::vector<int>> vvi; // An empty vector of vector of `int`s.
                                // "2-d" vector.
```

What are the types of `vi`, `vs` and `vvi`?

---

## Create a `std::vector`

`std::vector` is not a type itself. It must be combined with some `<T>` to form a type.

```
std::vector v;           // Error: missing template argument.
std::vector<int> vi;      // An empty vector of `int`s.
std::vector<std::string> vs; // An empty vector of strings.
std::vector<double> vd;    // An empty vector of `double`s.
std::vector<std::vector<int>> vvi; // An empty vector of vector of `int`s.
                                // "2-d" vector.
```

What are the types of `vi`, `vs` and `vvi`?

- `std::vector<int>`, `std::vector<std::string>`, `std::vector<std::vector<int>>`.

---

## Create a `std::vector`

There are several common ways of creating a `std::vector`:

```
std::vector<int> v{2, 3, 5, 7}; // A vector of `int`s,
                                // whose elements are {2, 3, 5, 7}.
std::vector<int> v2 = {2, 3, 5, 7}; // Equivalent to ↑

std::vector<std::string> vs{"hello", "world"}; // A vector of strings,
                                                // whose elements are {"hello", "world"}.
std::vector<std::string> vs2 = {"hello", "world"}; // Equivalent to ↑

std::vector<int> v3(10); // A vector of ten `int`s, all initialized to 0.
std::vector<int> v4(10, 42); // A vector of ten `int`s, all initialized to 42.
```

Note that all the elements in `v3` are initialized to `0`.

- We hate uninitialized values, so does the standard library.

---

## Create a `std::vector`

Create a `std::vector` as a copy of another one:

```
std::vector<int> v{2, 3, 5, 7};
std::vector<int> v2 = v; // `v2` is a copy of `v`
std::vector<int> v3(v); // Equivalent
std::vector<int> v4{v}; // Equivalent
```

### No need to write a loop!

Copy assignment is also enabled:

```
std::vector<int> v1 = something(), v2 = something_else();
v1 = v2;
```

- Element-wise copy is performed automatically.
- Memory is allocated automatically. The memory used to store the old data of `v1` is deallocated automatically.

## C++17 CTAD

"Class Template Argument Deduction": As long as enough information is supplied in the initializer, **the value type can be deduced automatically by the compiler.**

```
std::vector v1{2, 3, 5, 7}; // vector<int>
std::vector v2{3.14, 6.28}; // vector<double>
std::vector v3{10, 42};      // vector<int>, deduced from 42 (int)
std::vector v4{10};         // Error: cannot deduce template argument type
```

## Size of a `std::vector`

`v.size()` and `v.empty()`: same as those on `std::string`.

```
std::vector v{2, 3, 5, 7};
std::cout << v.size() << '\n';
if (v.empty()) {
    // ...
}
```

`v.clear()`: Remove all the elements.

## Append an element to the end of a `std::vector`

`v.push_back(x)`

```
int n;
std::cin >> n;
std::vector<int> v;
for (int i = 0; i != n; ++i) {
    int x;
    std::cin >> x;
    v.push_back(x);
}
std::cout << v.size() << '\n'; // n
```

## Remove the last element of a `std::vector`

`v.pop_back()`

Exercise: Given `v` of type `std::vector<int>`, remove all the consecutive even numbers in the end.

## Remove the last element of a `std::vector`

`v.pop_back()`

Exercise: Given `v` of type `std::vector<int>`, remove all the consecutive even numbers in the end.

```
while (!v.empty() && v.back() % 2 == 0)
    v.pop_back();
```

`v.back()`: returns the *reference* to the last element.

- How is it different from "returning the *value* of the last element"?

## `v.back()` and `v.front()`

Return the references to the last and the first elements, respectively.

It is a **reference**, through which we can modify the corresponding element.

```
v.front() = 42;
++v.back();
```

For `v.back()`, `v.front()` and `v.pop_back()`, **the behavior is undefined** if `v` is empty. They do not perform any bounds checking.

## Range-based for loops

A `std::vector` can also be traversed using a **range-based for** loop.

```
std::vector<int> vi = some_values();
for (int x : vi)
    std::cout << x << std::endl;
std::vector<std::string> vs = some_strings();
for (const std::string &s : vs) // use reference-to-const to avoid copy
    std::cout << s << std::endl;
```

Exercise: Use range-based `for` loops to count the number of uppercase letters in a `std::vector<std::string>`.

## Range-based for loops

Exercise: Use range-based `for` loops to count the number of uppercase letters in a `std::vector<std::string>`.

```
int cnt = 0;
for (const std::string &s : vs) { // Use reference-to-const to avoid copy
    for (char c : s) {
        if (std::isupper(c))
            ++cnt;
    }
}
```

## Access through subscripts

`v[i]` returns the **reference** to the element indexed `i`.

- `i`  $\in [0, N)$ , where  $N = v.size()$ .
- Subscript out of range is **undefined behavior**. `v[i]` performs no bounds checking.
  - In pursuit of efficiency, most operations on standard library containers do not perform bounds checking.
- A kind of "subscript" that has bounds checking: `v.at(i)`.
  - If `i` is out of range, a `std::out_of_range` exception is thrown.

## Feel the style of STL

Basic and low-level operations are performed automatically:

- Default initialization of `std::string` and `std::vector` results in an empty string / container, not indeterminate values.
- Copy of `std::string` and `std::vector` is done automatically, which performs member-wise copy.
- Memory management is done automatically.

Interfaces are consistent:

- `std::string` also has member functions like `.push_back(x)`, `.pop_back()`, `.at(i)`, `.size()`, `.clear()`, etc. which do the same things as on `std::vector`.
- Both can be traversed by range-`for`.

## CS100 Lecture 13

"C" in C++

### Contents

"C" in C++

- Type System
  - Stronger Type Checking
  - Explicit Casts
  - Type Deduction
- Functions
  - Default Arguments
  - Function Overloading
- Range-Based `for` Loops Revisited



## "Better C"

C++ was developed based on C.

From *The Design and Evolution of C++*:

C++ is a general-purpose programming language that

- is a **better C**,
- supports data abstraction,
- supports object-oriented programming.

C++ brought up new ideas and improvements of C, some of which also in turn influenced the development of C.

## "Better C"

- `bool`, `true` and `false` are built-in. No need to `#include <stdbool.h>`. `true` and `false` are of type `bool`, not `int`.
  - This is also true since C23.
- The return type of logical operators `&&`, `||`, `!` and comparison operators `<`, `<=`, `>`, `>=`, `==`, `!=` is `bool`, not `int`.
- The type of string literals `"hello"` is `const char [N+1]`, not `char [N+1]`.
  - Recall that string literals are stored in **read-only memory**. Any attempt to modify them results in undefined behavior.
- The type of character literals `'a'` is `char`, not `int`.

## "Better C"

- `const` variables initialized with literals are compile-time constants. They can be used as the length of arrays.

```
const int maxn = 1000;
int a[maxn]; // a normal array in C++, but VLA in C
```

- `int fun()` declares a function accepting no arguments. It is not accepting unknown arguments.
  - This is also true since C23.

## Type System

### Stronger type checking

Some arithmetic conversions are problematic: They are not value-preserving.

```
int x = some_int_value();
long long y = x; // OK. value-preserving
long long z = some_long_long_value();
int w = z;      // Is this OK?
```

- Conversion from `int` to `long long` is value-preserving, without doubt.
- Conversion from `long long` to `int` may lose precision. ("narrowing")

However, no warning or error is generated for such conversions in C.

### Stronger type checking

Some arithmetic conversions are problematic: They are not value-preserving.

```
long long z = some_long_long_value();
int w = z; // "narrowing" conversion
```

Stroustrup had decided to ban all implicit narrowing conversions in C++. However,

The experiment failed miserably. Every C program I looked at contained large numbers of assignments of `int`s to `char` variables. Naturally, since these were **working programs**, most of these assignments were perfectly safe. That is, either the value was small enough not to become truncated, or the truncation was expected or at least harmless in that particular context.

In the end, narrowing conversions are not banned completely in C++. They are not allowed only in a special context in modern C++. We will see it soon.

## Stronger type checking

Some type conversions (casts) can be very dangerous:

```
const int x = 42, *pci = &x;
int *pi = pci; // Warning in C, Error in C++
++*pi;         // undefined behavior
char *pc = pi; // Warning in C, Error in C++
void *pv = pi; char *pc2 = pv; // Even no warning in C! Error in C++.
int y = pc;    // Warning in C, Error in C++
```

- For `T`  $\neq$  `U`, `T *` and `U *` are different types. Treating a `T *` as `U *` leads to undefined behavior in most cases, but the C compiler gives only a warning!
- `void *` is a hole in the type system. You can cast anything to and from it **without even a warning**.

C++ does not allow the dangerous type conversions to happen *implicitly*.

## Explicit Casts

C++ provides four **named cast operators**:

- `static_cast<Type>(expr)`
- `const_cast<Type>(expr)`
- `reinterpret_cast<Type>(expr)`
- `dynamic_cast<Type>(expr)`  $\rightarrow$  will be covered in later lectures.

In contrast, the C style explicit cast `(Type)expr` looks way too innocent.

"An ugly behavior should have an ugly looking."

### `const_cast`

Cast away low-level constness (**DANGEROUS**):

```
int ival = 42;
const int &cref = ival;
int &ref = cref; // Error: casting away low-level constness
int &ref2 = const_cast<int &>(cref); // OK
int *ptr = const_cast<int *>(&cref); // OK
```

However, modifying a `const` object through a non-`const` access path (possibly formed by `const_cast`) results in **undefined behavior**!

```
const int cival = 42;
int &ref = const_cast<int &>(cival); // compiles, but dangerous
++ref; // undefined behavior (may crash)
```

### `reinterpret_cast`

Often used to perform conversion between different pointer types (**DANGEROUS**):

```
int ival = 42;
char *pc = reinterpret_cast<char *>(&ival);
```

We must never forget that the actual object addressed by `pc` is an `int`, not a character! Any use of `pc` that assumes it's an ordinary character pointer **is likely to fail** at run time, e.g.:

```
std::string str(pc); // undefined behavior
```

**Wherever possible, do not use it!**

### `static_cast`

Other types of conversions (which often look "harmless"):

```
double average = static_cast<double>(sum) / n;
int pos = static_cast<int>(std::sqrt(n));
```

Some typical usage:  $\rightarrow$  We will talk about them in later lectures.

```
static_cast<std::string &&>(str) // converts to a xvalue
static_cast<Derived *>(base_ptr) // downcast without runtime checking
```

## Minimize casting

[Best practice] Minimize casting, (*Effective C++* Item 27)

Type systems work as a **guard** against possible errors: Type mismatch often indicates a logical error.

[Best practice] When casting is necessary, prefer C++-style casts to old C-style casts.

- With old C-style casts, you can't even tell whether it is dangerous or not!

## Type deduction

C++ is very good at **type computations**:

```
std::vector v(10, 42);
```

- It should be `std::vector<int> v(10, 42);`, but the compiler can deduce that `int` from `42`.

```
int x = 42; double d = 3.14; std::string s = "hello";
std::cout << x << d << s;
```

- The compiler can detect the types of `x`, `d` and `s` and select the correct printing functions.

## auto

When declaring a variable with an initializer, we can use the keyword `auto` to let the compiler deduce the type.

```
auto x = 42; // `int`, because 42 is an `int`.
auto y = 3.14; // `double`, because 3.14 is a `double`.
auto z = x + y; // `double`, because the type of `x + y` is `double`.
auto m; // Error: cannot deduce the type. An initializer is needed.
```

`auto` can also be used to produce compound types:

```
auto &r = x; // `int &`, because `x` is an `int`.
const auto &rc = r; // `const int &`.
auto *p = &rc; // `const int *`, because `&rc` is `const int`.
```

## auto

What about this?

```
auto str = "hello";
```

## auto

What about this?

```
auto str = "hello"; // `const char *`
```

- Recall that the type of `"hello"` is `const char [6]`, not `std::string`. This is for compatibility with C.
- When using `auto`, the array-to-pointer conversion ("decay") is performed automatically.

## auto

Deduction of return type is also allowed (since C++14):

```
auto sum(int x, int y) {  
    return x + y;  
}
```

- The return type is deduced to `int`.

Since C++20, `auto` can also be used for function parameters! Such a function is actually a function template.

- This is beyond the scope of CS100.

```
auto sum(auto x, auto y) {  
    return x + y;  
}
```

## auto

`auto` lets us enjoy the benefits of the static type system.

Some types in C++ are very long:

```
std::vector<std::string>::const_iterator it = vs.begin();
```

Use `auto` to simplify it:

```
auto it = vs.begin();
```

## auto

`auto` lets us enjoy the benefits of the static type system.

Some types in C++ are not known to anyone but the compiler:

```
auto lam = [](int x, int y) { return x + y; } // A lambda expression.
```

Every lambda expression has its own type, whose name is only known by the compiler.

## decltype

`decltype(expr)` will deduce the type of the expression `expr` **without evaluating it**.

```
auto fun(int a, int b) { // The return type is deduced to be `int`.  
    std::cout << "fun() is called.\n"  
    return a + b;  
}  
int x = 10, y = 15;  
decltype(fun(x, y)) z; // Same as `int z`;  
                       // Unlike `auto`, no initializer is required here.  
                       // The type is deduced from the return type of `fun`.
```

- `decltype(fun(x, y))` only deduces the return type of `fun` without actually calling it. Therefore, **no output is produced**.

## Note on auto and decltype

The detailed rules of `auto` and `decltype` (as well as their differences) are complicated, and require some deeper understanding of C++ types and templates. You don't have to remember them.

Learn about them mainly through experiments.

- A good IDE should be of great help: Place your mouse on it, and your IDE should tell you the deduction result.

C23 also has `auto` type deduction.

# Functions

## Default arguments

Some functions have parameters that are given a particular value in most, but not all, calls. In such cases, we can declare that common value as a **default argument**.

```
std::string get_screen(std::size_t height = 24, std::size_t width = 80,
                      char background = ' ');
```

- By default, the screen is 24 times 80 filled with ' '.

```
auto default_screen = get_screen();
```

- To override the default arguments:

```
auto large_screen  = get_screen(66);           // 66x80, filled with ' '
auto larger_screen = get_screen(66, 256);      // 66x256, filled with ' '
auto special_screen = get_screen(66, 256, '#'); // 66x256, filled with '#'
```

## Default arguments

Arguments in the call are resolved by position.

```
auto scr = get_screen('#'); // Passing the ASCII value of '#' to `height`.
                        // `width` and `background` are set to
                        // default values (`80` and ` `).
```

- Some other languages have named parameters:

```
print(a, b, sep=" ", end="") # Python
```

There is no such syntax in C++.

Default arguments are only allowed for the last (right-most) several parameters:

```
std::string get_screen(std::size_t height = 24, std::size_t width,
                      char background); // Error.
```

## Function overloading

In C++, a group of functions can have the same name, as long as they can be differentiated when called.

```
int max(int a, int b) {
    return a < b ? b : a;
}
double max(double a, double b) {
    return a < b ? b : a;
}
const char *max(const char *a, const char *b) {
    return std::strcmp(a, b) < 0 ? b : a;
}
```

```
auto x = max(10, 20);           // Calls max(int, int)
auto y = max(3.14, 2.5);        // Calls max(double, double)
auto z = max("hello", "world"); // Calls max(const char *, const char *)
```

## Overloaded functions

Overloaded functions should be distinguished in the way they are called.

```
int fun(int);
double fun(int); // Error: functions that differ only in
                // their return type cannot be overloaded.
```

```
void move_cursor(Coord to);
void move_cursor(int r, int c); // OK, differ in the number of arguments
```

## Overloaded functions

Overloaded functions should be distinguished in the way they are called.

- The following are declaring **the same function**. They are not overloading.

```
void fun(int *);
void fun(int [10]);
```

- The following are the same for an array argument:

```
void fun(int *a);
void fun(int (&a)[10]);
int ival = 42; fun(&ival); // OK, calls fun(int *)
int arr[10];  fun(arr);   // Error: ambiguous call
```

Why?

## Overloaded functions

Overloaded functions should be distinguished in the way they are called.

- The following are the same for an array argument:

```
void fun(int *a);
void fun(int (&a)[10]);
int arr[10];  fun(arr); // Error: ambiguous call
```

- For `fun(int (&)[10])`, this is **an exact match**.
- For `fun(int *)`, this involves an array-to-pointer implicit conversion. We will see that this is **also considered an exact match**.

## Basic overload resolution

Suppose we have the following overloaded functions.

```
void fun(int);
void fun(double);
void fun(int *);
void fun(const int *);
```

Which will be the best match for a call `fun(a)`?

## Basic overload resolution

Suppose we have the following overloaded functions.

```
void fun(int);
void fun(double);
void fun(int *);
void fun(const int *);
```

Obvious: The arguments and the parameters match perfectly.

```
fun(42);    // fun(int)
fun(3.14); // fun(double)
const int arr[10];
fun(arr);  // fun(const int *)
```

Not so obvious:

```
int ival = 42;
// fun(int *) or fun(const int *)?
fun(&ival);
fun('a'); // fun(int) or fun(double)?
fun(3.14f); // fun(int) or fun(double)?
fun(NULL); // fun(int) or fun(int *)?
```

## Basic overload resolution

```
void fun(int);
void fun(double);
void fun(int *);
void fun(const int *);
```

- `fun(&ival)` matches `fun(int *)`
- `fun('a')` matches `fun(int)`
- `fun(3.14f)` matches `fun(double)`
- `fun(NULL)` ? We will see this later.

There are detailed rules that define these behaviors. **But our program should avoid such confusing overload sets.**

## Basic overload resolution

1. An exact match, including the following cases:
  - identical types
  - **match through decay of array** (*or function*) **type**
  - match through top-level `const` conversion
2. **Match through adding low-level** `const`
3. Match through [integral or floating-point promotion](#)
4. Match through [numeric conversion](#)
5. Match through a class-type conversion (in later lectures).

No need to remember all the details. But pay attention to some cases that are very common.

## The null pointer

`NULL` is a **macro** defined in standard library header files.

- In C, it may be defined as `(void *)0`, `0`, `(long)0` or other forms.

In C++, `NULL` cannot be `(void *)0` since the implicit conversion from `void *` to other pointer types is **not allowed**.

- It is most likely to be an integer literal with value zero.
- With the following overload declarations, `fun(NULL)` may call `fun(int)` on some platforms, and may be **ambiguous** on other platforms!

```
void fun(int);
void fun(int *);
```

```
fun(NULL); // May call fun(int),
           // or may be ambiguous.
```

---

## Better null pointer: `nullptr`

---

In short, `NULL` is a "fake" pointer.

Since C++11, a better null pointer is introduced: `nullptr` (also available in C23)

- `nullptr` has a unique type `std::nullptr_t` (defined in `<cstddef>`), which is neither `void *` nor an integer.
- `fun(nullptr)` will definitely match `fun(int *)`.

```
void fun(int);
void fun(int *);
```

```
fun(NULL); // May call fun(int),
           // or may be ambiguous.
fun(nullptr); // Calls fun(int *).
```

[Best practice] Use `nullptr` as the null pointer constant in C++.

---

## Avoid abuse of function overloading

---

Only overload operations that actually do similar things. A bad example:

```
Screen &moveHome(Screen &);
Screen &moveAbs(Screen &, int, int);
Screen &moveRel(Screen &, int, int, std::string direction);
```

If we overload this set of functions under the name `move`, some information is lost.

```
Screen &move(Screen &);
Screen &move(Screen &, int, int);
Screen &move(Screen &, int, int, std::string direction);
```

Which one is easier to understand?

```
moveHome(scrn); // OK, moves to home.
```

```
move(scrn); // Unclear: How to move?
```

---

## Range-based `for` loops revisited

---

### Range-based `for` loops

---

Traverse a `std::string`

```
int str_to_int(const std::string &str) {
    int value = 0;
    for (auto c : str) // char
        value = value * 10 + c - '0';
    return value;
}
```

Note: This function can be replaced by `std::stoi`.



---

## Range-based `for` loops

Traverse a `std::vector`

```
bool is_all_digits(const std::string &str) {
    for (auto c : str)
        if (!std::isdigit(c))
            return false;
    return true;
}

int count_numbers(const std::vector<std::string> &strs) {
    int cnt = 0;
    for (const auto &s : strs) // const std::string &s
        if (is_all_digits(s))
            ++cnt;
    return cnt;
}
```

---

## Traverse an array

An array can also be traversed by range- `for` :

```
int arr[100] = {}; // OK in C++ and C23.
// The following loop will read 100 integers.
for (auto &x : arr) // int &
    std::cin >> x;
```

- Note: The range-based `for` loop will traverse **the entire array**.

What else can be traversed using a range- `for` ?  $\rightarrow$  We will learn about this when introducing **iterators**.

---

## Pass an array by reference

```
void print(int *arr) {
    for (auto x : arr) // Error: `arr` is a pointer, not an array.
        std::cout << x << ' ';
    std::cout << '\n';
}
```

We can declare `arr` to be a **reference to array**:

```
void print(const int (&arr)[100]) {
    for (auto x : arr) // OK. `arr` is an array.
        std::cout << x << ' ';
    std::cout << '\n';
}
```

- `arr` is of type `const int (&)[100]` : a reference to an array of `100` elements, where each element is of type `const int`.

---

## Pass an array by reference

We can declare `arr` to be a **reference to array**:

```
void print(const int (&arr)[100]) {
    for (auto x : arr) // OK. `arr` is an array.
        std::cout << x << ' ';
    std::cout << '\n';
}
```

- `arr` is of type `const int (&)[100]` : a reference to an array of `100` elements, where each element is of type `const int`.

Note that only arrays of `100 int`s can fit here.

```
int a[100] = {}; print(a); // OK.
int b[101] = {}; print(b); // Error.
double c[100] = {}; print(c); // Error.
```

---

## Pass an array by reference

To allow arrays of any type, any length: Use a template function.

```
template <typename Type, std::size_t N>
void print(const Type (&arr)[N]) {
    for (const auto &x : arr)
        std::cout << x << ' ';
    std::cout << '\n';
}
```

We will learn about this in the end of this semester.

## CS100 Lecture 14

Class Basics I

---

### Contents

Class basics

- Members of a class
  - Access
  - The `this` pointer
- Constructors
  - Constructor initializer list
  - Default constructors

---

## Members of a class

### A simple `class`

The initial idea: A `class` is a new kind of `struct` that can have member functions:

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;
    void setName(const std::string &newName) {
        name = newName;
    }
    void printInfo() const {
        std::cout << "I am " << name << ", id " << id
                    << ", entrance year: " << entranceYear << std::endl;
    }
    bool graduated(int year) const {
        return year - entranceYear >= 4;
    }
};
```

---

### Member access

Member access: `a.mem`, where `a` is an **object** of the class type.

- Every member `{}^{\textcolor{red}{1}}{}` belongs to an object: each student has a name, id, entrance year, etc.
  - You need to specify *whose* name / id / ... you want to obtain.

To call a member function on an object: `a.memfun(args)`.

```
Student s = someValue();
s.printInfo(); // call its printInfo() to print related info
if (s.graduated(2023)) {
    // ...
}
```

## Access control

```
class Student {
private:
    std::string name;
    std::string id;
    int entranceYear;
public:
    void setName(const std::string &newName) { name = newName; }
    void printInfo() const {
        std::cout << "I am " << name << ", id " << id
                    << ", entrance year: " << entranceYear << std::endl;
    }
    bool graduated(int year) const { return year - entranceYear >= 4; }
};
```

- `private` members: Only accessible to code inside the class and `friend`s.
  - $\rightarrow$  We will introduce `friend`s in later lectures.
- `public` members: Accessible to all parts of the program.

## Access control

```
class Student {
private:
    std::string name;
    std::string id;
    int entranceYear;

public:
    void setName(const std::string &newName);
    void printInfo() const;
    bool graduated(int year) const;
};
```

Unlike some other languages (e.g. Java), an access specifier controls the access of all members after it, until the next access specifier or the end of the class definition.

## Access control

```
class Student {
// private:
    std::string name;
    std::string id;
    int entranceYear;
public:
    void setName(const std::string &newName);
    void printInfo() const;
    bool graduated(int year) const;
};
```

What if there is a group of members with no access specifier at the beginning?

- If it's `class`, they are `private`.
- If it's `struct`, they are `public`.

This is one of the **only two differences** between `struct` and `class` in C++.

## The `this` pointer

```
class Student {
    // ...
public:
    bool graduated(int year) const;
};

Student s = someValue();
if (s.graduated(2023))
    // ...
```

How many parameters does `graduated` have?

## The `this` pointer

```
class Student {
    // ...
public:
    bool graduated(int year) const;
};

Student s = someValue();
if (s.graduated(2023)) // ...
```

How many parameters does `graduated` have?

- **Seemingly one, but actually two:** `s` is also information that must be known when calling this function!

## The `this` pointer

```
class Student {
public:
    void setName(const std::string &n) {
        name = n;
    }

    bool graduated(int year) const {
        return year - entranceYear >= 4;
    }
};

Student s = someValue();
if (s.graduated(2023))
    // ...
s.setName("Alice");
```

- The code on the left can be viewed as:

```
void setName
(Student *this, const std::string &n) {
    this->name = n;
}

bool graduated
(const Student *this, int year) {
    return year - this->entranceYear >= 4;
}

Student s = someValue();
if (graduated(&s, 2023))
    // ...
setName(&s, "Alice");
```

## The `this` pointer

There is a pointer called `this` in each member function of class `X` which has type `X *` or `const X *`, pointing to the object on which the member function is called.

Inside a member function, access of any member `mem` is actually `this->mem`.

We can also write `this->mem` explicitly.

```
class Student {
public:
    bool graduated(int year) const {
        return year - this->entranceYear >= 4;
    }
};
```

Many languages have similar constructs, e.g. `self` in Python. ([C++23 has `self` too!](#))

## `const` member functions

The `const` keyword after the parameter list and before the function body `{` is used to declare a `const` **member function**.

- A `const` member function cannot modify its data members `{}^{\textcolor{red}{2}}$`.
- A `const` member function **guarantees** that no data member will be modified.
  - A non-`const` member function does not provide such guarantee.
  - In a `const` member function, calling a non-`const` member function on `*this` is not allowed.
- For a `const` object, **only `const` member functions can be called on it**.

**[Best practice]** If, logically, a member function should not modify the object's state, it should be made a `const` member function. Otherwise, it cannot be called on `const` objects.

---

## `const` member functions and the `this` pointer

This `const` is essentially applied to the `this` pointer:

- In `const` member functions of class `X`, `this` has type `const X *`.
- In non-`const` member functions of class `X`, `this` has type `X *`.

If `ptr` is of type `const T *`, the expression `ptr->mem` is also `const`-qualified.

- Recall that in a member function, access of a member `mem` is actually `this->mem`.
- Therefore, `mem` is also `const`-qualified in a `const` member function.

```
class Student {
public:
    void foo() const {
        name += 'a'; // Error: `name` is `const std::string` in a const member
                    // function. It cannot be modified.
    }
};
```

---

## `const` member functions

*Effective C++* Item 3: Use `const` whenever possible.

Decide whether the following member functions need a `const` qualification:

```
class Student {
    std::string name, id;
    int entranceYear;
public:
    const std::string &getName(); // returns the name of the student.
    const std::string &getID();   // returns the id of the student.
    bool valid();               // verifies whether the leading four digits in `id`
                               // is equal to `entranceYear`.
    void adjustID(); // adjust `id` according to `entranceYear`.
};
```

---

## `const` member functions

*Effective C++* Item 3: Use `const` whenever possible.

Decide whether the following member functions need a `const` qualification:

```
class Student {
    std::string name, id;
    int entranceYear;
public:
    const std::string &getName() const; // returns the name of the student.
    const std::string &getID() const;   // returns the id of the student.
    bool valid() const;               // verifies whether the leading four digits in `id`
                               // is equal to `entranceYear`.
    void adjustID(); // adjust `id` according to `entranceYear`.
};
```

The `const` ness of member functions should be determined **logically**.

---

## `const` member functions

```
class Student {
    std::string name, id;
    int entranceYear;
public:
    const std::string &getName() const { return name; }
    const std::string &getID() const { return id; }
    bool valid() const { return id.substr(0, 4) == std::to_string(entranceYear); }
    void adjustID() { id = std::to_string(entranceYear) + id.substr(4); }
};
```

`str.substr(pos, len)` returns the substring of `str` starting from the position indexed `pos` with length `len`.

- If `len` is not provided, it returns the **suffix** starting from the position indexed `pos`.

## Constructors

Often abbreviated as "ctors".

## Constructors

**Constructors** define how an object can be initialized.

- Constructors are often **overloaded**, because an object may have multiple reasonable ways of initialization.

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;
public:
    Student(const std::string &name_, const std::string &id_, int ey)
        : name(name_), id(id_), entranceYear(ey) {}
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};

Student a("Alice", "2020123123", 2020);
Student b("Bob", "2020123124"); // entranceYear = 2020
Student c; // Error: No default constructor. (to be discussed later)
```

## Constructors

```
class Student {
    std::string name;
    std::string id;
    int entranceYear;

public:
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};
```

- The constructor name is the class name: `Student`.
- Constructors do not have a return type (not even `void` `void ${}^{\textcolor{red}{3}}$`). The constructor body can contain a `return;` statement, which should not return a value.
- The function body of this constructor is empty: `{}`.

## Constructor initializer list

Constructors initialize **all data members** of the object.

The initialization of **all data members** is done **before entering the function body**.

How they are initialized is (partly) determined by the **constructor initializer list**:

```
class Student {
    // ...
public:
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}
};
```

The initializer list starts with `:`, and contains initializers for each data member, separated by `,`. The initializers must be of the form `(...)` or `{...}`, not `= ...`.

## Order of initialization

Data members are initialized in order **in which they are declared**, not the order in the initializer list.

- If the initializers appear in an order different from the declaration order, the compiler will generate a warning.

Typical mistake: `entranceYear` is initialized in terms of `id`, but `id` is not initialized yet!

```
class Student {
    std::string name;
    int entranceYear; // !!!
    std::string id;

public:
    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id.substr(0, 4))) {}
};
```

## Constructor initializer list

Data members are initialized in order **in which they are declared**, not the order in the initializer list.

- If the initializers appear in an order different from the declaration order, the compiler will generate a warning.
- For a data member that do not appear in the initializer list:
  - If there is an **in-class initializer** (see next page), it is initialized using the in-class initializer.
  - Otherwise, it is **default-initialized**.

What does **default-initialization** mean for class types?  $\rightarrow$  To be discussed later.

## In-class initializers

A member can have an in-class initializer. It must be in the form `{...}` or `= ...`.<sup>[1](#)</sup>

```
class Student {
    std::string name = "Alice";
    std::string id;
    int entranceYear{2024}; // equivalent to `int entranceYear = 2024;`.
public:
    Student() {} // `name` is initialized to `\"Alice\"`,
                // `id` is initialized to an empty string,
                // and `entranceYear` is initialized to 2024.
    Student(int ey) : entranceYear(ey) {} // `name` is initialized to `\"Alice\"`,
                                        // `id` is initialized to an empty string,
                                        // and `entranceYear` is initialized to `ey`.
};
```

The in-class initializer provides the "default" way of initializing a member in this class, as a substitute for default-initialization.

## Constructor initializer list

Below is a typical way of writing this constructor without an initializer list:

```
class Student {
    // ...
public:
    Student(const std::string &name_, const std::string &id_) {
        name = name_;
        id = id_;
        entranceYear = std::stoi(id_.substr(0, 4));
    }
};
```

How are these members actually initialized in this constructor?

## Constructor initializer list

Below is a typical way of writing this constructor without an initializer list:

```

class Student {
    // ...
public:
    Student(const std::string &name_, const std::string &id_) {
        name = name_;
        id = id_;
        entranceYear = std::stoi(id_.substr(0, 4));
    }
};

```

How are these members actually initialized in this constructor?

- First, before entering the function body, `name`, `id` and `entranceYear` are default-initialized. `name` and `id` are initialized to empty strings.
- Then, the assignments in the function body take place.

## Constructor initializer list

**[Best practice]** Always use an initializer list in a constructor.

- Not all types can be default-initialized. Not all types can be assigned to. (Any counterexamples?)

## Constructor initializer list

**[Best practice]** Always use an initializer list in a constructor.

Not all types can be default-initialized. Not all types can be assigned to.

- References `T &` cannot be default-initialized, and cannot be assigned to.
- `const` objects of built-in types cannot be default-initialized.
- `const` objects cannot be assigned to.
- A class can choose to allow or disallow default initialization or assignment. It depends on the design.  $\rightarrow$  See next page.

Moreover, if a data member is default-initialized and then assigned when could have been initialized directly, it may lead to low efficiency.

## Default constructors

A special constructor that takes no parameters.

- Guess what it's for?

## Default Constructors

A special constructor that takes no parameters.

- It defines the behavior of **default-initialization** of objects of that class type, since no arguments need to be passed when calling it.

```

class Point2d {
    double x, y;
public:
    Point2d() : x(0), y(0) {} // default constructor
    Point2d(double x_, double y_) : x(x_), y(y_) {}
};

Point2d p1;           // calls default ctor, (0, 0)
Point2d p2(3, 4);     // calls Point2d(double, double), (3, 4)
Point2d p3();         // Is this calling the default ctor?

```

## Default constructors

A special constructor that takes no parameters.

- It defines the behavior of **default-initialization** of objects of that class type, since no arguments need to be passed when calling it.



```

class Point2d {
    double x, y;
public:
    Point2d() : x(0), y(0) {} // default constructor
    Point2d(double x_, double y_) : x(x_), y(y_) {}
};

Point2d p1;           // calls default ctor, (0, 0)
Point2d p2(3, 4);     // calls Point2d(double, double), (3, 4)
Point2d p3();         // Is this calling the default ctor?

```

Be careful! `p3` is a **function** that takes no parameters and returns `Point2d`.

## Is a default constructor needed?

First, if you need to use arrays, you almost certainly need a default constructor:

```

Student s[1000]; // All elements are default-initialized
               // by the default constructor.
Student s2[1000] = {a, b}; // The first two elements are initialized to
                          // `a` and `b`. The rest are initialized by the
                          // default constructor.

```

A `std::vector` does not require that:

```

// In this code, the default constructor of `Student` is not called.
std::vector<Student> students;
for (auto i = 0; i != n; ++i)
    students.push_back(some_student());

```

## Is a default constructor needed?

If a class has no user-declared constructors, the compiler will try to synthesize a default constructor.

```

class X {}; // No user-declared constructors.
X x; // OK: calls the compiler-synthesized default constructor

```

The synthesized default constructor initializes the data members as follows:

- If a data member has an in-class initializer, it is initialized according to the in-class initializer.
- Otherwise, default-initialize that member. If it cannot be default-initialized, the compiler will give up -- no default constructor is generated.

## Is a default constructor needed?

If a class has any user-declared constructors but no default constructor, the compiler **will not** synthesize a default constructor.

You may ask for a default constructor with `= default;`:

```

class Student {
public:
    Student(const std::string &name_, const std::string &id_, int ey)
        : name(name_), id(id_), entranceYear(ey) {}

    Student(const std::string &name_, const std::string &id_)
        : name(name_), id(id_), entranceYear(std::stoi(id_.substr(0, 4))) {}

    Student() = default;
};

```

## Is a default constructor needed?

It depends on the **design**:

- If the class has a default constructor, what should be the behavior of it? Is there a reasonable "default state" for your class type?

For `Student`: What is a "default student"?

## Is a default constructor needed?

It depends on the **design**:

- If the class has a default constructor, what should be the behavior of it? Is there a reasonable "default state" for your class type?

For `Student`: What is a "default student"?

- There seems to be no such thing as a "default student" (in a normal design). Therefore, `Student` should not have a default constructor.

## Is a default constructor needed?

**[Best practice] When in doubt, leave it out.** If the class does not have a "default state", it should not have a default constructor!

- Do not define one arbitrarily or letting it `= default`. This leads to pitfalls.
- Calling the default constructor of something that has no "default state" should result in a **compile error**, instead of being allowed arbitrarily.

## Summary

Members of a class

- A class can have data members and member functions.
- Access control: `private`, `public`.
  - One difference between `class` and `struct`: Default access.
- The `this` pointer: has type `X *` (`const X *` in `const` member functions). It points to the object on which the member function is called.
- `const` member function: guarantees that no modification will happen.

## Summary

The followings hold for **all constructors**, no matter how they are defined:

- A constructor initializes **all** data members in order in which they are declared.
- The initialization of **all** data members is done before the function body of a constructor is executed.

In a constructor, a member is initialized as follows:

- If there is an initializer for it in the initializer list, use it.
- Otherwise, if it has an in-class initializer, use it.
- Otherwise, it is default-initialized. If it cannot be default-initialized, it leads to a compile-error.

## Summary

Default constructors

- The default constructor defines the behavior of default-initialization.
- The default constructor is the constructor with an empty parameter list.
- If we have not defined **any constructor**, the compiler will try to synthesize a **default constructor** as if it were defined as `ClassName() {}`.
  - The compiler may fail to do that if some member has no in-class initializer and is not default-initializable. In that case, the compiler gives up (without giving an error).
- We can use `= default` to ask for a synthesized default constructor explicitly.

## CS100 Lecture 15

Constructors, Destructors, Copy Control

## Contents

- Constructors and destructors
- Copy control

## Constructors and destructors

## Lifetime of an object

**Lifetime** of a local non-`static` object:

- Starts on initialization
- Ends when control flow goes out of its **scope**

```
for (int i = 0; i != n; ++i) {
    do_something(i);
    // Lifetime of `s` begins.
    std::string s = some_string();
    do_something_else(s, i);
    /* end of lifetime of `s` */ }
```

Every time the loop body is executed, `s` undergoes initialization and destruction.

- `std::string` **owns** some resources (memory where the characters are stored).
- `std::string` must *somehow* release that resources (deallocate that memory) at the end of its lifetime.

---

## Lifetime of an object

Lifetime of a global object:

- Starts on initialization (before the first statement of `main`)
- Ends when the program terminates.

Lifetime of a heap-based object:

- Starts on initialization: **A `new` expression will do this, but `malloc` does not!**
- Ends when it is destroyed: **A `delete` expression will do this, but `free` does not!**

\$\rightarrow\$ `new` / `delete` expressions are in this week's recitation.

---

## Constructors and Destructors

Take `std::string` as an example:

- Its initialization (done by its constructors) must allocate some memory for its content.
- When it is destroyed, it must *somehow* deallocate that memory.

---

## Constructors and Destructors

Take `std::string` as an example:

- Its initialization (done by its constructors) must allocate some memory for its content.
- When it is destroyed, it must *somehow* deallocate that memory.

**A destructor of a class is the function that is automatically called when an object of that class type is destroyed.**

---

## Constructors and Destructors

Syntax: `~ClassName() { /* ... */ }`

```
struct A {
    A() {
        std::cout << 'c';
    }
    ~A() {
        std::cout << 'd';
    }
};
```

```
for (int i = 0; i != 3; ++i) {
    A a;
    // do something ...
}
```

Output:

```
cdcdcd
```

## Destructor

Called **automatically** when the object is destroyed!

- How can we make use of this property?

## Destructor

Called **automatically** when the object is destroyed!

- How can we make use of this property?

We often do some **cleanup** in a destructor:

- If the object **owns some resources** (e.g. dynamic memory), destructors can be made use of to avoid leaking!

```
class A {
    SomeResourceHandle resource;

public:
    A(/* ... */) : resource(Obtain_resource(/* ... */)) {}
    ~A() {
        release_resource(resource);
    }
};
```

## Example: A dynamic array

Suppose we want to implement a "dynamic array":

- It looks like a VLA (variable-length array), but it is heap-based, which is safer.
- It should take good care of the memory it uses.

Expected usage:

```
int n; std::cin >> n;
Dynarray arr(n); // `n` is runtime determined
                // `arr` should have allocated memory for `n` `int`s now.
for (int i = 0; i != n; ++i) {
    int x; std::cin >> x;
    arr.at(i) = x * x; // subscript, looks as if `arr[i] = x * x`
}
// ...
// `arr` should deallocate its memory itself.
```

## Dynarray: members

- It should have a pointer that points to the memory, where elements are stored.
- It should remember its length.

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
};
```

- `m` stands for **member**.

**[Best practice]** Make data members `private`, to achieve good encapsulation.

## Dynarray: constructors

- We want `Dynarray a(n);` to construct a `Dynarray` that contains `n` elements.
  - To avoid troubles, we want the elements to be **value-initialized**!
    - **Value-initialization** is like "empty-initialization" in C. (In this week's recitation.)
  - `new int[n]{};`  Allocate a block of heap memory that stores `n` `int`s, and value-initialize them.
- Do we need a default constructor?
  - Review: What is a default constructor?
    - The constructor with no parameters.
  - What should be the correct behavior of it?

---

## Dynarray: constructors

- We want `dynarray a(n);` to construct a `dynarray` that contains `n` elements.
  - To avoid troubles, we want the elements to be **value-initialized**!
- Suppose we don't want a default constructor.

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray(std::size_t n) : m_storage(new int[n]{}), m_length(n) {}
};
```

If the class has a user-declared constructor, the compiler will not generate a default constructor.

---

## Dynarray: constructors

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray(std::size_t n) : m_storage(new int[n]{}), m_length(n) {}
};
```

Since `dynarray` has a user-declared constructor, it does not have a default constructor:

```
dynarray a; // Error.
```

---

## Dynarray: destructor

- Remember: The destructor is (automatically) called when the object is "dead".
- The memory is obtained in the constructor, and released in the destructor.

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray(std::size_t n)
        : m_storage(new int[n]{}), m_length(n) {}
    ~Dynarray() {
        delete[] m_storage; // Pay attention to `[]`!
    }
};
```

---

## Dynarray: destructor

Is this correct?

```
class Dynarray {
    // ...
    ~Dynarray() {
        if (m_length != 0)
            delete[] m_storage;
    }
};
```

**NO!** `new [0]` may also allocate some memory (implementation-defined, like `malloc`), which should also be deallocated.

---

## Dynarray: destructor

Is this correct?

```
class Dynarray {
// ...
~Dynarray() {
    delete[] m_storage;
    m_length = 0;
}
};
```

It is correct, but `m_length = 0;` is not needed. The destructor is executed **right before the** `Dynarray` **object "dies"**, so the value of `m_length` does not matter!

## Dynarray: some member functions

Design some useful member functions.

- A function to obtain its length (size).
- A function telling whether it is empty.

```
class Dynarray {
// ...
public:
    std::size_t size() const {
        return m_length;
    }
    bool empty() const {
        return m_length == 0;
    }
};
```

## Dynarray: some member functions

Design some useful member functions.

- A function returning **reference** to an element.

```
class Dynarray {
// ...
public:
    int &at(std::size_t i) {
        return m_storage[i];
    }
    const int &at(std::size_t i) const {
        return m_storage[i];
    }
};
```

Why do we need this "`const`" vs non-`const`" overloading?  $\rightarrow$  Learn it in recitations.

## Dynarray: Usage

```
void print(const Dynarray &a) {
    for (std::size_t i = 0;
         i != a.size(); ++i)
        std::cout << a.at(i) << ' ';
    std::cout << std::endl;
}

void reverse(Dynarray &a) {
    for (std::size_t i = 0,
         j = a.size() - 1; i < j; ++i, --j)
        std::swap(a.at(i), a.at(j));
}
```

```
int main() {
    int n; std::cin >> n;
    Dynarray array(n);
    for (int i = 0; i != n; ++i)
        std::cin >> array.at(i);
    reverse(array);
    print(array);
    return 0;
    // Dtor of `array` is called here,
    // which deallocates the memory
}
```

## Copy control

### Copy-initialization

We can easily construct a `std::string` to be a copy of another:

```
std::string s1 = some_value();
std::string s2 = s1; // s2 is initialized to be a copy of s1
std::string s3(s1); // equivalent
std::string s4{s1}; // equivalent, but modern
```

Can we do this for our `Dynarray`?

### Copy-initialization

Before we add anything, let's try what will happen:

```
Dynarray a(3);
a.at(0) = 2; a.at(1) = 3; a.at(2) = 5;
Dynarray b = a; // It compiles.
print(b); // 2 3 5
a.at(0) = 70;
print(b); // 70 3 5
```

Ooops! Although it compiles, the pointers `a.m_storage` and `b.m_storage` are pointing to the same address!

### Copy-initialization

Before we add anything, let's try what will happen:

```
Dynarray a(3);
Dynarray b = a;
```

Although it compiles, the pointers `a.m_storage` and `b.m_storage` are pointing to the same address!

This will cause disaster: consider the case if `b` "dies" before `a`:

```
Dynarray a(3);
if (some_condition) {
    Dynarray b = a; // `a.m_storage` and `b.m_storage` point to the same memory!
    // ...
} // At this point, dtor of `b` is invoked, which deallocates the memory.
std::cout << a.at(0); // Invalid memory access!
```

### Copy constructor

Let `a` be an object of type `Type`. The behaviors of **copy-initialization** (in one of the following forms)

```
Type b = a;
Type b(a);
Type b{a};
```

are determined by a constructor: **the copy constructor**.

- Note! The `=` in `Type b = a;` **is not an assignment operator!**

---

## Copy constructor

The copy constructor of a class `X` has a parameter of type `const X &`:

```
class Dynarray {
public:
    Dynarray(const Dynarray &other);
};
```

Why `const`?

- Logically, it should not modify the object being copied.

Why `&`?

- **Avoid copying.** Pass-by-value is actually **copy-initialization** of the parameter, which will cause infinite recursion here!

---

## Dynarray: copy constructor

What should be the correct behavior of it?

```
class Dynarray {
public:
    Dynarray(const Dynarray &other);
};
```

---

## Dynarray: copy constructor

- We want a copy of the content of `other`.

```
class Dynarray {
public:
    Dynarray(const Dynarray &other)
        : m_storage(new int[other.size()]), m_length(other.size()) {
        for (std::size_t i = 0; i != other.size(); ++i)
            m_storage[i] = other.at(i);
    }
};
```

Now the copy-initialization of `Dynarray` does the correct thing:

- The new object allocates a new block of memory.
- The **contents** are copied, not just the address.

---

## Synthesized copy constructor

If the class does not have a user-declared copy constructor, the compiler will try to synthesize one:

- The synthesized copy constructor will **copy-initialize** all the members, as if

```
class Dynarray {
public:
    Dynarray(const Dynarray &other)
        : m_storage(other.m_storage), m_length(other.m_length) {}
};
```

- If the synthesized copy constructor does not behave as you expect, **define it on your own!**

---

## Defaulted copy constructor

If the synthesized copy constructor behaves as we expect, we can explicitly require it:



```
class Dynarray {
public:
    Dynarray(const Dynarray &) = default;
    // Explicitly defaulted: Explicitly requires the compiler to synthesize
    // a copy constructor, with default behavior.
};
```

## Deleted copy constructor

What if we don't want a copy constructor?

```
class ComplicatedDevice {
    // some members
    // Suppose this class represents some complicated device,
    // for which there is no correct and suitable behavior for "copying".
};
```

Simply not defining the copy constructor does not work:

- The compiler will synthesize one for you.

## Deleted copy constructor

What if we don't want a copy constructor?

```
class ComplicatedDevice {
    // some members
    // Suppose this class represents some complicated device,
    // for which there is no correct and suitable behavior for "copying".
public:
    ComplicatedDevice(const ComplicatedDevice &) = delete;
};
```

By saying `= delete`, we define a **deleted** copy constructor:

```
ComplicatedDevice a = something();
ComplicatedDevice b = a; // Error: calling deleted function
```

## Copy-assignment operator

Apart from copy-initialization, there is another form of copying:

```
std::string s1 = "hello", s2 = "world";
s1 = s2; // s1 becomes a copy of s2, representing "world"
```

In `s1 = s2`, `=` is the **assignment operator**.

`=` is the assignment operator **only when it is in an expression**.

- `s1 = s2` is an expression.
- `std::string s1 = s2` is in a **declaration statement**, not an expression. `=` here is a part of the initialization syntax.

## Dynarray: copy-assignment operator

The copy-assignment operator is defined in the form of **operator overloading**:

- `a = b` is equivalent to `a.operator=(b)`.
- We will talk about more on operator overloading in a few weeks.

```
class Dynarray {
public:
    Dynarray &operator=(const Dynarray &other);
};
```

- The function name is `operator=`.
- In consistent with built-in assignment operators, `operator=` returns **reference to the left-hand side object** (the object being assigned).
  - It is `*this`.

---

## Dynarray: copy-assignment operator

We also want the copy-assignment operator to copy the contents, not only an address.

```
class Dynarray {
public:
    Dynarray &operator=(const Dynarray &other) {
        m_storage = new int[other.size()];
        for (std::size_t i = 0; i != other.size(); ++i)
            m_storage[i] = other.at(i);
        m_length = other.size();
        return *this;
    }
};
```

Is this correct?

---

## Dynarray: copy-assignment operator

**Avoid memory leaks! Deallocate the memory you don't use!**

```
class Dynarray {
public:
    Dynarray &operator=(const Dynarray &other) {
        delete[] m_storage; // !!!
        m_storage = new int[other.size()];
        for (std::size_t i = 0; i != other.size(); ++i)
            m_storage[i] = other.at(i);
        m_length = other.size();
        return *this;
    }
};
```

Is this correct?

---

## Dynarray: copy-assignment operator

What if **self-assignment** happens?

```
class Dynarray {
public:
    Dynarray &operator=(const Dynarray &other) {
        // If 'other' and '*this' are actually the same object,
        // the memory is deallocated and the data are lost! (DISASTER)
        delete[] m_storage;
        m_storage = new int[other.size()];
        for (std::size_t i = 0; i != other.size(); ++i)
            m_storage[i] = other.at(i);
        m_length = other.size();
        return *this;
    }
};
```

---

## Dynarray: copy-assignment operator

Assignment operators should be **self-assignment-safe**.

```

class Dynarray {
public:
    Dynarray &operator=(const Dynarray &other) {
        int *new_data = new int[other.size()];
        for (std::size_t i = 0; i != other.size(); ++i)
            new_data[i] = other.at(i);
        delete[] m_storage;
        m_storage = new_data;
        m_length = other.size();
        return *this;
    }
};

```

This is self-assignment-safe. (Think about it.)

## Synthesized, defaulted and deleted copy-assignment operator

Like the copy constructor:

- The copy-assignment operator can also be **deleted**, by declaring it as `= delete`;
- If you don't define it, the compiler will generate one that copy-assigns all the members, as if it is defined as:

```

class Dynarray {
public:
    Dynarray &operator=(const Dynarray &other) {
        m_storage = other.m_storage;
        m_length = other.m_length;
        return *this;
    }
};

```

- You can also require a synthesized one explicitly by saying `= default`;

## [IMPORTANT] The rule of three: Reasoning

Among the **copy constructor**, the **copy-assignment operator** and the **destructor**:

- If a class needs a user-provided version of one of them, **usually**, it needs a user-provided version of **each** of them.
- Why?

## [IMPORTANT] The rule of three: Reasoning

Among the **copy constructor**, the **copy-assignment operator** and the **destructor**:

- If a class needs a user-provided version of one of them,
- **usually**, it is a class that **manages some resources**,
- for which **the default behavior of the copy-control members does not suffice**.
- Therefore, all of the three special functions need a user-provided version.
  - Define them in a correct, well-defined manner.
  - If a class should not be copy-constructible or copy-assignable, **delete that function**.

## [IMPORTANT] The rule of three: Rules

Let  $S = \{\text{copy constructor}, \text{copy assignment operator}, \text{destructor}\}$ .

If for a class,  $\exists x, y \in S$  such that

- $x$  is user-declared, and  $y$  is not user-declared,

then the compiler *should not* generate  $y$ , according to the idea of "the rule of three".

## [IMPORTANT] The rule of three: Rules

Let  $S = \{\text{copy constructor}, \text{copy assignment operator}, \text{destructor}\}$ .

If for a class,  $\exists x, y \in S$  such that

- $x$  is user-declared, and  $y$  is not user-declared,

then the compiler **still generates  $y$** , but **this behavior has been deprecated since C++11**.

- This is a problem left over from history: At the time C++98 was adopted, the significance of the rule of three was not fully appreciated.

---

## [IMPORTANT] The rule of three

Into modern C++: **The Rule of Five**.

- $\rightarrow$  We will talk about it in later lectures.

Read *Effective Modern C++* Item 17 for a thorough understanding of this.

---

## Summary

Lifetime of an object:

- depends on its **storage**: local non-`static`, global, allocated, ...
- **Initialization** marks the beginning of the lifetime of an object.
  - Classes can control the way of initialization using **constructors**.
- When the lifetime of an object ends, it is **destroyed**.
  - If it is an object of class type, its **destructor** is called right before it is destroyed.

---

## Summary

Copy control

- Usually, the **copy control members** refer to the copy constructor, the copy assignment operator and the destructor.
- Copy constructor: `ClassName(const ClassName &)`
- Copy assignment operator: `ClassName &operator=(const ClassName &)`
  - It needs to be **self-assignment safe**.
- Destructor: `~ClassName()`
- `=default`, `=delete`
- The rule of three.

---

# CS100 Lecture 16

Class Basics II

---

## Contents

- Type alias members
- `static` members
- `friend`
- Definition and declaration
- Destructors revisited

---

## Type alias members

---

### Type aliases in C++: `using`.

A better way of declaring type aliases:

```
// C-style
typedef long long LL;
// C++-style
using LL = long long;
```

It is more readable when dealing with compound types:

```
// C-style
typedef int intarray_t[1000];
// C++-style
using intarray_t = int[1000];
```

```
// C-style
typedef int (&ref_to_array)[1000];
// C++-style
using ref_to_array = int (&)[1000];
```

`using` can also declare *alias templates* (in later lectures), while `typedef` cannot.

**[Best practice]** In C++, Use `using` to declare type aliases.

## Type alias members

A class can have **type alias members**.

```
class Dynarray {
public:
    using size_type = std::size_t;
    size_type size() const { return m_length; }
};
```

Usage: `ClassName::TypeAliasName`

```
for (Dynarray::size_type i = 0; i != a.size(); ++i)
    // ...
```

Note: Here we use `ClassName::` instead of `object.`, because such members belong to **the class**, not one single object.

## Type alias members

The class also has control over the accessibility of type alias members.

```
class A {
    using type = int;
};
A::type x = 42; // Error: Accessing private member of `A`.
```

The class has control over the accessibility of **anything that is called a *member* of it**.

## Type alias members in the standard library

All standard library containers (and `std::string`) define the type alias member `size_type` as the return type of `.size()`:

```
std::string::size_type i = s.size();
std::vector<int>::size_type j = v.size(); // Not `std::vector::size_type`!
                                         // The template argument `<int>`
                                         // is necessary here.
std::list<int>::size_type k = l.size();
```

Why?

## Type alias members in the standard library

All standard library containers (and `std::string`) define the type alias member `size_type` as the return type of `.size()`:

```
std::string::size_type i = s.size();
std::vector<int>::size_type j = v.size();
std::list<int>::size_type k = l.size();
```

- This type is **container-dependent**: Different containers may choose different types suitable for representing sizes.
  - The Qt containers often use `int` as `size_type`.
- Define `Container::size_type` to achieve good **consistency** and **generality**.

## static members

---

## static data members

A `static` data member:

```
class A {
    static int something;
    // other members ...
};
```

Just consider it as a **global variable**, except that

- its name is in the **class scope**: `A::something`, and that
- the accessibility may be restricted. Here `something` is `private`.

---

## static data members

A `static` data member:

```
class A {
    static int something;
    // other members ...
};
```

There is **only one** `A::something`: it does not belong to any object of `A`. It belongs to the **class** `A`.

- Like type alias members, we use `ClassName::` instead of `object.` to access them.

---

## static data members

A `static` data member:

```
class A {
    static int something;
    // other members ...
};
```

It can also be accessed by `a.something` (where `a` is an object of type `A`), but `a.something` and `b.something` refer to the same variable.

- If `f` is a function that returns an object of type `A`, `f().something` always accesses the same variable no matter what `f()` returns.
- In the very first externally available C++ compiler (Cfront 1.0, 1985), `f` in the expression `f().something` is not even called! This bug has been fixed soon.

---

## static data members: Example

Suppose we want to assign a unique id to each object of our class.

```
int cnt = 0;

class Dynarray {
    int *m_storage;
    std::size_t m_length;
    int m_id;
public:
    Dynarray(std::size_t n)
        : m_storage(new int[n]{}), m_length(n), m_id(cnt++) {}
    Dynarray() : m_storage(nullptr), m_length(0), m_id(cnt++) {}
    // ...
};
```

We use a global variable `cnt` as the "counter". Is this a good design?

---

## static data members: Example

The name `cnt` is confusing: A "counter" of what?

```
int X_cnt = 0, Y_cnt = 0, Z_cnt = 0;
struct X {
    int m_id;
    X() : m_id(X_cnt++) {}
};
struct Y {
    int m_id;
    Y() : m_id(Y_cnt++) {}
};
struct Z {
    int m_id;
    Z() : m_id(Z_cnt++) {}
};
```

- The program is in a mess with global variables all around.
- No prevention from potential mistakes:

```
struct Y {
    Y() : m_id(X_cnt++) {}
};
```

The mistake happens silently.

## static data members: Example

Restrict the name of this counter in the scope of the corresponding class, by declaring it as a `static` data member.

- This is exactly the idea behind `static` data members: A "global variable" restricted in class scope.

```
class Dynarray {
    static int s_cnt; // !!!
    int *m_storage;
    std::size_t m_length;
    int m_id;

public:
    Dynarray(/* ... */) : /* ... */, m_id(s_cnt++) {}
};
```

- `s` stands for `static`.

## static data members

```
class Dynarray {
    static int s_cnt; // !!!
    int *m_storage;
    std::size_t m_length;
    int m_id;

public:
    Dynarray(/* ... */) : /* ... */, m_id(s_cnt++) {}
};
```

You also need to give it a definition outside the class, according to some rules.

```
int Dynarray::s_cnt; // Zero-initialize, because it is `static`.
```

Or initialize it with some value explicitly:

```
int Dynarray::s_cnt = 42;
```

## static data members

Exercise: `std::string` has a `find` member function:

```
std::string s = something();
auto pos = s.find('a');
if (pos == std::string::npos) { // This means that 'a' is not found.
    // ...
} else {
    std::cout << s[pos] << '\n'; // If executed, it should print 'a'.
}
```

`std::string::npos` is returned when the required character is not found.

Define `npos` and `find` for your `dynarray` class, whose behavior should be similar to those of `std::string`.

## static member functions

A `static` member function:

```
class A {
public:
    static void fun(int x, int y);
};
```

Just consider it as a normal non-member function, except that

- its name is in the **class scope**: `A::fun(x, y)`, and that
- the accessibility may be restricted. Here `fun` is `public`.

## static member functions

A `static` member function:

```
class A {
public:
    static void fun(int x, int y);
};
```

`A::fun` does not belong to any object of `A`. It belongs to the **class** `A`.

- There is no `this` pointer inside `fun`.

It can also be called by `a.fun(x, y)` (where `a` is an object of type `A`), but here `a` will not be bound to a `this` pointer, and `fun` has no way of accessing any non-`static` member of `a`.

## friend

## friend functions

Recall the `Student` class:

```
class Student {
    std::string m_name;
    std::string m_id;
    int m_entranceYear;
public:
    Student(const std::string &name, const std::string &id)
        : m_name(name), m_id(id), m_entranceYear(std::stoi(id.substr(0, 4))) {}
    auto graduated(int year) const { return year - m_entranceYear >= 4; }
    // ...
};
```

Suppose we want to write a function to display the information of a `Student`.

## friend functions



```
void print(const Student &stu) {
    std::cout << "Name: " << stu.m_name << ", id: " << stu.m_id
               << "entrance year: " << stu.m_entranceYear << '\n';
}
```

This won't compile, because `m_name`, `m_id` and `m_entranceYear` are `private` members of `Student`.

- One workaround is to define `print` as a member of `Student`.
- However, there do exist some functions that cannot be defined as a member.

## friend functions

Add a `friend` declaration, so that `print` can access the private members of `Student`.

```
class Student {
    friend void print(const Student &); // The parameter name is not used in this
                                       // declaration, so it is omitted.

    std::string m_name;
    std::string m_id;
    int m_entranceYear;
public:
    Student(const std::string &name, const std::string &id)
        : m_name(name), m_id(id), m_entranceYear(std::stoi(id.substr(0, 4))) {}
    auto graduated(int year) const { return year - m_entranceYear >= 4; }
    // ...
};
```

## friend functions

Add a `friend` declaration.

```
class Student {
    friend void print(const Student &);

    // ...
};
```

A `friend` is **not** a member! You can put this `friend` declaration **anywhere in the class body**. The access modifiers have **no effect** on it.

- We often declare all the `friend`s of a class in the beginning or at the end of class definition.

## friend classes

A class can also declare another class as its `friend`.

```
class X {
    friend class Y;
    // ...
};
```

In this way, any code from the class `Y` can access the private members of `X`.

# Definition and declaration

## Definition and declaration

For a function:

```
// Only a declaration: The function body is not present.
void foo(int, const std::string &);
// A definition: The function body is present.
void foo(int x, const std::string &s) {
    // ...
}
```

---

## Class definition

For a class, a **definition** consists of **the declarations of all its members**.

```
class Widget {
public:
    widget();
    widget(int, int);
    void set_handle(int);

    // `const` is also a part of the function type, which should be present
    // in its declaration.
    const std::vector<int> &get_gadgets() const;

    // ...
private:
    int m_handle;
    int m_length;
    std::vector<int> m_gadgets;
};
```

---

## Define a member function outside the class body

A member function can be declared in the class body, and then defined outside.

```
class Widget {
public:
    const std::vector<int> &get_gadgets() const; // A declaration only.
    // ...
}; // Now the definition of `widget` is complete.

// Define the function here. The function name is `Widget::get_gadgets`.
const std::vector<int> &Widget::get_gadgets() const {
    return m_gadgets; // Just like how you do it inside the class body.
                    // The implicit `this` pointer is still there.
}
```

---

## The `::` operator

```
class Widget {
public:
    using gadgets_list = std::vector<int>;
    static int special_member;
    const gadgets_list &get_gadgets() const;
    // ...
};
const Widget::gadgets_list &Widget::get_gadgets() const {
    return m_gadgets;
}
```

- The members `Widget::gadgets_list` and `Widget::special_member` are accessed through `ClassName::`.
- The name of the member function `get_gadgets` is `Widget::get_gadgets`.

---

## Class declaration and incomplete type

To declare a class without providing a definition:

```
class A;
struct B;
```

If we only see the **declaration** of a class, we have no knowledge about its members, how many bytes it takes, how it can be initialized, ...

- Such class type is an **incomplete type**.
- We cannot create an object of such type, nor can we access any of its members.
- The only thing we can do is to declare a pointer or a reference to it.

---

## Class declaration and incomplete type

If we only see the **declaration** of a class, we have no knowledge about its members, how many bytes it takes, how it can be initialized, ...

- Such class type is an **incomplete type**.
- We cannot create an object of such type, nor can we access any of its members.
- The only thing we can do is to declare a pointer or a reference to it.

```
class Student; // We only have this declaration.

void print(const Student &stu) { // OK. Declaring a reference to it is OK.
    std::cout << stu.getName(); // Error. We don't know anything about its members.
}

class Student {
public:
    const std::string &getName() const { /* ... */ }
    // ...
};
```

---

## Destructors revisited

---

### Destructors revisited

---

A **destructor** (dtor) is a member function that is called automatically when an object of that class type is "dead".

- For global and `static` objects, on termination of the program.
- For local objects, when control reaches the end of its scope.
- For objects created by `new` / `new[]`, when their address is passed to `delete` / `delete[]`.

The destructor is often responsible for doing some **cleanup**: Release the resources it owns, do some logging, cut off its connection with some external objects, ...

---

## Destructors

---

```
class Student {
    std::string m_name;
    std::string m_id;
    int m_entranceYear;
public:
    Student(const std::string &, const std::string &);
    const std::string &getName() const;
    bool graduated(int) const;
    void setName(const std::string &);
    void print() const;
};
```

Does our `Student` class have a destructor?

---

## Destructors

---

Does our `Student` class have a destructor?

- It **must** have. Whenever you create an object of type `Student`, its destructor needs to be invoked somewhere in this program.  $\textcolor{red}{1}$

What does `Student::~~Student` need to do? Does `Student` own any resources?

---

## Destructors

---

Does our `Student` class have a destructor?

- It **must** have. Whenever you create an object of type `Student`, its destructor needs to be invoked somewhere in this program.  $\textcolor{red}{1}$

What does `Student::~~Student` need to do? Does `Student` own any resources?

- It seems that a `Student` has no resources, so nothing special needs to be done.
- However, it has two `std::string` members! Their destructors must be called, otherwise the memory is leaked!

---

## Destructors

---

To define the destructor of `Student`: Just write an empty function body, and everything is done.

```
class Student {
    std::string m_name;
    std::string m_id;
    int m_entranceYear;
public:
    ~Student() {}
};
```

## Destructors

```
class Student {
    std::string m_name;
    std::string m_id;
    int m_entranceYear;
public:
    ~Student() {}
};
```

- When the function body is executed, the object is *not yet* "dead".
  - You can still access its members.

```
~Student() { std::cout << m_name << '\n'; }
```

- After the function body is executed, **all its data members** are destroyed automatically, **in reverse order** in which they are declared.
  - For members of class type, their destructors are invoked automatically.

## Constructors vs destructors

```
Student(const std::string &name)
    : m_name(name) /* ... */ {
    // ...
}
```

- A class may have multiple ctors (overloaded).
- The data members are initialized **before** the execution of function body.
- The data members are initialized **in order** in which they are declared.

```
~Student() {
    // ...
}
```

- A class has only one dtor.  $\textcolor{red}{1}$
- The data members are destroyed **after** the execution of function body.
- The data members are destroyed **in reverse order** in which they are declared.

## Compiler-generated destructors

For most cases, a class needs a destructor.

Therefore, the compiler always generates one  $\textcolor{red}{2}$  if there is no user-declared destructor.

- The compiler-generated destructor is `public` by default.
- The compiler-generated destructor is as if it were defined with an empty function body `{}`.
- It does nothing but to destroy the data members.

We can explicitly require one by writing `= default;`, just as for other copy control members.

## Summary

Type alias members

- Type alias members belong to the class, not individual objects, so they are accessed via `ClassName::AliasName`.
- The class can controls the accessibility of type alias members.

`static` members

- `static` data members are like global variables, but in the class's scope.

- `static` member functions are like normal non-member functions, but in the class's scope. There is no `this` pointer in a `static` member function.
- A `static` member belongs to the class, instead of any individual object.

---

## Summary

### friend

- A `friend` declaration allows a function or class to access private (and protected) members of another class.
- A `friend` is not a member.

### Definitions and declarations

- A class definition includes declarations of all its members.
- A member function can be declared in the class body and then defined outside.
- A class type is an incomplete type if only its declaration (without a definition) is present.

---

## Summary

### Destructors

- Destructors are called automatically when an object's lifetime ends. They often do some clean up.
- The members are destroyed **after** the function body is executed. They are destroyed in reverse order in which they are declared.
- The compiler generates a destructor (in most cases) if none is provided. It just destroys all its members.

---

# CS100 Lecture 17

## Rvalue References and Move

---

## Contents

- Motivation: Copy is slow.
  - Rvalue references
- Move operations
  - Move constructor
  - Move assignment operator
  - The rule of five
- `std::move`
- NRVO, move and copy elision

---

## Motivation: Copy is slow.

```
std::string a = some_value(), b = some_other_value();
std::string s;
s = a;
s = a + b;
```

Consider the two assignments: `s = a` and `s = a + b`.

How is `s = a + b` evaluated?

---

## Motivation: Copy is slow.

```
s = a + b;
```

1. Evaluate `a + b` and store the result in a temporary object, say `tmp`.
2. Perform the assignment `s = tmp`.
3. The temporary object `tmp` is no longer needed, hence destroyed by its destructor.

Can we make this faster?

---

## Motivation: Copy is slow.

```
s = a + b;
```

1. Evaluate `a + b` and store the result in a temporary object, say `tmp`.
2. Perform the assignment `s = tmp`.
3. The temporary object `tmp` is no longer needed, hence destroyed by its destructor.

Can we make this faster?

- The assignment `s = tmp` is done by **copying** the contents of `tmp`?
- But `tmp` is about to "die"! Why can't we just *steal* the contents from it?

---

## Motivation: Copy is slow.

Let's look at the other assignment:

```
s = a;
```

- **Copy** is necessary here, because `a` lives long. It is not destroyed immediately after this statement is executed.
- You cannot just "steal" the contents from `a`. The contents of `a` must be preserved.

---

## Distinguish between the different kinds of assignments

```
s = a;
```

```
s = a + b;
```

What is the key difference between them?

- `s = a` is an assignment from an **lvalue**,
- while `s = a + b` is an assignment from an **rvalue**.

If we only have the copy assignment operator, there is no way of distinguishing them.

**\* Define two different assignment operators, one accepting an lvalue and the other accepting an rvalue?**

---

## Rvalue References

A kind of reference that is bound to **rvalues**:

```
int &r = 42;           // Error: Lvalue reference cannot be bound to rvalue.
int &&rr = 42;          // Correct: `rr` is an rvalue reference.
const int &cr = 42;    // Also correct:
                      // Lvalue reference-to-const can be bound to rvalue.
const int &&crr = 42;   // Correct, but useless:
                      // Rvalue reference-to-const is seldom used.

int i = 42;
int &&r2 = i;           // Error: Rvalue reference cannot be bound to lvalue.
int &r2 = i * 42;       // Error: Lvalue reference cannot be bound to rvalue.
const int &cr2 = i * 42; // Correct
int &&rr3 = i * 42;     // Correct
```

- Lvalue references (to non-`const`) can only be bound to lvalues.
- Rvalue references can only be bound to rvalues.

---

## Overload Resolution

Such overloading is allowed:

```
void fun(const std::string &);
void fun(std::string &&);
```

- `fun(s1 + s2)` matches `fun(std::string &&)`, because `s1 + s2` is an rvalue.
- `fun(s)` matches `fun(const std::string &)`, because `s` is an lvalue.
- Note that if `fun(std::string &&)` does not exist, `fun(s1 + s2)` also matches `fun(const std::string &)`.

We will see how this kind of overloading benefit us soon.

# Move Operations

## Overview

The **move constructor** and the **move assignment operator**.

```
struct Widget {
    widget(widget &&) noexcept;
    widget &operator=(widget &&) noexcept;
    // Compared to the copy constructor and the copy assignment operator:
    widget(const widget &);
    widget &operator=(const widget &);
};
```

- Parameter type is **rvalue reference**, instead of lvalue reference-to-`const`.
- `noexcept` is (almost always) **necessary**!  $\rightarrow$  We will talk about it in later lectures.

## The Move Constructor

Take the `dynarray` as an example.

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray(const Dynarray &other) // copy constructor
        : m_storage(new int[other.m_length]), m_length(other.m_length) {
        for (std::size_t i = 0; i != m_length; ++i)
            m_storage[i] = other.m_storage[i];
    }
    Dynarray(Dynarray &&other) noexcept // move constructor
        : m_storage(other.m_storage), m_length(other.m_length) {
        other.m_storage = nullptr;
        other.m_length = 0;
    }
};
```

## The Move Constructor

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray(Dynarray &&other) noexcept // move constructor
        : m_storage(other.m_storage), m_length(other.m_length) {

    }
};
```

1. *Steal* the resources of `other`, instead of making a copy.

## The Move Constructor

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
public:
    Dynarray(Dynarray &&other) noexcept // move constructor
        : m_storage(other.m_storage), m_length(other.m_length) {
        other.m_storage = nullptr;
        other.m_length = 0;
    }
};
```

1. *Steal* the resources of `other`, instead of making a copy.
2. Make sure `other` is in a valid state, so that it can be safely destroyed.

\* Take ownership of `other`'s resources!

---

## The Move Assignment Operator

Take ownership of `other`'s resources!

```
class Dynarray {
public:
    Dynarray &operator=(Dynarray &&other) noexcept {

        m_storage = other.m_storage; m_length = other.m_length;

        return *this;
    }
};
```

1. *Steal* the resources from `other`.
- 

## The Move Assignment Operator

```
class Dynarray {
public:
    Dynarray &operator=(Dynarray &&other) noexcept {

        m_storage = other.m_storage; m_length = other.m_length;
        other.m_storage = nullptr; other.m_length = 0;

        return *this;
    }
};
```

1. *Steal* the resources from `other`.
2. Make sure `other` is in a valid state, so that it can be safely destroyed.

Are we done?

---

## The Move Assignment Operator

```
class Dynarray {
public:
    Dynarray &operator=(Dynarray &&other) noexcept {

        delete[] m_storage;
        m_storage = other.m_storage; m_length = other.m_length;
        other.m_storage = nullptr; other.m_length = 0;

        return *this;
    }
};
```

0. **Avoid memory leaks!**
1. *Steal* the resources from `other`.
2. Make sure `other` is in a valid state, so that it can be safely destroyed.

Are we done?

---

## The Move Assignment Operator



```
class Dynarray {
public:
    Dynarray &operator=(Dynarray &&other) noexcept {
        if (this != &other) {
            delete[] m_storage;
            m_storage = other.m_storage; m_length = other.m_length;
            other.m_storage = nullptr; other.m_length = 0;
        }
        return *this;
    }
};
```

#### 0. Avoid memory leaks!

1. *Steal* the resources from `other`.
2. Make sure `other` is in a valid state, so that it can be safely destroyed.

\* Self-assignment safe!

## Lvalues are Copied; Rvalues are Moved

Before we move on, let's define a function for demonstration.

Suppose we have a function that concatenates two `Dynarray` s:

```
dynarray concat(const Dynarray &a, const Dynarray &b) {
    Dynarray result(a.size() + b.size());
    for (std::size_t i = 0; i != a.size(); ++i)
        result.at(i) = a.at(i);
    for (std::size_t i = 0; i != b.size(); ++i)
        result.at(a.size() + i) = b.at(i);
    return result;
}
```

Which assignment operator should be called?

```
a = concat(b, c);
```

## Lvalues are Copied; Rvalues are Moved

Lvalues are copied; rvalues are moved ...

```
a = concat(b, c); // calls move assignment operator,
                  // because `concat(b, c)` is an rvalue.
a = b; // calls copy assignment operator
```

## Lvalues are Copied; Rvalues are Moved

Lvalues are copied; rvalues are moved ...

```
a = concat(b, c); // calls move assignment operator,
                  // because `concat(b, c)` generates an rvalue.
a = b; // copy assignment operator
```

... but rvalues are copied if there is no move operation.

```
// If Dynarray has no move assignment operator, this is a copy assignment.
a = concat(b, c)
```

## Synthesized Move Operations

Like copy operations, we can use `=default` to require a synthesized move operation that has the default behaviors.

```
struct X {
    X(X &&) = default;
    X &operator=(X &&) = default;
};
```

- The synthesized move operations call the corresponding move operations of each member in the order in which they are declared.
- The synthesized move operations are `noexcept`.

Move operations can also be deleted by `=delete`, but be careful ... `{\textcolor{red}{1}}$`

## The Rule of Five: Idea

The updated *copy control members*:

- `copy constructor`
- `copy assignment operator`
- `move constructor`
- `move assignment operator`
- `destructor`

If one of them has a user-provided version, the copy control of the class is thought of to have special behaviors. (Recall "the rule of three".)

## The Rule of Five: Rules

- The `move constructor` or the `move assignment operator` will not be generated `{\textcolor{red}{2}}$` if any of the rest four members have a user-declared version.
- The `copy constructor` or `copy assignment operator`, if not provided by the user, will be implicitly `delete`d if the class has a user-provided `move operation`.
- The generation of the `copy constructor` or `copy assignment operator` is **deprecated** (since C++11) when the class has a user-declared `copy operation` or a `destructor`.
  - This is why some of you see this error:

```
Implicitly-declared copy assignment operator is deprecated, because the
class has a user-provided copy constructor.
```

## The Rule of Five

The *copy control members* in modern C++:

- `copy constructor`
- `copy assignment operator`
- `move constructor`
- `move assignment operator`
- `destructor`

**The Rule of Five:** Define zero or five of them.

## How to Invoke a Move Operation?

Suppose we give our `Dynarray` a label:

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
    std::string m_label;
};
```

The move assignment operator should invoke the **move assignment operator** on `m_label`. But how?

```
m_label = other.m_label; // calls copy assignment operator,
                        // because `other.m_label` is an lvalue.
```

`std::move`

## std::move

Defined in `<utility>`

`std::move(x)` performs an **lvalue to rvalue cast**:

```
int ival = 42;
int &&rref = ival; // Error
int &&rref2 = std::move(ival); // Correct
```

Calling `std::move(x)` tells the compiler that:

- `x` is an lvalue, but
- we want to treat `x` as an **rvalue**.

## std::move

`std::move(x)` indicates that we want to treat `x` as an **rvalue**, which means that `x` will be *moved from*.

The call to `std::move` **promises** that we do not intend to use `x` again,

- except to assign to it or to destroy it.

A call to `std::move` is usually followed by a call to some function that moves the object, after which **we cannot make any assumptions about the value of the moved-from object**.

```
void foo(x && x); // moves `x`
void foo(const x &x); // copies `x`
foo(std::move(x)); // matches `foo(x&&)` , so that `x` is moved.
```

"`std::move` does not *move* anything. It just makes a *promise*."

## Use std::move

Suppose we give every `Dynarray` a special "label", which is a string.

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
    std::string m_label;
public:
    Dynarray(Dynarray &&other) noexcept
        : m_storage(other.m_storage), m_length(other.m_length),
          m_label(std::move(other.m_label)) { // !!
        other.m_storage = nullptr;
        other.m_length = 0;
    }
};
```

The standard library facilities ought to define efficient and correct move operations.

## Use std::move

Suppose we give every `Dynarray` a special "label", which is a string.

```
class Dynarray {
    int *m_storage;
    std::size_t m_length;
    std::string m_label;
public:
    Dynarray &operator=(Dynarray &&other) noexcept {
        if (this != &other) {
            delete[] m_storage;
            m_storage = other.m_storage; m_length = other.m_length;
            m_label = std::move(other.m_label);
            other.m_storage = nullptr; other.m_length = 0;
        }
        return *this;
    }
};
```

The standard library facilities ought to define efficient and correct move operations.

## Use `std::move`

Why do we need `std::move`?

```
class Dynarray {
public:
    Dynarray(Dynarray &&other) noexcept
        : m_storage(other.m_storage), m_length(other.m_length),
          m_label(other.m_label) { // Isn't this correct?
        other.m_storage = nullptr;
        other.m_length = 0;
    }
};
```

`other` is an rvalue reference, so ...?

## An rvalue reference is an lvalue.

`other` is an rvalue reference, **which is an lvalue**.

- To move the object that the rvalue reference is bound to, we must call `std::move`.

```
class Dynarray {
public:
    Dynarray(Dynarray &&other) noexcept
        : m_storage(other.m_storage), m_length(other.m_length),
          m_label(other.m_label) { // `other.m_label` is copied, not moved.
        other.m_storage = nullptr;
        other.m_length = 0;
    }
};
```

An rvalue reference is an lvalue! Does that make sense?

## Lvalues persist; Rvalues are ephemeral.

The lifetime of rvalues is often very short, compared to that of lvalues.

- Lvalues have persistent state, whereas rvalues are either **literals** or **temporary objects** created in the course of evaluating expressions.

An rvalue reference **extends** the lifetime of the rvalue that it is bound to.

```
std::string s1 = something(), s2 = some_other_thing();
std::string &&rr = s1 + s2; // The state of the temporary object is "captured"
                          // by the rvalue reference, without which the
                          // temporary object will be destroyed.
std::cout << rr << '\n'; // Now we can use `rr` just like a normal string.
```

Golden rule: **Anything that has a name is an lvalue**.

- The rvalue reference has a name, so it is an lvalue.

## NRVO, Move and Copy Elision

### Returning a Temporary (pure rvalue)

```
std::string foo(const std::string &a, const std::string &b) {
    return a + b; // a temporary
}
std::string s = foo(a, b);
```

- First, a temporary is generated to store the result of `a + b`.
- How is this temporary returned?

## Returning a Temporary (pure rvalue)

```
std::string foo(const std::string &a, const std::string &b) {  
    return a + b; // a temporary  
}  
std::string s = foo(a, b);
```

Since C++17, **no copy or move** is made here. The initialization of `s` is the same as

```
std::string s(a + b);
```

This is called **copy elision**.

## Returning a Named Object

```
Dynarray concat(const Dynarray &a, const Dynarray &b) {  
    Dynarray result(a.size() + b.size());  
    for (std::size_t i = 0; i != a.size(); ++i)  
        result.at(i) = a.at(i);  
    for (std::size_t i = 0; i != b.size(); ++i)  
        result.at(a.size() + i) = b.at(i);  
    return result;  
}  
a = concat(b, c);
```

- `result` is a local object of `concat`.
- Since C++11, `return result` performs a **move initialization** of a temporary object, say `tmp`.
- Then a **move assignment** to `a` is performed.

## Named Return Value Optimization, NRVO

```
Dynarray concat(const Dynarray &a, const Dynarray &b) {  
    Dynarray result(a.size() + b.size());  
    // ...  
    return result;  
}  
Dynarray a = concat(b, c); // Initialization
```

NRVO transforms this code to

```
// Pseudo C++ code.  
void concat(Dynarray &result, const Dynarray &a, const Dynarray &b) {  
    // Pseudo C++ code. For demonstration only.  
    result.Dynarray::Dynarray(a.size() + b.size()); // construct in-place  
    // ...  
}  
Dynarray a@; // Uninitialized.  
concat(a@, b, c);
```

so that no copy or move is needed.

## Named Return Value Optimization, NRVO

Note:

- NRVO was invented decades ago (even before C++98).
- NRVO is an **optimization**, but not mandatory.
- Even if NRVO is performed, the move constructor should still be available.
  - Because the compiler can choose not to perform NRVO.
  - The program should be syntactically correct ("well-formed"), no matter how the compiler treats it.

## Summary

Rvalue references

- are bound to rvalues, and extends the lifetime of the rvalue.
- Functions accepting `x &&` and `const x &` can be overloaded.
- An rvalue reference is an lvalue.

Move operations

- take ownership of resources from the other object.
- After a move operation, the moved-from object should be in a valid state that can be safely assigned to or destroyed.
- `=default`
- The rule of five: Define zero or five of the special member functions.

## Summary

`std::move`

- does not move anything. It only performs an lvalue-to-rvalue cast.
- `std::move(x)` makes a promise that `x` can be safely moved from.

In modern C++, unnecessary copies are greatly avoided by:

- copy-elision, which avoids the move or copy of temporary objects, and
- move, with the `return` ed lvalue treated as an rvalue, and
- NRVO, which constructs in-place the object to be initialized.

# CS100 Lecture 18

Smart Pointers

## Contents

- Ideas
- `std::unique_ptr`
- `std::shared_ptr`

## Ideas

## Memory management is difficult!

For raw pointers obtained from `new` / `new[]` expressions, a manual `delete` / `delete[]` is required.

```
void runGame(const std::vector<Option> &options, const Settings &settings) {
    auto pwindow = new window(settings.width, settings.height, settings.mode);
    auto pGame = new Game(options, settings, pwindow);
    // Run the game ...
    while (true) {
        auto key = getUserKeyAction();
        // ...
    }
    delete pGame; // You must not forget this.
    delete pwindow; // You must not forget this.
}
```

Will you always remember to `delete`?

## Will you always remember to `delete`?

```
void runGame(const std::vector<Option> &options, const Settings &settings) {
    auto pwindow = new window(settings.width, settings.height, settings.mode);
    auto pGame = new Game(options, settings, pwindow);
    if (/* condition1 */) {
        // ...
        return; // `pwindow` and `pGame` should also be `delete`d here!
    }
    // Run the game ...
    while (true) {
        auto key = getUserKeyAction();
        // ...
        if (/* condition2 */) {
            // ...
            return; // `pwindow` and `pGame` should also be `delete`d here!
        }
    }
}
```

```

    }
}
delete pGame;
delete pwindow;
}

```

## Idea: Make use of destructors.

```

struct WindowPtr { // A "smart pointer".
    Window *ptr;
    WindowPtr(Window *p) : ptr(p) {}
    ~WindowPtr() { delete ptr; } // The destructor will `delete` the object.
};

```

When the control reaches the end of the scope in which the `WindowPtr` lives, the destructor of `WindowPtr` will be called automatically.

```

void runGame(const std::vector<Option> &options, const Settings &settings) {
    WindowPtr pwindow(new Window(settings.width, settings.height, settings.mode));
    if (/* condition1 */) {
        // ...
        return; // `pwindow` is destroyed automatically, with its destructor called.
    }
    // ...
    // `pwindow` is destroyed automatically, with its destructor called.
}

```

## What if `WindowPtr` is copied?

Now `WindowPtr` only has a compiler-generated copy constructor, which copies the value of `ptr`.

```

{
    WindowPtr pwindow(new Window(settings.width, settings.height, settings.mode));
    auto copy = pwindow; // `copy.ptr` and `pwindow.ptr` point to the same object!
} // The object is deleted twice! Disaster!

```

What should be the behavior of `auto copy = pwindow;`? Possible designs are:

1. Copy the object, as if `WindowPtr copy(new Window(*pwindow.ptr));`.
2. Copy the pointer, as if `WindowPtr copy(pwindow.ptr);`.
  - To avoid disasters caused by multiple `delete`s, some special design is needed.
3. Disable it. If there is no unique reasonable design, disable that operation.

## What if `WindowPtr` is copied?

What should be the behavior of `auto copy = pwindow;`? Possible designs are:

1. Copy the object, as if `WindowPtr copy(new Window(*pwindow.ptr));`.
  - **"Value semantics"**
  - Typical example: Standard library containers. When you copy a `std::string`, a new string is created, with the **contents** copied.
  - May be referred to as "deep copy" in some other languages.
2. Copy the pointer, as if `WindowPtr copy(pwindow.ptr);`.
  - To avoid disasters caused by multiple `delete`s, some special design is needed.
  - **"Pointer semantics", or "Reference semantics"**
  - "shallow copy" in some other languages.
3. Disable it. If there is no unique reasonable design, disable that operation.
  - In this case, `pwindow` **exclusively owns** the `Window` object.

## Overview of smart pointers

A "smart pointer" is a pointer that manages its resources.

Possible behaviors of copy of a smart pointer:

1. Copy the object. (Value semantics)
  - **Standard library containers.** e.g. `std::string`, `std::vector`, `std::set`, ...
2. Copy the pointer, but with some special design. (Pointer semantics)

- `std::shared_ptr<T>` . Defined in standard library file `<memory>` .
3. Disable it. (Unique ownership)
- `std::weak_ptr<T>` . Defined in standard library file `<memory>` .

The smart pointers `std::shared_ptr<T>` , `std::unique_ptr<T>` and `std::weak_ptr<T>` are **the C++'s answer to garbage collection**.

- `std::weak_ptr` is not covered in CS100.

## Overview of smart pointers

The smart pointers `std::shared_ptr<T>` , `std::unique_ptr<T>` and `std::weak_ptr<T>` are **the C++'s answer to garbage collection**.

Smart pointers support the similar operations as raw pointers:

- `*sp` returns reference to the pointed-to object.
- `sp->mem` is equivalent to `(*sp).mem`.
- `sp` is *contextually convertible* to `bool` : It can be treated as a "condition".
  - It can be placed at the "condition" part of `if` , `for` , `while` , `do` statements.
  - It can be used as operands of `&&` , `||` , `!` or the first operand of `?:` .
  - In all cases, the conversion result is `true` **iff** `sp` holds an object (not "null").

**[Best practice]** In modern C++, prefer smart pointers to raw pointers.

## `std::unique_ptr`

### Design: Unique ownership of the object

A "unique-pointer" saves a raw pointer internally, pointing to the object it owns.

When the unique-pointer is destroyed, it disposes of the object it owns.

```
class WindowPtr {
    window *ptr;
public:
    WindowPtr(window *p = nullptr) : ptr(p) {}
    ~WindowPtr() { delete ptr; }
    WindowPtr(const WindowPtr &) = delete;
    WindowPtr &operator=(const WindowPtr &) = delete;
    WindowPtr(WindowPtr &&other) noexcept : ptr(other.ptr) { other.ptr = nullptr; }
    WindowPtr &operator=(WindowPtr &&other) noexcept {
        if (&other != this) {
            delete ptr; ptr = other.ptr; other.ptr = nullptr;
        }
        return *this;
    }
};
```

**Move** of a unique-pointer: **transfer of ownership**.

- **Move-only type**

## `std::unique_ptr`

Like `std::vector` , `std::unique_ptr` is also a class template. It is not a type itself.

- `std::unique_ptr<PointeeType>` is the complete type name, where `PointeeType` is the type of the object that it points to.
- For `T $!neq$ U` , `std::unique_ptr<T>` and `std::unique_ptr<U>` are **two different and independent types**.

Same for `std::shared_ptr` , which we will talk about later.

## Creating a `std::unique_ptr`: Two common ways

- Pass a pointer created by `new` to the constructor:

```
std::unique_ptr<Student> p(new Student("Bob", 2020123123));
```

- Here `<Student>` can be omitted. The compiler is able to deduce it.
- Use `std::make_unique<T>` , and pass the initializers to it.



```
std::unique_ptr<Student> p1 = std::make_unique<Student>("Bob", 2020123123);
auto p2 = std::make_unique<Student>("Alice", 2020321321);
```

- `std::make_unique<T>(args...)` *perfectly forwards* the arguments `args...` to the constructor of `T`, as if the object were created by `new T(args...)`.
- `std::make_unique<T>` returns a `std::unique_ptr<T>` to the created object.

## Default initialization of a `std::unique_ptr`

```
std::unique_ptr<T> up;
```

The default constructor of `std::unique_ptr<T>` initializes `up` to be a "null pointer".

`up` is in the state that does not own any object.

- This is a defined and deterministic behavior! It is **not** holding some indeterminate value.
  - The standard library hates indeterminate values, just as we do.

## `std::unique_ptr`: Automatic memory management

```
void foo() {
    auto pAlice = std::make_unique<Student>("Alice", 2020321321);
    // Do something...
    if (some_condition()) {
        auto pBob = std::make_unique<Student>("Bob", 2020123123);
        // ...
    } // `Student::~~Student()` is called for Bob,
    // because the lifetime of `pBob` ends.
} // `Student::~~Student()` is called for Alice,
// because the lifetime of `pAlice` ends.
```

A `std::unique_ptr` automatically calls the destructor once it gets destroyed or assigned a new value.

- No manual `delete` needed!

## `std::unique_ptr`: Move-only

```
auto p = std::make_unique<std::string>(5, 'c');
std::cout << *p << std::endl;           // Prints "ccccc".
auto q = p;                             // Error. Copy is not allowed.
auto r = std::move(p);                   // Correct.
// Now the ownership of this string has been transferred to `r`.
std::cout << *r << std::endl; // Prints "ccccc".
if (!p) // true
    std::cout << "p is \"null\" now." << std::endl;
```

`std::unique_ptr` is not copyable, but only movable.

- Remember, only one `std::unique_ptr` can point to the managed object.
- Move of a `std::unique_ptr` is the transfer of ownership of the managed object.

## `std::unique_ptr`: Move-only

```
auto p = std::make_unique<std::string>(5, 'c');
std::cout << *p << std::endl;           // Prints "ccccc".
auto q = p;                             // Error. Copy is not allowed.
auto r = std::move(p);                   // Correct.
// Now the ownership of this string has been transferred to `r`.
std::cout << *r << std::endl; // Prints "ccccc".
if (!p) // true
    std::cout << "p is \"null\" now." << std::endl;
```

After `auto up2 = std::move(up1);`, `up1` becomes "null". The object that `up1` used to manage now belongs to `up2`.

The assignment `up2 = std::move(up1)` destroys the object that `up2` used to manage, and lets `up2` take over the object managed by `up1`. After that, `up1` becomes "null".

## Express your intent precisely.

You may accidentally write the following code:

```
// Given that `pwindow` is a `std::unique_ptr<window>`.
auto p = pwindow; // Oops, attempting to copy a `std::unique_ptr`.
```

The compiler gives an error, complaining about the use of deleted copy constructor.

What are you going to do?

- A. Change it to `auto p = std::move(pwindow);`.
- B. Give up on smart pointers, and switch back to raw pointers.
- C. Copy-and-paste the compiler output and ask ChatGPT.

## Express your intent precisely.

You may accidentally write the following code:

```
// Given that `pwindow` is a `std::unique_ptr<window>`.
auto p = pwindow; // Oops, attempting to copy a `std::unique_ptr`.
```

The compiler gives an error, complaining about the use of deleted copy constructor.

1. Syntactically, a `std::unique_ptr` is not copyable, but you are copying it. **(Direct cause of the error)**
2. Logically, a `std::unique_ptr` must exclusively manage the pointed-to object. Why would you copy a `std::unique_ptr`?
  - The **root cause of the error** is related to your intent: What are you going to do with `p`?

## Express your intent precisely.

```
// Given that `pwindow` is a `std::unique_ptr<window>`.
auto p = pwindow; // Oops, attempting to copy a `std::unique_ptr`.
```

What are you going to do with `p`?

- If you want to copy the pointed-to object, change it to `auto p = std::make_unique<window>(*pwindow);`.
- If you want `p` to be just an **observer**, write `auto p = pwindow.get();`.
  - `pwindow.get()` returns a **raw pointer** to the object, which is of type `window *`.
  - Be careful! As an observer, `p` should never interfere in the lifetime of the object. A simple `delete p;` will cause disaster.

## Express your intent precisely.

```
// Given that `pwindow` is a `std::unique_ptr<window>`.
auto p = pwindow; // Oops, attempting to copy a `std::unique_ptr`.
```

What are you going to do with `p`?

- If you want `p` to take over the object managed by `pwindow`, change it to `auto p = std::move(pwindow);`.
  - Be careful! `pwindow` will no longer own that object.
- If you want to `p` to be another smart pointer that **shares** the ownership with `pwindow`, `std::unique_ptr` is not suitable here.  $\rightarrow$  See `std::shared_ptr` later.

## Returning a `std::unique_ptr`

```
struct Window {
    // A typical "factory" function.
    static std::unique_ptr<window> create(const Settings &settings) {
        auto pw = std::make_unique<window>(/* some arguments */);
        logWindowCreation(pw);
        // ...
        return pw;
    }
};
auto state = window::create(my_settings);
```

A temporary is move-constructed from `pw`, and then is used to move-construct `state`.

- These two moves can be optimized out by NRVO.

## Other operations on `std::unique_ptr`

`up.reset()`, `up.release()`, `up1.swap(up2)`, `up1 == up2`, etc.

[Full list](#) of operations supported on a `std::unique_ptr`.

## `std::unique_ptr` for array type

By default, the destructor of `std::unique_ptr<T>` uses a `delete` expression to destroy the object it holds.

What happens if `std::unique_ptr<T> up(new T[n]);`?

## `std::unique_ptr` for array type

By default, the destructor of `std::unique_ptr<T>` uses a `delete` expression to destroy the object it holds.

What happens if `std::unique_ptr<T> up(new T[n]);`?

- The memory is obtained using `new[]`, but deallocated by `delete`! **Undefined behavior.**

## `std::unique_ptr` for array type

A template specialization: `std::unique_ptr<T[]>`.

- Specially designed to represent pointers that point to a "dynamic array" of objects.
- It has some array-specific operators, e.g. `operator[]`. In contrast, it does not support `operator*` and `operator->`.
- It uses `delete[]` instead of `delete` to destroy the objects.

```
auto up = std::make_unique<int[]>(n);
std::unique_ptr<int[]> up2(new int[n]{}); // equivalent
for (auto i = 0; i != n; ++i)
    std::cout << up[i] << ' ';
```

## `std::unique_ptr` for array type

A template specialization: `std::unique_ptr<T[]>`:

- Specially designed to represent pointers that point to a "dynamic array" of objects.
- It has some array-specific operators, e.g. `operator[]`. In contrast, it does not support `operator*` and `operator->`.
- It uses `delete[]` instead of `delete` to destroy the objects.

## Use standard library containers instead!

They almost always do a better job. `std::unique_ptr<T[]>` is seldom needed.

## `std::unique_ptr` is zero-overhead.

`std::unique_ptr` stores nothing more than a raw pointer. `sizeof(std::unique_ptr) == sizeof(T*)`

It does nothing more than better copy / move control and automatic object destruction.

**Zero-overhead:** Using a `std::unique_ptr` does not cost more time or space than using raw pointers.

**[Best practice]** Use `std::unique_ptr` for exclusive-ownership resource management.

## `std::shared_ptr`

## Motivation

A `std::unique_ptr` exclusively owns an object, but sometimes this is not convenient.

```

struct WindowManager {
    void addWindow(const std::unique_ptr<Window> &pw) {
        mwindows.push_back(pw); // Error. Attempts to copy a `std::unique_ptr`.
    }
private:
    std::vector<std::unique_ptr<Window>> mwindows;
};

struct Window {
    static std::unique_ptr<Window> create(const Settings &settings) {
        auto pw = std::make_unique<Window>(/* some arguments */);
        logWindowCreation(pw);
        settings.getWindowManager().addWindow(pw);
        return pw;
    }
};

```

## Motivation

Design a "shared-pointer" that allows the object it manages to be *shared*.

When should the object be destroyed?

- A `std::unique_ptr` destroys the object it manages when the pointer itself is destroyed.
- If we allow many shared-pointers to point to the same object, how can we know when to destroy that object?

## Idea: Reference counting

```

class WindowPtr {
    WindowWithCounter *ptr;
public:
    WindowPtr(WindowPtr &&other) noexcept : ptr(other.ptr) { other.ptr = nullptr; }
    WindowPtr &operator=(WindowPtr &&other) noexcept {
        if (this != &other) {
            if (--ptr->refCount == 0)
                delete ptr;
            ptr = other.ptr; other.ptr = nullptr;
        }
        return *this;
    }
};

```

## Reference counting

By maintaining a variable that counts how many shared-pointers are pointing to the object, we can know when to destroy the object.

This strategy is adopted by Python.

It can prevent memory leak in many cases, but not all cases!  $\rightarrow$  See the question in the end of this lecture's slides.

## `std::shared_ptr`

A smart pointer that uses **reference counting** to manage shared objects.

Create a `shared_ptr`:

```

std::shared_ptr<Type> sp2(new Type(args));
auto sp = std::make_shared<Type>(args); // equivalent, but better

```

For example:

```

// sp points to a string "ccccccccc".
auto sp = std::make_shared<std::string>(10, 'c');

auto pwindow = std::make_shared<Window>(80, 24, my_settings.mode);

```

## Create a `shared_ptr`

Note: For `std::unique_ptr`, both of the following ways are ok (since C++17):

```
auto up = std::make_unique<Type>(args);
std::unique_ptr<Type> up2(new Type(args));
```

For `std::shared_ptr`, `std::make_shared` is preferable to directly using `new`.

```
auto sp = std::make_shared<Type>(args);    // preferred
std::shared_ptr<Type> sp2(new Type(args)); // ok, but less preferred
```

Read *Effective Modern C++* Item 21. (Note that this book is based on C++14.)

[Best practice] Prefer `std::make_shared` to directly using `new` when creating a `std::shared_ptr`.

## Operations

`*` and `->` can be used as if it is a raw pointer:

```
auto sp = std::make_shared<std::string>(10, 'c');
std::cout << *sp << std::endl;           // "ccccccccc"
std::cout << sp->size() << std::endl;    // "10"
```

`sp.use_count()`: The value of the reference counter.

```
auto sp = std::make_shared<std::string>(10, 'c');
{
    auto sp2 = sp;
    std::cout << sp.use_count() << std::endl; // 2
} // `sp2` is destroyed, but the managed object is not destroyed.
std::cout << sp.use_count() << std::endl;    // 1
```

## Operations

Full list of supported operations on `std::shared_ptr`.

`std::shared_ptr` is relatively easy to use, since you are free to create many `std::shared_ptr`'s pointing to one object.

However, `std::shared_ptr` has time and space overhead. Copy of a `std::shared_ptr` requires maintenance of reference counter.

## Summary

`std::unique_ptr`

- Exclusive-ownership.
- Move-only. Move is the transfer of ownership.
- Zero-overhead.

`std::shared_ptr`

- Shared-ownership.
- Uses reference counting.
  - Copy increments the reference counter.
  - When the counter is decremented to zero, the object is destroyed.

## Question

Does `std::shared_ptr` prevent memory leak in all cases? Think about what happens in the following code.

```
struct Node {
    int value;
    std::shared_ptr<Node> next;
    Node(int x, std::shared_ptr<Node> p) : value{x}, next{std::move(p)} {}
};
void foo() {
    auto p = std::make_shared<Node>(1, nullptr);
    p->next = std::make_shared<Node>(2, p);
    p.reset();
}
/*
```

这段代码中确实存在内存泄漏的问题。问题在于`foo`函数中创建的`std::shared_ptr<Node>`对象`p`和`p->next`。`p`是一个`std::shared_ptr<Node>`，它指向一个`Node`对象，该对象的`value`是1，并且`next`指针是`nullptr`。

然后, `p->next`被赋值为一个新的`std::shared_ptr<Node>`, 这个新的`Node`对象的`value`是2, 并且它的`next`指针指向`p` (即第一个节点)。  
这里, 第一个节点的`shared_ptr`引用计数是1, 因为只有`p`指向它。  
第二个节点的`shared_ptr`引用计数也是1, 因为它是独立创建的。  
当执行`p.reset()`时, `p`所指向的节点的引用计数减1, 变为0, 因此第一个节点会被销毁。  
但是, 第二个节点的`shared_ptr`引用计数还是1, 因为它的`shared_ptr`是通过`p->next`间接引用的, 并没有直接通过`p.reset()`释放。所以第二个节点不会被销毁, 导致内存泄漏。  
\*/

# CS100 Lecture 19

Operator Overloading

## Contents

- Basics
- Example: `Rational`
  - Arithmetic and relational operators
  - Increment and decrement operators (`++`, `--`)
  - IO operators (`<<`, `>>`)
- Example: `Dynarray`
  - Subscript operator (`[]`)
- Example: `WindowPtr`
  - Dereference (indirection) operator (`*`)
  - Member access through pointer (`->`)
- User-defined type conversions

## Basics

Operator overloading: Provide the behaviors of **operators** for class types.

We have already seen some:

- The **copy assignment operator** and the **move assignment operator** are two special overloads for `operator=`.
- The `IOStream` library provides overloaded `operator<<` and `operator>>` to perform input and output.
- The string library provides `operator+` for concatenation of strings, and `<`, `<=`, `>`, `>=`, `==`, `!=` for comparison in lexicographical order.
- Standard library containers and `std::string` have `operator[]`.
- Smart pointers have `operator*` and `operator->`.

## Basics

Overloaded operators can be defined in two forms:

- as a member function, in which the leftmost operand is bound to `this`:
  - `a[i]  $\rightarrow$  a.operator[](i)`
  - `a = b  $\rightarrow$  a.operator=(b)`
  - `*a  $\rightarrow$  a.operator*()`
  - `f(arg1, arg2, arg3, ...)  $\rightarrow$  f.operator()(arg1, arg2, arg3, ...)`
- as a non-member function:
  - `a == b  $\rightarrow$  operator==(a, b)`
  - `a + b  $\rightarrow$  operator+(a, b)`

## Basics

Some operators cannot be overloaded:

`obj.mem`, `::`, `?:`, `obj.*memptr` (not covered in CS100)

Some operators can be overloaded, but are strongly not recommended:

`cond1 && cond2`, `cond1 || cond2`

- Reason: Since `x && y` would become `operator&&(x, y)`, there is no way to overload `&&` (or `||`) that preserves the **short-circuit evaluation** property.

## Basics

- At least one operand should be a class type. Modifying the behavior of operators on built-in types is not allowed.

```
int operator+(int, int);    // Error.
MyInt operator-(int, int); // Still error.
```

- Inventing new operators is not allowed.

```
double operator**(double x, double exp); // Error.
```

- Overloading does not modify the **associativity**, **precedence** and the **operands' evaluation order**.

```
std::cout << a + b; // Equivalent to `std::cout << (a + b)`.
```

## Example: Rational

### A class for rational numbers

```
class Rational {
    int m_num;        // numerator
    unsigned m_denom; // denominator
    void simplify() { // Private, because this is our implementation detail.
        int gcd = std::gcd(m_num, m_denom); // std::gcd in <numeric> (since C++17)
        m_num /= gcd; m_denom /= gcd;
    }
public:
    Rational(int x = 0) : m_num{x}, m_denom{1} {} // Also a default constructor.
    Rational(int num, unsigned denom) : m_num{num}, m_denom{denom} { simplify(); }
    double to_double() const {
        return static_cast<double>(m_num) / m_denom;
    }
};
```

We want to have arithmetic operators supported for `Rational`.

### `Rational`: arithmetic operators

A good way: define `operator+=` and the **unary** `operator-`, and then define other operators in terms of them.

```
class Rational {
    friend Rational operator-(const Rational &); // Unary `operator-` as in `-x`.
public:
    Rational &operator+=(const Rational &rhs) {
        m_num = m_num * static_cast<int>(rhs.m_denom) // Be careful with `unsigned`!
            + static_cast<int>(m_denom) * rhs.m_num;
        m_denom *= rhs.m_denom;
        simplify();
        return *this; // `x += y` should return a reference to `x`.
    }
};

Rational operator-(const Rational &x) {
    return {-x.m_num, x.m_denom};
    // The above is equivalent to `return Rational(-x.m_num, x.m_denom);`.
}
```

### `Rational`: arithmetic operators

Define the arithmetic operators in terms of the compound assignment operators.

```

class Rational {
public:
    Rational &operator+=(const Rational &rhs) {
        // Makes use of `operator+=` and the unary `operator-`.
        return *this += -rhs;
    }
};

Rational operator+(const Rational &lhs, const Rational &rhs) {
    return Rational(lhs) += rhs; // Makes use of `operator+=`.
}

Rational operator-(const Rational &lhs, const Rational &rhs) {
    return Rational(lhs) -= rhs; // Makes use of `operator-`.
}

```

## [Best practice] Avoid repetition.

```

class Rational {
public:
    Rational &operator+=(const Rational &rhs) {
        m_num = m_num * static_cast<int>(rhs.m_denom)
            + static_cast<int>(m_denom) * rhs.m_num;
        m_denom *= rhs.m_denom;
        simplify();
        return *this;
    }
};

```

The arithmetic operators for `Rational` are simple yet requires carefulness.

- Integers with different signed-ness need careful treatment.
- Remember to `simplify()`.

Fortunately, we only need to pay attention to these things in `operator+=`. Everything will be right if `operator+=` is right.

## [Best practice] Avoid repetition.

The code would be very error-prone if you implement every function from scratch!

```

class Rational {
public:
    Rational &operator+=(const Rational &rhs) {
        m_num = m_num * static_cast<int>(rhs.m_denom)
            + static_cast<int>(m_denom) * rhs.m_num;
        m_denom *= rhs.m_denom;
        simplify();
        return *this;
    }
    Rational &operator-=(const Rational &rhs) {
        m_num = m_num * static_cast<int>(rhs.m_denom)
            - static_cast<int>(m_denom) * rhs.m_num;
        m_denom *= rhs.m_denom;
        simplify();
        return *this;
    }
    friend Rational operator+(const Rational &,
                             const Rational &);
    friend Rational operator-(const Rational &,
                             const Rational &);
};

```

```

Rational operator+(const Rational &lhs,
                   const Rational &rhs) {
    return {
        lhs.m_num * static_cast<int>(rhs.m_denom)
            + static_cast<int>(lhs.m_denom) * rhs.m_num,
        lhs.m_denom * rhs.m_denom
    };
}

Rational operator-(const Rational &lhs,
                   const Rational &rhs) {
    return {
        lhs.m_num * static_cast<int>(rhs.m_denom)
            - static_cast<int>(lhs.m_denom) * rhs.m_num,
        lhs.m_denom * rhs.m_denom
    };
}

```



---

## Rational: arithmetic operators

---

Exercise: Define `operator*` (multiplication) and `operator/` (division) as well as `operator*=` and `operator/=` for `Rational`.

---

## Rational: arithmetic and relational operators

---

What if we define them (say, `operator+`) as member functions?

```
class Rational {
public:
    Rational(int x = 0) : m_num{x}, m_denom{1} {}
    Rational operator+(const Rational &rhs) const {
        return {
            m_num * static_cast<int>(rhs.m_denom)
            + static_cast<int>(m_denom) * rhs.m_num,
            m_denom * rhs.m_denom
        };
    }
};
```

---

## Rational: arithmetic and relational operators

---

What if we define them (say, `operator+`) as member functions?

```
class Rational {
public:
    Rational(int x = 0) : m_num{x}, m_denom{1} {}
    Rational operator+(const Rational &rhs) const {
        // ...
    }
};
```

```
Rational r = some_value();
auto s = r + 0; // OK, `r.operator+(0)`, effectively `r.operator+(Rational(0))`
auto t = 0 + r; // Error! `0.operator+(r)` ???
```

## Rational: arithmetic and relational operators

---

To allow implicit conversions on both sides, the operator should be defined as **non-member functions**.

```
Rational r = some_value();
auto s = r + 0; // OK, `operator+(r, 0)`, effectively `operator+(r, Rational(0))`
auto t = 0 + r; // OK, `operator+(0, r)`, effectively `operator+(Rational(0), r)`
```

**[Best practice]** The "symmetric" operators, whose operands are often exchangeable, often should be defined as non-member functions.

---

## Rational: relational operators

---

Define `<` and `==`, and define others in terms of them. (Before C++20)

- Since C++20: Define `==` and `<=>`, and the compiler will generate others.

A possible way: Use `to_double` and compare the floating-point values.

```
bool operator<(const Rational &lhs, const Rational &rhs) {
    return lhs.to_double() < rhs.to_double();
}
```

- This does not require `operator<` to be a `friend`.
  - However, this is subject to floating-point errors.
- 

## Rational: relational operators

---

Another way (possibly better):

```

class Rational {
    friend bool operator<(const Rational &, const Rational &);
    friend bool operator==(const Rational &, const Rational &);
};
bool operator<(const Rational &lhs, const Rational &rhs) {
    return static_cast<int>(rhs.m_denom) * lhs.m_num
        < static_cast<int>(lhs.m_denom) * rhs.m_num;
}
bool operator==(const Rational &lhs, const Rational &rhs) {
    return lhs.m_num == rhs.m_num && lhs.m_denom == rhs.m_denom;
}

```

If there are member functions to obtain the numerator and the denominator, these functions don't need to be `friend`.

## Rational: relational operators

**[Best practice]** Avoid repetition.

Define others in terms of `<` and `==`:

```

bool operator>(const Rational &lhs, const Rational &rhs) {
    return rhs < lhs;
}
bool operator<=(const Rational &lhs, const Rational &rhs) {
    return !(lhs > rhs);
}
bool operator>=(const Rational &lhs, const Rational &rhs) {
    return !(lhs < rhs);
}
bool operator!=(const Rational &lhs, const Rational &rhs) {
    return !(lhs == rhs);
}

```

## Relational operators

Define relational operators in a consistent way:

- `a != b` should mean `!(a == b)`
- `!(a < b)` and `!(a > b)` should imply `a == b`

C++20 has devoted some efforts to the design of **consistent comparison**: [P0515r3](#).

## Relational operators

Avoid abuse of relational operators:

```

struct Point2d { double x, y; };
bool operator<(const Point2d &lhs, const Point2d &rhs) {
    return lhs.x < rhs.x; // Is this the unique, best behavior?
}
// Much better design: Use a named function.
bool less_in_x(const Point2d &lhs, const Point2d &rhs) {
    return lhs.x < rhs.x;
}

```

**[Best practice]** Operators should be used for operations that are likely to be unambiguous to users.

- If an operator has plausibly more than one interpretation, use named functions instead. Function names can convey more information.

`std::string` has `operator+` for concatenation. Why doesn't `std::vector` have one?

## ++ and --

`++` and `--` are often defined as **members**, because they modify the object.

To differentiate the postfix version `x++` and the prefix version `++x`: **The postfix version has a parameter of type `int`**.

- The compiler will translate `++x` to `x.operator++()`, `x++` to `x.operator++(0)`.

```
class Rational {
public:
    Rational &operator++() { ++m_num; simplify(); return *this; }
    Rational operator++(int) { // This `int` parameter is not used.
        // The postfix version is almost always defined like this.
        auto tmp = *this;
        ++*this; // Makes use of the prefix version.
        return tmp;
    }
};
```

## ++ and --

```
class Rational {
public:
    Rational &operator++() { ++m_num; simplify(); return *this; }
    Rational operator++(int) { // This `int` parameter is not used.
        // The postfix version is almost always defined like this.
        auto tmp = *this;
        ++*this; // Make use of the prefix version.
        return tmp;
    }
};
```

The prefix version returns reference to `*this`, while the postfix version returns a copy of `*this` before incrementation.

- Same as the built-in behaviors.

## IO operators

Implement `std::cin >> r` and `std::cout << r`.

Input operator:

```
std::istream &operator>>(std::istream &, Rational &);
```

Output operator:

```
std::ostream &operator<<(std::ostream &, const Rational &);
```

- `std::cin` is of type `std::istream`, and `std::cout` is of type `std::ostream`.
- The left-hand side operand should be returned, so that we can write

```
std::cin >> a >> b >> c; std::cout << a << b << c;
```

## Rational: output operator

```
class Rational {
    friend std::ostream &operator<<(std::ostream &, const Rational &);
};
std::ostream &operator<<(std::ostream &os, const Rational &r) {
    return os << r.m_num << '/' << r.m_denom;
}
```

If there are member functions to obtain the numerator and the denominator, it don't have to be a `friend`.

```
std::ostream &operator<<(std::ostream &os, const Rational &r) {
    return os << r.get_numerator() << '/' << r.get_denominator();
}
```

## Rational: input operator

Suppose the input format is `a b` for the rational number  $\frac{a}{b}$ , where `a` and `b` are integers.

```
std::istream &operator>>(std::istream &is, Rational &r) {
    int x, y; is >> x >> y;
    if (!is) { // Pay attention to input failures!
        x = 0;
        y = 1;
    }
    if (y < 0) { y = -y; x = -x; }
    r = Rational(x, y);
    return is;
}
```

## Example: Dynarray

### operator[]

```
class Dynarray {
public:
    int &operator[](std::size_t n) {
        return m_storage[n];
    }
    const int &operator[](std::size_t n) const {
        return m_storage[n];
    }
};
```

The use of `a[i]` is interpreted as `a.operator[](i)`.

(C++23 allows `a[i, j, k]!`)

Homework: Define `operator[]` and relational operators for `Dynarray`.

## Example: WindowPtr

### WindowPtr: indirection (dereference) operator

Recall the `WindowPtr` class we defined in the previous lecture.

```
struct WindowWithCounter {
    window thewindow;
    int refCount = 1;
};
class WindowPtr {
    WindowWithCounter *m_ptr;
public:
    window &operator*() const { // why should it be const?
        return m_ptr->thewindow;
    }
};
```

We want `*sp` to return reference to the managed object.

### WindowPtr: indirection (dereference) operator

Why should `operator*` be `const`?

```
class WindowPtr {
    WindowWithCounter *m_ptr;
public:
    window &operator*() const {
        return m_ptr->thewindow;
    }
};
```

On a `const WindowPtr` ("top-level" `const`), obtaining a non-`const` reference to the managed object may still be allowed.

- The (smart) pointer is `const`, but the managed object is not.

- `this` is `const WindowPtr *`, so `m_ptr` is `WindowWithCounter *const`.

## WindowPtr: member access through pointer

To make `operator->` consistent with `operator*` (make `ptr->mem` equivalent to `(*ptr).mem`), `operator->` is almost always defined like this:

```
class WindowPtr {
public:
    window *operator->() const {
        return std::addressof(operator*());
    }
};
```

`std::addressof(x)` is almost always equivalent to `&x`, but the latter may not return the address of `x` if `operator&` for `x` has been overloaded!

# User-defined type conversions

## Type conversions

A **type conversion** is a function  $f: T \mapsto U$  for two different types  $T$  and  $U$ .  
Type conversions can happen either **implicitly** or **explicitly**. A conversion is **explicit** if and only if the target type `U` is written explicitly in the conversion expression.  
Explicit conversions can happen in one of the following forms:

| expression                            | explanation                   | example                                   |
|---------------------------------------|-------------------------------|-------------------------------------------|
| <code>what_cast&lt;U&gt;(expr)</code> | through named casts           | <code>static_cast&lt;int&gt;(3.14)</code> |
| <code>U(expr)</code>                  | looks like a constructor call | <code>std::string("xx"), int(3.14)</code> |
| <code>(U)expr</code>                  | old C-style conversion        | Not recommended. Don't use it.            |

## Type conversions

A **type conversion** is a function  $f: T \mapsto U$  for two different types  $T$  and  $U$ .  
Type conversions can happen either **implicitly** or **explicitly**. A conversion is **explicit** if and only if the target type `U` is written explicitly in the conversion expression.

- Arithmetic conversions are often allowed to happen implicitly:

```
int sum = /* ... */ , n = /* ... */;
auto average = 1.0 * sum / n; // `sum` and `n` are converted to `double`,
                             // so `average` has type `double`.
```

- The dangerous conversions for built-in types must be explicit:

```
const int *cip = something();
auto ip = const_cast<int *>(cip);           // int *
auto cp = reinterpret_cast<char *>(ip);     // char *
```

## Type conversions

A **type conversion** is a function  $f: T \mapsto U$  for two different types  $T$  and  $U$ .  
Type conversions can happen either **implicitly** or **explicitly**. A conversion is **explicit** if and only if the target type `U` is written explicitly in the conversion expression.

- This is also a type conversion, isn't it?

```
std::string s = "hello"; // from `const char [6]` to `std::string`
```

- This is also a type conversion, isn't it?

```
std::size_t n = 1000;
std::vector<int> v(n); // from `std::size_t` to `std::vector<int>`
```

How do these type conversions happen? Are they implicit or explicit?

## Type conversions

We can define a type conversion for our class `X` in one of the following ways:

1. A constructor with exactly one parameter of type `T` is a conversion from `T` to `X`.
  - Example: `std::string` has a constructor accepting a `const char *`. `std::vector` has a constructor accepting a `std::size_t`.
2. A **type conversion operator**: a conversion from `X` to some other type.

```
class Rational {
public:
    // conversion from `Rational` to `double`.
    operator double() const { return 1.0 * m_num / m_denom; }
};
Rational r(3, 4);
double dval = r; // 0.75
```

## Type conversion operator

A type conversion operator is a member function of class `X`, which defines the type conversion from `X` to some other type `T`.

```
class Rational {
public:
    // conversion from `Rational` to `double`.
    operator double() const { return 1.0 * m_num / m_denom; }
};
Rational r(3, 4);
double dval = r; // 0.75
```

- The name of the function is `operator T`.
- The return type is `T`, which is not written before the name.
- A type conversion is usually a **read-only** operation, so it is usually `const`.

## Explicit type conversion

Some conversions should be allowed to happen implicitly:

```
void foo(const std::string &str) { /* ... */ }
foo("hello"); // implicit conversion from `const char [6]` to `const char *`,
              // and then to `std::string`.
```

Some should never happen implicitly!

```
void bar(const std::vector<int> &vec) { /* ...*/ }
bar(1000); // ??? Too weird!
bar(std::vector<int>(1000)) // OK.
std::vector<int> v1(1000); // OK.
std::vector<int> v2 = 1000; // No! This should never happen. Too weird!
```

## Explicit type conversion

To disallow the implicit use of a constructor as a type conversion, write `explicit` before the return type:

```
class string { // Suppose this is the `std::string` class.
public:
    string(const char *cstr); // Not marked `explicit`. Implicit use is allowed.
};

template <typename T> class vector { // Suppose this is the `std::vector` class.
public:
    explicit vector(std::size_t n); // Implicit use is not allowed.
};
```

```
class Dynarray {
public:
    explicit Dynarray(std::size_t n) : m_length{n}, m_storage{new int[n]{} } {}
};
```

## Explicit type conversion

To disallow the implicit use of a type conversion operator, also write `explicit`:

```
class Rational {
public:
    explicit operator double() const { return 1.0 * m_num / m_denom; }
};
Rational r(3, 4);
double d = r; // Error.
void foo(double x) { /* ... */ }
foo(r); // Error.
foo(double(r)); // OK.
foo(static_cast<double>(r)); // OK.
```

## [Best practice] Avoid the abuse of type conversion operators.

Type conversion operators can lead to unexpected results!

```
class Rational {
public:
    operator double() const { return 1.0 * m_num / m_denom; }
    operator std::string() const {
        return std::to_string(m_num) + " / " + std::to_string(m_denom);
    }
};
int main() {
    Rational r(3, 4);
    std::cout << r << '\n'; // Oops! Is it `0.75` or `3 / 4`?
}
```

In the code above, either **mark the type conversions as** `explicit`, or remove them and **define named functions** like `to_double()` and `to_string()` instead.

## Contextual conversion to `bool`

A special rule for conversion to `bool`.

Suppose `expr` is an expression of a class type `X`, and suppose `X` has an `explicit` type conversion operator to `bool`. In the following contexts, that conversion is applicable even if it is not written as `bool(expr)` or `static_cast<bool>(expr)`:

- `if (expr)`, `while (expr)`, `for (...; expr; ...)`, `do ... while (expr)`
- as the operand of `!`, `&&`, `||`
- as the first operand of `?::` `expr ? something : something_else`

## Contextual conversion to `bool`

Exercise: We often test whether a pointer is non-null like this:

```
if (ptr) {
    // ...
}
auto val = ptr ? ptr->some_value : 0;
```

Define a conversion from `WindowPtr` to `bool`, so that we can test whether a `WindowPtr` is non-null in the same way.

- Should this conversion be allowed to happen implicitly? If not, mark it `explicit`.

## Summary

Operator overloading

- As a non-member function: `@a $\rightarrow$ operator@ (a)`, `a @ b $\rightarrow$ operator@ (a, b)`

- As a member function: `@a $\rightarrow$ a.operator@()`, `a @ b $\rightarrow$ a.operator@(b)`
  - The postfix `++` and `--` are special: They have a special `int` parameter to make them different from the prefix ones.
  - The arrow operator `->` is special: Although it looks like a binary operator in `ptr->mem`, it is unary and involves special rules.
    - You don't need to understand the exact rules for `->`.
- Avoid repetition.
- Avoid abuse of operator overloading.

## Summary

Type conversions

- Implicit vs explicit
- User-defined type conversions: either through a constructor or through a type conversion operator.
- To disable the implicit use of the user-defined type conversion: `explicit`
- Avoid abuse of type conversion operators.
- Conversion to `bool` has some special rules (*contextual conversion*).

# CS100 Lecture 20

Iterators and Algorithms

## Contents

- Iterators
- Algorithms

## Iterators

A generalized "pointer" used for accessing elements in different containers.

Every container has its iterator, whose type is `Container::iterator`.

e.g. `std::vector<int>::iterator`, `std::forward_list<std::string>::iterator`

- `auto` comes to our rescue!

## Iterators

For any container object `c`,

- `c.begin()` returns the iterator to the first element of `c`.
- `c.end()` returns the iterator to **the position following the last element** of `c` ("off-the-end", "past-the-end").

``

## Iterators

A pair of iterators (`b`, `e`) is often used to indicate a range `[b, e)`.

Such ranges are **left-inclusive**. Benefits:

- `e - b` is the **length (size)** of the range, i.e. the number of elements. There is no extra `+1` or `-1` in this expression.
- If `b == e`, the range is empty. In other words, to check whether the range is empty, we only need to do an equality test, which is easily supported by all kinds of iterators.

## Iterators

Basic operations, supported by almost all kinds of iterators:

- `*it`: returns a reference to the element that `it` refers to.
- `it->mem`: equivalent to `(*it).mem`.
- `++it`, `it++`: moves `it` one step forward, so that `it` refers to the "next" element.
  - `++it` returns a reference to `it`, while `it++` returns a copy of `it` before incrementation.
- `it1 == it2`: checks whether `it1` and `it2` refer to the same position in the container.
- `it1 != it2`: equivalent to `!(it1 == it2)`.

These are supported by the iterators of all sequence containers, as well as `std::string`.



---

## Iterators

Use the basic operations to traverse a sequence container:

```
void swapcase(std::string &str) {
    for (auto it = str.begin(); it != str.end(); ++it) {
        if (std::islower(*it))
            *it = std::toupper(*it);
        else if (std::isupper(*it))
            *it = std::tolower(*it);
    }
}

void print(const std::vector<int> &vec) {
    for (auto it = vec.begin(); it != vec.end(); ++it)
        std::cout << *it << ' ';
}
```

---

## Iterators

**Built-in pointers are also iterators:** They are the iterator for built-in arrays.

For an array `Type a[N]` :

- The "begin" iterator is `a` .
- The "end" (off-the-end) iterator is `a + N` .

The standard library functions `std::begin(c)` and `std::end(c)` (defined in `<iterator>` and many other header files):

- return `c.begin()` and `c.end()` if `c` is a container;
- return `c` and `c + N` if `c` is an array of length `N` .

---

## Range-for demystified

The range-based for loop

```
for (@declaration : container)
    @loop_body
```

is equivalent to

```
{
    auto b = std::begin(container);
    auto e = std::end(container);
    for (; b != e; ++b) {
        @declaration = *b;
        @loop_body
    }
}
```

---

## Iterators: dereferenceable

Like pointers, an iterator can be dereferenced ( `*it` ) only when it refers to an existing element. ("**dereferenceable**")

- `*v.end()` is undefined behavior.
- `++it` is undefined behavior if `it` is not dereferenceable. In other words, moving an iterator out of the range `[begin, off_the_end]` is undefined behavior.

---

## Iterators: invalidation

```
Type *storage = new Type[n];
Type *iter = storage;
delete[] storage;
// Now `iter` does not refer to any existing element.
```

Some operations on some containers will **invalidate** some iterators:

- make these iterators not refer to any existing element.

For example:

- `push_back(x)` on a `std::vector` may cause the reallocation of storage. All iterators obtained previously are invalidated.
- `pop_back()` on a `std::vector` will invalidate the iterators that points to the deleted element.

---

## Never use invalidated iterators or references!

---

```
void foo(std::vector<int> &vec) {  
    auto it = vec.begin();  
    while (some_condition(vec))  
        vec.push_back(*it++); // Undefined behavior.  
}
```

After several calls to `push_back`, `vec` may reallocate a larger chunk of memory to store its elements. This will invalidate all pointers, references and iterators that point to somewhere in the previous memory block.

---

## More operations on iterators

---

The iterators of containers that support `*it`, `it->mem`, `++it`, `it++`, `it1 == it2` and `it1 != it2` are [ForwardIterators](#).

[BidirectionalIterator](#): a `ForwardIterator` that can be moved in both directions

- supports `--it` and `it--`.

[RandomAccessIterator](#): a `BidirectionalIterator` that can be moved to any position in constant time.

- supports `it + n`, `n + it`, `it - n`, `it += n`, `it -= n` for an integer `n`.
- supports `it[n]`, equivalent to `*(it + n)`.
- supports `it1 - it2`, returns the **distance** of two iterators.
- supports `<`, `<=`, `>`, `>=`.

---

## More operations on iterators

---

The iterators of containers that support `*it`, `it->mem`, `++it`, `it++`, `it1 == it2` and `it1 != it2` are [ForwardIterators](#).

[BidirectionalIterator](#): a `ForwardIterator` that can be moved in both directions

- supports `--it` and `it--`.

[RandomAccessIterator](#): a `BidirectionalIterator` that can be moved to any position in constant time.

- supports `it + n`, `n + it`, `it - n`, `it += n`, `it -= n`, `it[n]`, `it1 - it2`, `<`, `<=`, `>`, `>=`.
- `std::string::iterator` and `std::vector<T>::iterator` are in this category.

Which category is the built-in pointer in?

---

## More operations on iterators

---

The iterators of containers that support `*it`, `it->mem`, `++it`, `it++`, `it1 == it2` and `it1 != it2` are [ForwardIterators](#).

[BidirectionalIterator](#): a `ForwardIterator` that can be moved in both directions

- supports `--it` and `it--`.

[RandomAccessIterator](#): a `BidirectionalIterator` that can be moved to any position in constant time.

- supports `it + n`, `n + it`, `it - n`, `it += n`, `it -= n`, `it[n]`, `it1 - it2`, `<`, `<=`, `>`, `>=`.
- `std::string::iterator` and `std::vector<T>::iterator` are in this category.

Which category is the built-in pointer in? - `RandomAccessIterator`.

---

## Initialization from iterator range

---

`std::string`, `std::vector`, as well as other standard library containers, support the initialization from an iterator range:

```
std::vector<char> v = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i'};  
std::vector v2(v.begin() + 2, v.end() - 3); // {'c', 'd', 'e', 'f'}  
std::string s(v.begin(), v.end()); // "abcdefghi"
```

---

## Algorithms

---

# Algorithms

Full list of standard library algorithms can be found [here](#).

No one can remember all of them, but some are quite commonly used.

## Algorithms: interfaces

**Parameters:** The STL algorithms accept pairs of iterators to represent "ranges":

```
int a[N], b[N]; std::vector<int> v;
std::sort(a, a + N);
std::sort(v.begin(), v.end());
std::copy(a, a + N, b); // copies elements in [a, a+N) to [b, b+N)
std::sort(v.begin(), v.begin() + 10); // only the first 10 elements are sorted.
```

Since C++20, `std::ranges::xxx` can be used, which has more modern interfaces

```
std::ranges::sort(a);
std::ranges::copy(a, b);
```

## Algorithms: interfaces

**Parameters:** The algorithms suffixed `_n` use a **beginning iterator** `begin` and an **integer** `n` to represent a range `[begin, begin + n)`.

Example: Use STL algorithms to rewrite the constructors of `Dynarray` :

```
Dynarray::Dynarray(const int *begin, const int *end)
: m_storage{new int[end - begin]}, m_length(end - begin) {
    std::copy(begin, end, m_storage);
}
Dynarray::Dynarray(const Dynarray &other)
: m_storage{new int[other.size()]}, m_length{other.size()} {
    std::copy_n(other.m_storage, other.size(), m_storage);
}
Dynarray::Dynarray(std::size_t n, int x = 0)
: m_storage{new int[n]}, m_length{n} {
    std::fill_n(m_storage, m_length, x);
}
```

## Algorithms: interfaces

**Return values:** "Position" is typically represented by an iterator. For example:

```
std::vector<int> v = someValues();
auto pos = std::find(v.begin(), v.end(), 42);
assert(*pos == 42);
auto maxPos = std::max_element(v.begin(), v.end());
```

- `pos` is an **iterator** pointing to the first occurrence of `42` in `v`.
- `maxPos` is an **iterator** pointing to the max element in `v`.

"Not found" / "No such element" is often indicated by returning `end`.

```
if (std::find(v.begin(), v.end(), something) != v.end()) {
    // ...
}
```

## if: new syntax in C++17

"Not found" / "No such element" is often indicated by returning `end`.

```
if (std::find(v.begin(), v.end(), something) != v.end()) { /* (*) */ }
```

If we want to use the returned iterator in (\*):

```
if (auto pos = std::find(v.begin(), v.end(), something); pos != v.end())
    std::cout << *pos << '\n';
```

The new syntax of `if` in C++17: `if (init_expr; condition)`.

- `init_expr` is just like the first part of the `for` statement.
- The scope of the variable declared in `init_expr` is within this `if` statement (containing the `else` clause, if present).

## Algorithms: requirements

An algorithm may have **requirements** on

- the iterator categories of the passed-in iterators, and
- the type of elements that the iterators point to.

Typically, `std::sort` requires *RandomAccessIterators*, while `std::copy` allows any *InputIterators*.

Typically, all algorithms that need to compare elements rely only upon `operator<` and `operator==` of the elements.

- You don't have to define all the six comparison operators of `X` in order to `sort` a `vector<X>`. `sort` only requires `operator<`.

## Algorithms

Since we pass **iterators** instead of **containers** to algorithms, **the standard library algorithms never modify the length of the containers**.

- STL algorithms never insert or delete elements in the containers (unless the iterator passed to them is some special *iterator adapter*).

For example: `std::copy` only **copies** elements, instead of inserting elements.

```
std::vector<int> a = someValues();
std::vector<int> b(a.size());
std::vector<int> c{};
std::copy(a.begin(), a.end(), b.begin()); // OK
std::copy(a.begin(), a.end(), c.begin()); // Undefined behavior!
```

## Some common algorithms (<algorithm>)

Non-modifying sequence operations:

- `count(begin, end, x)`, `find(begin, end, x)`, `find_end(begin, end, x)`, `find_first_of(begin, end, x)`, `search(begin, end, pattern_begin, pattern_end)`

Modifying sequence operations:

- `copy(begin, end, dest)`, `fill(begin, end, x)`, `reverse(begin, end)`, ...
- `unique(begin, end)`: drop duplicate elements.
  - requires the elements in the range `[begin, end)` to be **sorted** (in ascending order by default).
  - **It does not remove any elements!** Instead, it moves all the duplicated elements to the end of the sequence, and returns an iterator `pos`, so that `[begin, pos)` has no duplicate elements.

## Some common algorithms (<algorithm>)

Example: `unique`

```
std::vector v{1, 1, 2, 2, 2, 3, 5};
auto pos = std::unique(v.begin(), v.end());
// Now [v.begin(), pos) contains {1, 2, 3, 5}.
// [pos, v.end()) has the values {1, 2, 2}, but the exact order is not known.
v.erase(pos, v.end()); // Typical use with the container's 'erase' operation
// Now v becomes {1, 2, 3, 5}.
```

`unique` does not remove the duplicate elements! To remove them, use the container's `erase` operation.

## Some common algorithms (<algorithm>)

Partitioning, sorting and merging algorithms:

- `partition`, `is_partitioned`, `stable_partition`
- `sort`, `is_sorted`, `stable_sort`

- `nth_element`
- `merge`, `inplace_merge`

Binary search on sorted ranges:

- `lower_bound`, `upper_bound`, `binary_search`, `equal_range`

Heap algorithms:

- `is_heap`, `make_heap`, `push_heap`, `pop_heap`, `sort_heap`

Learn the underlying algorithms and data structures of these functions in CS101!

## Some common algorithms

Min/Max and comparison algorithms: ( `<algorithm>` )

- `min_element(begin, end)`, `max_element(begin, end)`, `minmax_element(begin, end)`
- `equal(begin1, end1, begin2)`, `equal(begin1, end1, begin2, end2)`
- `lexicographical_compare(begin1, end1, begin2, end2)`

Numeric operations: ( `<numeric>` )

- `accumulate(begin, end, initialValue)` : Sum of elements in `[begin, end)`, with initial value `initialValue`.
  - `accumulate(v.begin(), v.end(), 0)` returns the sum of elements in `v`.
- `inner_product(begin1, end1, begin2, initialValue)` : Inner product of two vectors  $\mathbf{a}^T \mathbf{b}$ , added with the initial value `initialValue`.

## Predicates

Consider the `Point2d` class:

```
struct Point2d {
    double x, y;
};
std::vector<Point2d> points = someValues();
```

Suppose we want to sort `points` in ascending order of the `x` coordinate.

- `std::sort` requires `operator<` in order to compare the elements,
- but it is not recommended to overload `operator<` here! (What if we want to sort some `Point2d` s in another way?)

(C++20 modern way: `std::ranges::sort(points, {}, &Point2d::x);`)

## Predicates

`std::sort` has another version that accepts another argument `cmp` :

```
bool cmp_by_x(const Point2d &lhs, const Point2d &rhs) {
    return lhs.x < rhs.x;
}
std::sort(points.begin(), points.end(), cmp_by_x);
```

`sort(begin, end, cmp)`

- `cmp` is a **Callable** object. When called, it accepts two arguments whose type is the same as the element type, and returns `bool`.
- `std::sort` will use `cmp(x, y)` instead of `x < y` to compare elements.
- After sorting, `cmp(v[i], v[i + 1])` is true for every `i`  $\in$  `[0, v.size()-1)`.

## Predicates

To sort numbers in reverse (descending) order:

```
bool greater_than(int a, int b) { return a > b; }
std::sort(v.begin(), v.end(), greater_than);
```

To sort them in ascending order of absolute values:

```
bool abs_less(int a, int b) { return std::abs(a) < std::abs(b); } // <cmath>
std::sort(v.begin(), v.end(), abs_less);
```

## Predicates

Many algorithms accept a Callable object. For example, `find_if(begin, end, pred)` finds the first element in `[begin, end)` such that `pred(element)` is true.

```
bool less_than_10(int x) {
    return x < 10;
}
std::vector<int> v = someValues();
auto pos = std::find_if(v.begin(), v.end(), less_than_10);
```

`for_each(begin, end, operation)` performs `operation(element)` for each element in the range `[begin, end)`.

```
void print_int(int x) { std::cout << x << ' '; }
std::for_each(v.begin(), v.end(), print_int);
```

## Predicates

Many algorithms accept a Callable object. For example, `find_if(begin, end, pred)` finds the first element in `[begin, end)` such that `pred(element)` is true.

What if we want to find the first element less than `k`, where `k` is determined at run-time?

## Predicates

What if we want to find the first element less than `k`, where `k` is determined at run-time?

```
struct LessThan {
    int k_;
    LessThan(int k) : k_{k} {}
    bool operator()(int x) const {
        return x < k_;
    }
};
auto pos = std::find_if(v.begin(), v.end(), LessThan(k));
```

- `LessThan(k)` constructs an object of type `LessThan`, with the member `k_` initialized to `k`.
- This object has an `operator()` overloaded: **the function-call operator**.
  - `LessThan(k)(x)` is equivalent to `LessThan(k).operator()(x)`, which is `x < k`.

## Function objects

Modern way:

```
struct LessThan {
    int k_; // No constructor is needed, and k_ is public.
    bool operator()(int x) const { return x < k_; }
};
auto pos = std::find_if(v.begin(), v.end(), LessThan{k}); // {} instead of ()
```

A **function object** (aka "functor") is an object `fo` with `operator()` overloaded.

- `fo(arg1, arg2, ...)` is equivalent to `fo.operator()(arg1, arg2, ...)`. Any number of arguments is allowed.

## Function objects

Exercise: use a function object to compare integers by their absolute values.

```
struct AbsCmp {
    bool operator()(int a, int b) const {
        return std::abs(a) < std::abs(b);
    }
};
std::sort(v.begin(), v.end(), AbsCmp{});
```

## Lambda expressions

Defining a function or a function object is not good enough:

- These functions or function objects are almost used only once, but
- too many lines of code is needed, and
- you have to add names to the global scope.

Is there a way to define an **unnamed**, immediate callable object?

---

## Lambda expressions

To sort by comparing absolute values:

```
std::sort(v.begin(), v.end(),
    [](int a, int b) -> bool { return std::abs(a) < std::abs(b); });
```

To sort in reverse order:

```
std::sort(v.begin(), v.end(),
    [](int a, int b) -> bool { return a > b; });
```

To find the first element less than `k`:

```
auto pos = std::find_if(v.begin(), v.end(),
    [k](int x) -> bool { return x < k; });
```

---

## Lambda expressions

The return type can be omitted and deduced by the compiler.

```
std::sort(v.begin(), v.end(),
    [](int a, int b) { return std::abs(a) < std::abs(b); });
```

```
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

```
auto pos = std::find_if(v.begin(), v.end(), [k](int x) { return x < k; });
```

---

## Lambda expressions

A lambda expression has the following syntax:

```
[capture_list](params) -> return_type { function_body }
```

The compiler will generate a function object according to it.

```
int k = 42;
auto f = [k](int x) -> bool { return x < k; };
bool b1 = f(10); // true
bool b2 = f(100); // false
```

---

## Lambda expressions

```
[capture_list](params) -> return_type { function_body }
```

It is allowed to write complex statements in `function_body`, just as in a function.

```

struct Point2d { double x, y; };
std::vector<Point2d> points = somePoints();
// prints the l2-norm of every point
std::for_each(points.begin(), points.end(),
    [](const Point2d &p) {
        auto norm = std::sqrt(p.x * p.x + p.y * p.y);
        std::cout << norm << std::endl;
    });

```

## Lambda expressions: capture

To capture more variables:

```

auto pos = std::find_if(v.begin(), v.end(),
    [lower, upper](int x) { return lower <= x && x <= upper;});

```

To capture by reference (so that copy is avoided)

```

std::string str = someString();
std::vector<std::string> wordList;
// finds the first string that is lexicographically greater than `str`,
// but shorter than `str`.
auto pos = std::find_if(wordList.begin(), wordList.end(),
    [&str](const std::string &s) { return s > str && s.size() < str.size();});

```

Here `&str` indicates that `str` is captured by reference. `&` here is not the address-of operator!

## More on lambda expressions

- *C++ Primer* Section 10.3
- *Effective Modern C++* Chapter 6 (Item 31-34)

Note that *C++ Primer (5th edition)* is based on C++11 and *Effective Modern C++* is based on C++14. Lambda expressions are evolving at a very fast pace in modern C++, with many new things added and many limitations removed.

More fancy ways of writing lambda expressions are not covered in CS100.

## Back to algorithms

So many things in the algorithm library! How can we remember them?

- Remember the **conventions**:
  - No insertion/deletion of elements
  - Iterator range `[begin, end)`
  - Functions named with the suffix `_n` uses `[begin, begin + n)`
  - Pass functions, function objects, and lambdas for customized operations
  - Functions named with the suffix `_if` requires a boolean predicate
- Remember the common ones: `copy`, `find`, `for_each`, `sort`, ...
- Look them up in [cppreference](#) before use.

## Summary

Iterators

- A generalized "pointer" used for accessing elements in different containers.
- Iterator range: a left-inclusive interval `[b, e)`.
- `c.begin()`, `c.end()`
- Basic operations: `*it`, `it->mem`, `++it`, `it++`, `it1 == it2`, `it1 != it2`.
- Range-based `for` loops are in fact traversal using iterators.
- More operations: BidirectionalIterator supports `it--` and `--it`. RandomAccessIterator supports all pointer arithmetics.
- Initialization of standard library containers from an iterator range.

## Summary

Algorithms

- Normal functions accept iterator range `[b, e)`. Functions with `_n` accept an iterator and an integer, representing the range `[begin, begin + n)`.



- Position is represented by an iterator.
- STL algorithms never insert or delete elements in the containers.
- Some algorithms accept a predicate argument, which is a callable object. It can be a function, a pointer to function, an object of some type that has an overloaded `operator()`, or a lambda.
- Lambda: `[capture_list][params] -> return_type { function_body }`

# CS100 Lecture 21

Inheritance and Polymorphism I

## Contents

- Inheritance
- Dynamic binding and polymorphism

## Inheritance

### Example: An item for sale

```
class Item {
    std::string m_name;
    double m_price = 0.0;
public:
    Item() = default;
    Item(const std::string &name, double price)
        : m_name(name), m_price(price) {}
    const auto &getName() const { return m_name; }
    auto netPrice(int cnt) const {
        return cnt * m_price;
    }
};
```

### Defining a subclass

A discounted item **is an** item, and has more information:

- `std::size_t m_minQuantity;`
- `double m_discount;`

The net price for such an item is

$$\text{netPrice}(n) = \begin{cases} n \cdot \text{price}, & \text{if } n < \text{minQuantity}, \\ n \cdot \text{discount} \cdot \text{price}, & \text{otherwise.} \end{cases}$$

### Defining a subclass

Use **inheritance** to model the "is-a" relationship:

- A discounted item **is an** item.

```
class DiscountedItem : public Item {
    int m_minQuantity = 0;
    double m_discount = 1.0;
public:
    // constructors
    // netPrice
};
```

### protected members

A `protected` member is private, except that it is accessible in subclasses.

- `m_price` needs to be `protected`, of course.
- Should `m_name` be `protected` or `private`?
  - `private` is ok if the subclass does not modify it. It is accessible through the public `getName` interface.
  - `protected` is also reasonable.

---

## protected members

---

```
class Item {
    std::string m_name;
protected:
    double m_price = 0.0;
public:
    Item() = default;
    Item(const std::string &name, double price)
        : m_name(name), m_price(price) {}
    const auto &getName() const { return m_name; }
    auto netPrice(int cnt) const {
        return cnt * m_price;
    }
};
```

---

## Inheritance

---

By defining `DiscountedItem` to be a subclass of `Item`, every `DiscountedItem` object contains a subobject of type `Item`.

- Every data member and member function, except the ctors and dtors, is inherited, **no matter what access level they have**.

What can be inferred from this?

---

## Inheritance

---

By defining `DiscountedItem` to be a subclass of `Item`, every `DiscountedItem` object contains a subobject of type `Item`.

- Every data member and member function, except the ctors and dtors, is inherited, **no matter what access level they have**.

What can be inferred from this?

- A constructor of `DiscountedItem` must first initialize the base class subobject by calling a constructor of `Item`'s.
- The destructor of `DiscountedItem` must call the destructor of `Item` after having destroyed its own members (`m_minQuantity` and `m_discount`).
- `sizeof(Derived) >= sizeof(Base)`

---

## Inheritance

---

Key points of inheritance:

- Every object of the derived class (subclass) contains a base class subobject.
- Inheritance should not break the encapsulation of the base class.
  - e.g. To initialize the base class subobject, **we must call a constructor of the base class**. It is not allowed to initialize data members of the base class subobject directly.

---

## Constructor of DiscountedItem

---

```
class DiscountedItem : public Item {
    int m_minQuantity = 0;
    double m_discount = 1.0;
public:
    DiscountedItem(const std::string &name, double price,
                  int minQ, double disc)
        : Item(name, price), m_minQuantity(minQ), m_discount(disc) {}
};
```

It is not allowed to write this:

```
DiscountedItem(const std::string &name, double price,
              int minQ, double disc)
    : m_name(name), m_price(price), m_minQuantity(minQ), m_discount(disc) {}
```

---

## Constructor of derived classes

---

Before the initialization of the derived class's own data members, the base class subobject **must** be initialized by having one of its ctors called.

- What if we don't call the base class's ctor explicitly?

```
DiscountedItem(...)
: /* ctor of Item is not called */ m_minQuantity(minQ), m_discount(d) {}
```

## Constructor of derived classes

Before the initialization of the derived class's own data members, the base class subobject **must** be initialized by having one of its ctors called.

- What if we don't call the base class's ctor explicitly?
  - The default constructor of the base class is called.
  - If the base class is not default-constructible, an error.
- What does this constructor do?

```
DiscountedItem() = default;
```

## Constructor of derived classes

Before the initialization of the derived class's own data members, the base class subobject **must** be initialized by having one of its ctors called.

- What if we don't call the base class's ctor explicitly?
  - The default constructor of the base class is called.
  - If the base class is not default-constructible, an error.
- What does this constructor do?

```
DiscountedItem() = default;
```

- Calls `Item::Item()` to default-initialize the base class subobject before initializing `m_minQuantity` and `m_discount`.

## Constructor of derived classes

In the following code, does the constructor of `DiscountedItem` compile?

```
class Item {
protected:
    std::string m_name;
    double m_price;
public:
    Item(const std::string &name, double p) : m_name(name), m_price(p) {}
};
class DiscountedItem : public Item {
    int m_minQuantity;
    double m_discount;
public:
    DiscountedItem(const std::string &name, double p, int mq, double disc) {
        m_name = name; m_price = p; m_minQuantity = mq; m_discount = disc;
    }
};
```

## Constructor of derived classes

In the following code, does the constructor of `DiscountedItem` compile?

```
class Item {
    // ...
public:
    // Since `Item` has a user-declared constructor, it does not have
    // a default constructor.
    Item(const std::string &name, double p) : m_name(name), m_price(p) {}
};
class DiscountedItem : public Item {
    // ...
public:
    DiscountedItem(const std::string &name, double p, int mq, double disc)
    // Before entering the function body, `Item::Item()` is called --> Error!
    { /* ... */ }
};
```

[Best practice] Use constructor initializer lists whenever possible.

# Dynamic binding

## Upcasting

If `D` is a subclass of `B`:

- A `B*` can point to a `D`, and
- A `B&` can be bound to a `D`.

```
DiscountedItem di = someValue();
Item &ir = di; // correct
Item *ip = &di; // correct
```

Reason: The **is-a** relationship! A `D` **is a** `B`.

But on such references or pointers, only the members of `B` can be accessed.

## Upcasting: Example

```
void printItemName(const Item &item) {
    std::cout << "Name: " << item.getName() << std::endl;
}
DiscountedItem di("A", 10, 2, 0.8);
Item i("B", 15);
printItemName(i); // "Name: B"
printItemName(di); // "Name: A"
```

`const Item &item` can be bound to either an `Item` or a `DiscountedItem`.

## Static type and dynamic type

- **static type** of an expression: The type known at compile-time.
- **dynamic type** of an expression: The real type of the object that the expression is representing. This is known at run-time.

```
void printItemName(const Item &item) {
    std::cout << "Name: " << item.getName() << std::endl;
}
```

The static type of the expression `item` is `const Item`, but its dynamic type is not known until run-time. (It may be `const Item` or `const DiscountedItem`.)

## virtual functions

`Item` and `DiscountedItem` have different ways of computing the net price.

```
void printItemInfo(const Item &item) {
    std::cout << "Name: " << item.getName()
              << ", price: " << item.netPrice(1) << std::endl;
}
```

- Which `netPrice` should be called?
- How do we define two different `netPrice`s?

## virtual functions

```
class Item {
public:
    virtual double netPrice(int cnt) const {
        return m_price * cnt;
    }
    // other members
};
class DiscountedItem : public Item {
public:
```

```
double netPrice(int cnt) const override {
    return cnt < m_minQuantity ? cnt * m_price : cnt * m_price * m_discount;
}
// other members
};
```

Note: `auto` cannot be used to deduce the return type of `virtual` functions.

## Dynamic binding

```
void printItemInfo(const Item &item) {
    std::cout << "Name: " << item.getName()
               << ", price: " << item.netPrice(1) << std::endl;
}
```

The dynamic type of `item` is determined at run-time.

Since `netPrice` is a `virtual` function, which version is called is also determined at run-time:

- If the dynamic type of `item` is `Item`, it calls `Item::netPrice`.
- If the dynamic type of `item` is `DiscountedItem`, it calls `DiscountedItem::netPrice`.

late binding, or dynamic binding

## virtual-override

To **override** (覆盖/覆写) a `virtual` function,

- The function parameter list must be the same as that of the base class's version.
- The return type should be **identical to** (or *covariant with*) that of the corresponding function in the base class.
  - We will talk about "covariant with" in later lectures or recitations.
- **The `const` ness should be the same!**

To make sure you are truly overriding the `virtual` function (instead of making a overloaded version), use the `override` keyword.

\* **Not to be confused with "overloading" (重载)** .

## virtual-override

An overriding function is also `virtual`, even if not explicitly declared.

```
class DiscountedItem : public Item {
    virtual double netPrice(int cnt) const override; // correct, explicitly virtual
};
class DiscountedItem : public Item {
    double netPrice(int cnt) const; // also correct, but not recommended
};
```

The `override` keyword lets the compiler check and report if the function is not truly overriding.

**[Best practice]** To override a virtual function, write the `override` keyword explicitly. The `virtual` keyword can be omitted.

## virtual destructors

```
Item *ip = nullptr;
if (some_condition)
    ip = new Item(/* ... */);
else
    ip = new DiscountedItem(/* ... */);
// ...
delete ip;
```

Whose destructor should be called?

- Only looking at the static type of `*ip` is not enough.

## virtual destructors

```
Item *ip = nullptr;
if (some_condition)
    ip = new Item(/* ... */);
else
    ip = new DiscountedItem(/* ... */);
// ...
delete ip;
```

Whose destructor should be called? - It needs to be determined at run-time!

- To use dynamic binding correctly, you almost always need a `virtual` destructor.

## virtual destructors

```
Item *ip = nullptr;
if (some_condition)
    ip = new Item(/* ... */);
else
    ip = new DiscountedItem(/* ... */);
// ...
delete ip;
```

- The implicitly-defined (compiler-generated) destructor is **non-virtual**, but we can explicitly require a `virtual` one:

```
virtual ~Item() = default;
```

- If the dtor of the base class is `virtual`, the compiler-generated dtor for the derived class is also `virtual`.

## (Almost) completed Item and DiscountedItem

```
class Item {
    std::string m_name;

protected:
    double m_price = 0.0;

public:
    Item() = default;
    Item(const std::string &name, double price) : m_name(name), m_price(price) {}
    const auto &getName() const { return name; }
    virtual double net_price(int n) const {
        return n * price;
    }
    virtual ~Item() = default;
};
```

## (Almost) completed Item and DiscountedItem

```
class DiscountedItem : public Item {
    int m_minQuantity = 0;
    double m_discount = 1.0;

public:
    DiscountedItem(const std::string &name, double price,
                  int minQ, double disc)
        : Item(name, price), m_minQuantity(minQ), m_discount(disc) {}
    double netPrice(int cnt) const override {
        return cnt < m_minQuantity ? cnt * m_price : cnt * m_price * m_discount;
    }
};
```

## Usage with smart pointers

Smart pointers are implemented by wrapping the raw pointers, so they can also be used for dynamic binding.

```
std::vector<std::shared_ptr<Item>> myItems;
for (auto i = 0; i != n; ++i) {
    if (someCondition) {
        myItems.push_back(std::make_shared<Item>(someParams));
    } else {
        myItems.push_back(std::make_shared<DiscountedItem>(someParams));
    }
}
```

A `std::unique_ptr<Derived>` can be implicitly converted to a `std::unique_ptr<Base>`.

A `std::shared_ptr<Derived>` can be implicitly converted to a `std::shared_ptr<Base>`.

## Copy-control

Remember to copy/move the base subobject! One possible way:

```
class Derived : public Base {
public:
    Derived(const Derived &other)
        : Base(other), /* Derived's own members */ { /* ... */ }
    Derived &operator=(const Derived &other) {
        Base::operator=(other); // call Base's operator= explicitly
        // copy Derived's own members
        return *this;
    }
    // ...
};
```

Why `Base(other)` and `Base::operator=(other)` work?

- The parameter type is `const Base &`, which can be bound to a `Derived` object.

## Synthesized copy-control members

Guess!

- What are the behaviors of the compiler-generated copy-control members?
- In what cases will they be `deleted`?

## Synthesized copy-control members

Remember that the base class's subobject is always handled first.

These rules are quite natural:

- What are the behaviors of the compiler-generated copy-control members?
  - First, it calls the base class's corresponding copy-control member.
  - Then, it performs the corresponding operation on the derived class's own data members.
- In what cases will they be `deleted`?
  - If the base class's corresponding copy-control member is not accessible (e.g. non-existent, or `private`),
  - or if any of the data members' corresponding copy-control member is not accessible.

## Slicing

Dynamic binding only happens on references or pointers to base class.

```
DiscountedItem di("A", 10, 2, 0.8);
Item i = di; // what happens?
auto x = i.netPrice(3); // which netPrice?
```

## Slicing

Dynamic binding only happens on references or pointers to base class.

```
DiscountedItem di("A", 10, 2, 0.8);
Item i = di; // what happens?
auto x = i.netPrice(3); // which netPrice?
```

`Item i = di;` calls the **copy constructor of `Item`**

- but `Item`'s copy constructor handles only the base part.
- So `DiscountedItem`'s own members are **ignored**, or "**sliced down**".
- `i.netPrice(3)` calls `Item::netPrice`.

## Downcasting

```
Base *bp = new Derived{};
```

If we only have a `Base` pointer, but we are quite sure that it points to a `Derived` object

- Accessing the members of `Derived` through `bp` is not allowed.
- How can we perform a "**downcasting**"?

## Polymorphic class

A class is said to be **polymorphic** if it has (declares or inherits) at least one virtual function.

- Either a `virtual` normal member function or a `virtual` dtor is ok.

If a class is polymorphic, all classes derived from it are polymorphic.

- There is no way to "refuse" to inherit any member functions, so `virtual` member functions must be inherited.
- The dtor must be `virtual` if the dtor of the base class is `virtual`.

## Downcasting: For polymorphic class only

```
dynamic_cast<Target>(expr).
```

```
Base *bp = new Derived{};
Derived *dp = dynamic_cast<Derived *>(bp);
Derived &dr = dynamic_cast<Derived &>(*bp);
```

- `Target` must be a **reference** or a **pointer** type.
- `dynamic_cast` will perform **runtime type identification (RTTI)** to check the dynamic type of the expression.
  - If the dynamic type is `Derived`, or a derived class (direct or indirect) of `Derived`, the downcasting succeeds.
  - Otherwise, the downcasting fails. If `Target` is a pointer, returns a null pointer. If `Target` is a reference, throws an exception `std::bad_cast`.

## `dynamic_cast` can be very slow

`dynamic_cast` performs a runtime **check** to see whether the downcasting should succeed, which uses runtime type information.

This is **much slower** than other types of casting, e.g. `const_cast`, or arithmetic conversions.

**[Best practice]** Avoid `dynamic_cast` whenever possible.

## Guaranteed successful downcasting: Use `static_cast`.

If the downcasting is guaranteed to be successful, you may use `static_cast`

```
auto dp = static_cast<Derived *>(bp); // quicker than dynamic_cast,
// but performs no checks. If the dynamic type is not Derived, UB.
```

## Avoiding `dynamic_cast`

Typical abuse of `dynamic_cast`:



```
struct A {
    virtual ~A() {}
};
struct B : A {};
struct C : A {};
```

```
std::string getType(const A *ap) {
    if (dynamic_cast<const B *>(ap))
        return "B";
    else if (dynamic_cast<const C *>(ap))
        return "C";
    else
        return "A";
}
```

## Avoiding `dynamic_cast`

Use a group of `virtual` functions!

```
struct A {
    virtual ~A() {}
    virtual std::string name() const {
        return "A";
    }
};
struct B : A {
    std::string name()const override{
        return "B";
    }
};
struct C : A {
    std::string name()const override{
        return "C";
    }
};
```

```
auto getType(const A *ap) {
    return ap->name();
}
```

## Summary

Inheritance

- Every object of type `Derived` contains a subobject of type `Base`.
  - Every member of `Base` is inherited, no matter whether it is accessible or not.
- Inheritance should not break the base class's encapsulation.
  - The access control of inherited members is not changed.
  - Every constructor of `Derived` calls a constructor of `Base` to initialize the base class subobject **before** initializing its own data members.
  - The destructor of `Derived` calls the destructor of `Base` to destroy the base class subobject **after** destroying its own data members.

## Summary

Dynamic binding

- Upcasting: A pointer, reference or smart pointer to `Base` can be bound to an object of type `Derived`.
  - static type and dynamic type
- `virtual` functions: A function that can be overridden by derived classes.
  - The base class and the derived class can provide different versions of this function.
- Dynamic (late) binding
  - A call to a virtual function on a pointer or reference to `Base` will actually call the corresponding version of that function according to the dynamic type.
- Avoid downcasting if possible.

## CS100 Lecture 22

Inheritance and Polymorphism II

## Contents

- Abstract base class
- More on the "is-a" relationship (*Effective C++* Item 32)
- Inheritance of interface vs inheritance of implementation (*Effective C++* Item 34)

## Abstract base class

### Shapes

Define different shapes: Rectangle, Triangle, Circle, ...

Suppose we want to draw things like this:

```
void drawThings(ScreenHandle &screen,
                const std::vector<std::shared_ptr<Shape>> &shapes) {
    for (const auto &shape : shapes)
        shape->draw(screen);
}
```

and print information:

```
void printShapeInfo(const Shape &shape) {
    std::cout << "Area: " << shape.area()
               << "Perimeter: " << shape.perimeter() << std::endl;
}
```

### Shapes

Define a base class `Shape` and let other shapes inherit it.

```
class Shape {
public:
    Shape() = default;
    virtual void draw(ScreenHandle &screen) const;
    virtual double area() const;
    virtual double perimeter() const;
    virtual ~Shape() = default;
};
```

Different shapes should define their own `draw`, `area` and `perimeter`, so these functions should be `virtual`.

### Shapes

```
class Rectangle : public Shape {
    Point2d mTopLeft, mBottomRight;

public:
    Rectangle(const Point2d &t1, const Point2d &br)
        : mTopLeft(t1), mBottomRight(br) {} // Base is default-initialized
    void draw(ScreenHandle &screen) const override { /* ... */ }
    double area() const override {
        return (mBottomRight.x - mTopLeft.x) * (mBottomRight.y - mTopLeft.y);
    }
    double perimeter() const override {
        return 2 * (mBottomRight.x - mTopLeft.x + mBottomRight.y - mTopLeft.y);
    }
};
```

### Pure `virtual` functions

How should we define `Shape::draw`, `Shape::area` and `Shape::perimeter`?

- For the general concept "Shape", there is no way to determine the behaviors of these functions.

## Pure virtual functions

How should we define `Shape::draw`, `Shape::area` and `Shape::perimeter`?

- For the general concept "Shape", there is no way to determine the behaviors of these functions.
- Direct call to `Shape::draw`, `Shape::area` and `Shape::perimeter` should be forbidden.
- We shouldn't even allow an object of type `Shape` to be instantiated! The class `Shape` is only used to **define the concept "Shape" and required interfaces**.

## Pure virtual functions

If a `virtual` function does not have a reasonable definition in the base class, it should be declared as **pure virtual** by writing `=0`.

```
class Shape {
public:
    virtual void draw(ScreenHandle &) const = 0;
    virtual double area() const = 0;
    virtual double perimeter() const = 0;
    virtual ~Shape() = default;
};
```

Any class that has a **pure virtual function** is an **abstract class**. Pure `virtual` functions (usually) cannot be called `{}^{\textcolor{red}{1}}{}`, and abstract classes cannot be instantiated.

## Pure virtual functions and abstract classes

Any class that has a **pure virtual function** is an **abstract class**. Pure `virtual` functions (usually) cannot be called `{}^{\textcolor{red}{1}}{}`, and abstract classes cannot be instantiated.

```
Shape shape; // Error.
Shape *p = new Shape; // Error.
auto sp = std::make_shared<Shape>(); // Error.
std::shared_ptr<Shape> sp2 = std::make_shared<Rectangle>(p1, p2); // OK.
```

We can define pointer or reference to an abstract class, but never an object of that type!

## Pure virtual functions and abstract classes

An impure `virtual` function **must be defined**. Otherwise, the compiler will fail to generate necessary runtime information (the virtual table), which leads to an error.

```
class X {
    virtual void foo(); // Declaration, without a definition
    // Even if `foo` is not used, this will lead to an error.
};
```

Linkage error:

```
/usr/bin/ld: /tmp/ccV9TNfM.o: in function `main':
a.cpp:(.text+0x1e): undefined reference to `vtable for X'
```

## Make the interface robust, not error-prone.

Is this good?

```
class Shape {
public:
    virtual double area() const {
        return 0;
    }
};
```

What about this?

```
class Shape {
public:
    virtual double area() const {
        throw std::logic_error{"area() called on Shape!"};
    }
};
```

## Make the interface robust, not error-prone.

```
class Shape {
public:
    virtual double area() const {
        return 0;
    }
};
```

If `Shape::area` is called accidentally, the error will happen *silently*!

## Make the interface robust, not error-prone.

```
class Shape {
public:
    virtual double area() const {
        throw std::logic_error{"area() called on Shape!"};
    }
};
```

If `Shape::area` is called accidentally, an exception will be raised.

However, **a good design should make errors fail to compile.**

**[Best practice]** If an error can be caught in compile-time, don't leave it until run-time.

## Polymorphism (多态)

Polymorphism: The provision of a single interface to entities of different types, or the use of a single symbol to represent multiple different types.

- Run-time polymorphism: Achieved via **dynamic binding**.
- Compile-time polymorphism: Achieved via **function overloading, templates, concepts (since C++20)**, etc.

Run-time polymorphism:

```
struct Shape {
    virtual void draw() const = 0;
};
void drawStuff(const Shape &s) {
    s.draw();
}
```

Compile-time polymorphism:

```
template <typename T>
concept Shape = requires(const T x) {
    x.draw();
};
void drawStuff(Shape const auto &s) {
    s.draw();
}
```

## More on the "is-a" relationship

*Effective C++* Item 32

## Public inheritance: The "is-a" relationship

By writing that class `D` publicly inherits from class `B`, you are telling the compiler (as well as human readers of your code) that

- Every object of type `D` *is* also *an* object of type `B`, but not vice versa.
- `B` represents a **more general concept** than `D`, and that `D` represents a **more specialized concept** than `B`.

More specifically, you are asserting that **anywhere an object of type `B` can be used, an object of type `D` can be used just as well.**

- On the other hand, if you need an object of type `D`, an object of type `B` won't do.

## Example: Every student *is a* person.

```
class Person { /* ... */ };
class Student : public Person { /* ... */ };
```

- Every student *is a* person, but not every person is a student.
- Anything that is true of a person is also true of a student:
  - A person has a date of birth, so does a student.
- Something is true of a student, but not true of people in general.
  - A student is enrolled in a particular school, but a person may not.

The notion of a person is **more general** than is that of a student; a student is **a specialized type** of person.

## Example: Every student *is a* person.

The **is-a** relationship: Anywhere an object of type `Person` can be used, an object of type `Student` can be used just as well, **but not vice versa**.

```
void eat(const Person &p);    // Anyone can eat.
void study(const Student &s); // Only students study.
Person p;
Student s;
eat(p);    // Fine. `p` is a person.
eat(s);    // Fine. `s` is a student, and a student is a person.
study(s);  // Fine.
study(p);  // Error! `p` isn't a student.
```

## Your intuition can mislead you.

- A penguin **is a** bird.
- A bird can fly.

If we naively try to express this in C++, our effort yields:

```
class Bird {
public:
    virtual void fly();    // Birds can fly.
    // ...
};
class Penguin : public Bird { // A penguin is a bird.
    // ...
};
```

```
Penguin p;
p.fly();    // Oh no!! Penguins cannot fly, but this code compiles!
```

## No. Not every bird can fly.

*In general*, birds have the ability to fly.

- Strictly speaking, there are several types of non-flying birds.

Maybe the following hierarchy models the reality much better?

```
class Bird { /* ... */ };
class FlyingBird : public Bird {
    virtual void fly();
};
class Penguin : public Bird { // Not FlyingBird
    // ...
};
```

---

## No. Not every bird can fly.

Maybe the following hierarchy models the reality much better?

```
class Bird { /* ... */ };
class FlyingBird : public Bird {
    virtual void fly();
};
class Penguin : public Bird {    // Not FlyingBird
    // ...
};
```

- **Not necessarily.** If your application has much to do with beaks and wings, and nothing to do with flying, the original two-class hierarchy might be satisfactory.
- **There is no one ideal design for every software.** The best design depends on what the system is expected to do.

---

## What about report a runtime error?

```
void report_error(const std::string &msg); // defined elsewhere
class Penguin : public Bird {
public:
    virtual void fly() {
        report_error("Attempt to make a penguin fly!");
    }
};
```

---

## What about report a runtime error?

```
void report_error(const std::string &msg); // defined elsewhere
class Penguin : public Bird {
public:
    virtual void fly() { report_error("Attempt to make a penguin fly!"); }
};
```

**No.** This does not say "Penguins can't fly." This says "**Penguins can fly, but it is an error for them to actually try to do it.**"

To actually express the constraint "Penguins can't fly", you should prevent the attempt from **compiling**.

```
Penguin p;
p.fly(); // This should not compile.
```

[Best practice] Good interfaces prevent invalid code from **compiling**.

---

## Example: A square *is a* rectangle.

Should class `Square` publicly inherit from class `Rectangle`?

---

## Example: A square *is a* rectangle.

Consider this code.

```
class Rectangle {
public:
    virtual void setHeight(int newHeight);
    virtual void setwidth(int newwidth);
    virtual int getHeight() const;
    virtual int getWidth() const;
    // ...
};
void makeBigger(Rectangle &r) {
    r.setwidth(r.getWidth() + 10);
}
```

```
class Square : public Rectangle {
    // A square is a rectangle,
    // where height == width.
    // ...
};

Square s(10); // A 10x10 square.
makeBigger(s); // Oh no!
```

## Is this really an "is-a" relationship?

We said before that the "is-a" relationship means that **anywhere an object of type `B` can be used, an object of type `D` can be used just as well.**

However, something applicable to a rectangle is not applicable to a square!

**Conclusion: Public inheritance means "is-a". Everything that applies to base classes must also apply to derived classes, because every derived class object is a base class object.**

## Inheritance of interface vs inheritance of implementation

*Effective C++ Item 34*

### Example: Airplanes for XYZ Airlines.

Suppose XYZ has only two kinds of planes: the Model A and the Model B, and both are flown in exactly the same way.

```
class Airplane {
public:
    virtual void fly(const Airport &destination) {
        // Default code for flying an airplane to the given destination.
    }
};

class ModelA : public Airplane { /* ... */ };
class ModelB : public Airplane { /* ... */ };
```

- `Airplane::fly` is declared `virtual` because *in principle*, different airplanes should be flown in different ways.
- `Airplane::fly` is defined, to avoid copy-and-pasting code in the `ModelA` and `ModelB` classes.

### Example: Airplanes for XYZ Airlines.

Now suppose that XYZ decides to acquire a new type of airplane, the Model C, **which is flown differently from the Model A and the Model B.**

XYZ's programmers add the class `ModelC` to the hierarchy, but forget to redefine the `fly` function!

```
class ModelC : public Airplane {
    // `fly` is not overridden.
    // ...
};
```

This surely leads to a disaster:

```
auto pc = std::make_unique<ModelC>();
pc->fly(PVG); // No! Attempts to fly Model C in the Model A/B way!
```

## Impure virtual function: Interface + default implementation

The problem here is not that `Airplane::fly` has default behavior, but that `ModelC` was allowed to inherit that behavior **without explicitly saying that it wanted to.**

**\* By defining an impure virtual function, we have the derived class inherit a function *interface as well as a default implementation*.**

- Interface: Every class inheriting from `Airplane` can `fly`.
- Default implementation: If `ModelC` does not override `Airplane::fly`, it will have the inherited implementation automatically.

---

## Separate default implementation from interface

To sever the connection between the *interface* of the virtual function and its *default implementation*:

```
class Airplane {
public:
    virtual void fly(const Airport &destination) = 0; // pure virtual
    // ...
protected:
    void defaultFly(const Airport &destination) {
        // Default code for flying an airplane to the given destination.
    }
};
```

- The pure virtual function `fly` provides the **interface**: Every derived class can `fly`.
- The **default implementation** is written in `defaultFly`.

---

## Separate default implementation from interface

If `ModelA` and `ModelB` want to adopt the default way of flying, they simply make a call to `defaultFly`.

```
class ModelA : public Airplane {
public:
    virtual void fly(const Airport &destination) {
        defaultFly(destination);
    }
    // ...
};
class ModelB : public Airplane {
public:
    virtual void fly(const Airport &destination) {
        defaultFly(destination);
    }
    // ...
};
```

---

## Separate default implementation from interface

For `ModelC`:

- Since `Airplane::fly` is pure virtual, `ModelC` must define its own version of `fly`.
- If it **does** want to use the default implementation, **it must say it explicitly** by making a call to `defaultFly`.

```
class ModelC : public Airplane {
public:
    virtual void fly(const Airport &destination) {
        // The "Model C way" of flying.
        // Without the definition of this function, `ModelC` remains abstract,
        // which does not compile if we create an object of such type.
    }
};
```

---

## Still not satisfactory?

Some people object to the idea of having separate functions for providing the interface and the default implementation, such as `fly` and `defaultFly` above.

- For one thing, it pollutes the class namespace with closely related function names.
  - This really matters, especially in complicated projects. Extra mental effort might be required to distinguish the meaning of overly similar names.

Read the rest part of *Effective C++* Item 34 for another solution to this problem.

---

## Inheritance of interface vs inheritance of implementation



We have come to the conclusion that

- Pure virtual functions specify **inheritance of interface** only.
- Simple (impure) virtual functions specify **inheritance of interface + a default implementation**.
  - The default implementation can be overridden.

Moreover, non-virtual functions specify **inheritance of interface + a mandatory implementation**.

Note: In public inheritance, *interfaces are always inherited*.

---

## Summary

Pure virtual function and abstract class

- A pure virtual function is a virtual function declared `= 0`.
  - Call to a pure virtual function is not allowed. `{}^{\textcolor{red}{1}}{}`
  - Pure virtual functions define the interfaces and force the derived classes to override it.
- A class that has a pure virtual function is an abstract class.
  - We cannot create an object of an abstract class type.
  - Abstract classes are often used to represent abstract, general concepts.

---

## Summary

Public inheritance models the "is-a" relationship.

- Everything that applies to base classes must also apply to derived classes.
- The "Birds can fly, and a penguin is a bird" example.
- The "A square is a rectangle" example.

---

## Summary

Inheritance of interface vs inheritance of implementation

- In public inheritance, interfaces are always inherited.
- Pure virtual functions: inheritance of **interface** only.
- Simple (impure) virtual functions: inheritance of **interface + a default implementation**.
  - The default implementation can be overridden.
- non-virtual functions: inheritance of **interface + a mandatory implementation**.

---

## Notes

`{}^{\textcolor{red}{1}}{}` A pure virtual function can have a definition. In that case, it can be called via the syntax `ClassName::functionName(args)`, not via a virtual function call (dynamic binding).

In some cases, we want a class to be made abstract, but it does not have any pure virtual function. A possible workaround is to declare the destructor to be pure virtual, and then provide a definition for it:

```
struct Foo {  
    virtual ~Foo() = 0;  
};  
Foo::~~Foo() = default; // Provide a definition outside the class.
```

The "another solution" mentioned in page 36 is also related to this.